

Otomachi Una Library

接口参考手册

Junlin Ye

2026-09-29

按当前源码核对的 80 份竞赛模板接口参考。先看用途与示例，再按接口名查参数、返回值和边界；同类重载放在一起说明。

组织方式参考 `cppreference` 的声明、行为、复杂度与示例分层，具体契约以本库当前实现为准。

每节对应一个实际源码文件。使用底稿 `basic/template.cpp`，按依赖顺序粘贴所需模板。不要把全库同时编译进一份程序。

范围：当前模板的公开调用、运算符、结果字段及必要的改写入口。构建内部的递归步骤、缓存和底层算术辅助不作为独立 API；名称即使在 `struct` 中可见，也不代表可以任意调用。

目录

1 阅读与接入	4
2 基础 / basic	5
2.1 basic/coordinate-compression.cpp	5
2.2 basic/fast-io.cpp	6
2.3 basic/template.cpp	8
2.4 basic/vector.cpp	10
3 数据结构 / data-structure	12
3.1 data-structure/bitset.cpp	12
3.2 data-structure/disjoint-set-rollback.cpp	14
3.3 data-structure/disjoint-set.cpp	15
3.4 data-structure/fast-set.cpp	16
3.5 data-structure/fenwick/fenwick.cpp	17
3.6 data-structure/fenwick/max.cpp	18
3.7 data-structure/fenwick/min.cpp	19
3.8 data-structure/fenwick/sum.cpp	20
3.9 data-structure/fhq-treap.cpp	21
3.10 data-structure/li-chao-tree-static.cpp	23
3.11 data-structure/li-chao-tree.cpp	25
3.12 data-structure/linear-rmq.cpp	26
3.13 data-structure/ordered-disjoint-interval-tree-fast.cpp	27
3.14 data-structure/ordered-disjoint-interval-tree.cpp	28
3.15 data-structure/priority-queue-two-stacks.cpp	29
3.16 data-structure/rollback-int.cpp	30
3.17 data-structure/rollback-int64.cpp	31
3.18 data-structure/segment-tree/dynamic.cpp	32
3.19 data-structure/segment-tree/iterative-prefix.cpp	33
3.20 data-structure/segment-tree/lazy.cpp	35
3.21 data-structure/segment-tree/merge.cpp	37
3.22 data-structure/segment-tree/point.cpp	39
3.23 data-structure/segment-tree/range-add-max.cpp	41
3.24 data-structure/segment-tree/range-add-min.cpp	43
3.25 data-structure/segment-tree/range-add-sum.cpp	45
3.26 data-structure/sparse-table.cpp	47
4 图论 / graph	48
4.1 graph/block-cut-tree.cpp	48
4.2 graph/chordal.cpp	49
4.3 graph/dijkstra.cpp	50
4.4 graph/directed-mst.cpp	51

4.5	graph/edge-biconnected.cpp	52
4.6	graph/flow/max-flow-hlpp.cpp	53
4.7	graph/flow/min-cost-flow-isap.cpp	55
4.8	graph/flow/min-cost-flow-simplex.cpp	57
4.9	graph/matching/bipartite.cpp	59
4.10	graph/matching/general.cpp	60
4.11	graph/matching/regular-bipartite.cpp	61
4.12	graph/steiner-tree.cpp	62
4.13	graph/two-sat.cpp	63
5	树 / tree	64
5.1	tree/cartesian-tree.cpp	64
5.2	tree/centroid-decomposition.cpp	65
5.3	tree/divide-combine-tree.cpp	66
5.4	tree/heavy-light-decomposition.cpp	67
6	字符串 / string	68
6.1	string/aho-corasick.cpp	68
6.2	string/generalized-suffix-automaton.cpp	69
6.3	string/kmp-z.cpp	70
6.4	string/lyndon.cpp	71
6.5	string/manacher.cpp	72
6.6	string/palindromic-automaton.cpp	73
6.7	string/rolling-hash.cpp	74
6.8	string/runs.cpp	75
6.9	string/suffix-array.cpp	76
6.10	string/suffix-automaton.cpp	78
7	数学 / math	79
7.1	math/big-int.cpp	79
7.2	math/dynamic-mod-int.cpp	81
7.3	math/gaussian-elimination.cpp	83
7.4	math/linear-basis/field-prefix.cpp	85
7.5	math/linear-basis/field.cpp	86
7.6	math/linear-basis/xor-prefix.cpp	87
7.7	math/linear-basis/xor.cpp	88
7.8	math/mod-int.cpp	89
7.9	math/rational.cpp	91
7.10	math/xor-equations.cpp	92
8	数论 / number-theory	93
8.1	number-theory/dujiao-sieve.cpp	93
8.2	number-theory/floor-sum.cpp	94

8.3	number-theory/linear-sieve.cpp	95
8.4	number-theory/modular-arithmetic.cpp	96
8.5	number-theory/pollard-rho.cpp	97
8.6	number-theory/stern-brocot.cpp	98
9	多项式 / polynomial	100
9.1	polynomial/fwt.cpp	100
9.2	polynomial/int64.cpp	101
9.3	polynomial/mtt-fast.cpp	102
9.4	polynomial/mtt.cpp	106
9.5	polynomial/ntt-fast.cpp	110
9.6	polynomial/ntt.cpp	114
10	计算几何 / geometry	118
10.1	geometry/geometry-2d.cpp	118

1 阅读与接入

接口项怎么读

每项依次给出“声明 / 调用形式”“参数、效果与返回”“复杂度”。运算符用表达式展示，连续同类重载可能省略重复类型，用省略号时参数规则与同组第一个相同；实际 `const`、引用与模板声明仍以源文件为准。

复杂度中的 n 、 m 默认是本节数据量， N 是补齐后的变换长度。若涉及用户提供的函数、域运算或比较，另乘其单次代价。`void` 接口通过修改对象或引用参数产生结果；未明确写失败返回值时，不满足前置条件通常会触发 `assert`，而不是返回特殊答案。

粘贴顺序与运行

使用 GNU C++17 或更新版本。`basic/template.cpp` 提供头文件、`ll/ull`、调试宏和 `main`；算法代码粘在 `main` 前，示例放在 `main` 内。不同章节示例独立，不要将它们同时粘进一个 `main`。

```
#include<bits/stdc++.h>
using namespace std;
using ll=long long;
using ull=unsigned long long;
// Paste common definitions and dependencies here.
// Paste the selected algorithm here.
int main(){
    ios::sync_with_stdio(false);cin.tie(0);
    // Run one example, then replace it with problem input.
}
```

- 回滚并查集先粘贴一种 `rollback` 保存器；`FastODT` 先 `FastSet`；析合树先 `linear-rmq`；`SA/LCP/LCS` 先 `linear-rmq`；`Runs` 再加 `suffix-array`。
- `SBT` 先 `Rat`。`Gauss/域线性基`使用 `mint` 时先选模整数。多项式文件已内置 `mint`，不再粘贴独立 `ModInt`。
- 同名实现只选一份或手动改名：`segt`、`Tree`、`Cost_Flow_Graph`、`mint`、`poly`、`init`、全局 `S/T/rnd` 等都可能冲突。
- `-DDEBUG` 开启 `debug/debugl` 和 `cerr`；`-DLOCAL` 开启底稿的 `Otomachi_Una.in/out`。`assert` 不要在调试时关闭。

多数图树使用 1 起点号；`vector`、`Bitset`、矩阵、多项式从 0 起。字符串模板通常要求一个前导空格。区间边界逐接口标注，特别注意静态李超树是半开区间。

多测先 `set/setN/setM`，再加数据与 `build/solve`。本文示例说明接口，未声称对所有算法重新完成正确性验证；已有实现的数值约束、复杂度及随机化限制同样需要满足。

2 基础 / basic

2.1 basic/coordinate-compression.cpp

离散化并原地回写

用途

先收集所有会用到的值，再排序去重，把已登记变量改写成 1..k 的排名。时间 $O(n \log n)$ 。

最小示例

```
compress<int> c; c.set();
int a=30,b=10,d=30;
c.pb(a); c.pb(b); c.pb(d); c.build();
assert(a==2 && b==1 && d==2 && c[2]==30);
assert(c.id(30)==2);
```

接口参考

1. 声明 / 调用形式

```
compress<T>; void set()
```

参数、效果与返回：创建收集器；set 清空值及待回写地址，开始下一组数据。

复杂度： $O(n)$ 。

2. 声明 / 调用形式

```
void pb(T &x); void pb(const T &x)
```

参数、效果与返回：非 const 左值：登记原值并保存地址，build 时把 x 改成排名；const 左值或临时值：只登记，不回写。地址须保持有效，不能让所在 vector 扩容。

复杂度：均摊 $O(1)$ 。

3. 声明 / 调用形式

```
void build()
```

参数、效果与返回：排序去重，并一次性回写已登记变量；之后才能 id、range、operator[]。再次插入并 build 会使旧排名失效。

复杂度： $O(n \log n)$ 。

4. 声明 / 调用形式

```
int id(T x) const; T operator[](int k) const
```

参数、效果与返回：id 输入原值，返回 1 起排名，x 必须已登记；c[k] 输入 1..size() 排名，返回原值。

复杂度：id: $O(\log n)$ ；下标: $O(1)$ 。

5. 声明 / 调用形式

```
int size() const; array<int,2> range(const T &l,const T &r) const
```

参数、效果与返回：size 返回当前存储值数量，build 后才是去重数。range 返回原值闭区间 [l,r] 对应的排名闭区间；左端大于右端表示空。端点类型为 T。

复杂度：size: $O(1)$ ；range: $O(\log n)$ 。

注意事项

pb 后、build 前变量必须存活且地址稳定：不要登记即将离开作用域的局部变量，也不要让所属 vector 扩容。id 参数是原值，不是已改写后的排名。range 的端点参数类型为 T。

2.2 basic/fast-io.cpp

缓冲 IO 与 128 位整数 IO

用途

fread/fwrite 缓冲加速，适用于 Windows 和 Linux 下的 GNU C++17。整数、字符、字符串使用 input/output；cin/cout 也支持有符号及无符号 128 位整数。

最小示例

```
// Input: 2 10 20
input.set(stdin); output.set(stdout);
int n; input >> n; lll sum=0;
for(int i=0;i<n;i++){lll x; input >> x; sum+=x;}
output << sum << '\n'; output.flush(); // 30
// Alternative: ordinary streams, without mixing the same FILE.
lll v; istringstream in("-170141183460469231731687303715884105728");
in >> v; ostringstream out; out << v;
assert(out.str()=="-170141183460469231731687303715884105728");
```

接口参考

1. 声明 / 调用形式

```
Reader input; Writer output; void set(FILE *f,size_t sz=1<<16)
```

参数、效果与返回：Reader 默认 stdin，Writer 默认 stdout；freopen 后重新 set。sz 为正的缓冲字节数。Writer::set 会先刷新旧输出；旧 FILE 此时必须仍有效。

复杂度：O(sz)。

2. 声明 / 调用形式

```
bool Reader::read(T &x); Reader& operator>>(T &x)
```

参数、效果与返回：T 可为整数、char、string，含有符号/无符号 __int128，不支持浮点。char 跳过空白，string 读单词；整数必须合法且不越界。返回成功标志；流式读取返回 Reader 引用。

复杂度：O(读入字符数)。

3. 声明 / 调用形式

```
int Reader::get(); int Reader::next()
```

参数、效果与返回：get 读取原始下一个字节，不跳空白；next 跳过 <=32 的空白后读一个字节。返回 0..255 或 EOF，这两个底层接口不更新最近一次 read 的 ok 标志。

复杂度：均摊 O(1)。

4. 声明 / 调用形式

```
explicit Reader::operator bool() const
```

参数、效果与返回：if(input>>x) 检查最近一次读取是否成功；适合 while(input>>x) 读到 EOF。

复杂度：O(1)。

5. 声明 / 调用形式

```
void Writer::put(char c); Writer& operator<<(T x)
```

参数、效果与返回：输出整数十进制、char、string 或 C 字符串；output«x«'\n'。流式运算返回自身引用，可连续使用。

复杂度：O(输出字符数)。

6. 声明 / 调用形式

```
void Writer::flush()
```

参数、效果与返回：写出用户缓冲；析构也会刷新。交互题还须 fflush(stdout)。不要在同一流混用 input/cin 或 output/cout。

复杂度：O(缓冲中的字符数)。

7. 声明 / 调用形式

```
istream& operator>>(istream&,lll&); istream& operator>>(istream&,_uint128_t&)
```

参数、效果与返回：lll 是 __int128 别名，__uint128_t 为无符号 128 位整数；标准流十进制读取，非法或越界置 failbit。与快速输入是两种独立用法。

复杂度：O(字符数)。

8. 声明 / 调用形式

```
ostream& operator<<(ostream&,lll); ostream& operator<<(ostream&,_uint128_t)
```

参数、效果与返回：用 cout 或 ostringstream 输出完整十进制数，返回流引用。

复杂度：O(十进制位数)。

注意事项

不要在同一输入流上混用 input 和 cin，或在同一输出流上混用 output 和 cout。文件必须在 Writer 刷新前保持打开；重定向输出前先 flush。流式 128 位输入遇到非法或越界值设置 failbit；快速整数输入以合法数据为前提。原生 MSVC 不支持本模板使用的 __int128。

2.3 basic/template.cpp

公共底稿

用途

提供类型别名、调试宏和其他模板依赖的辅助函数。自带标准头文件和 main，可直接作为提交程序的底稿。

最小示例

```
int a=7; chkmin(a,3);
assert(a==3 && first_bit(12)==2 && last_bit(12)==3);
debug(a);
cerr << "diagnostic\n";
```

接口参考

1. 声明 / 调用形式

```
ll; ull; inf<T>
```

参数、效果与返回：ll/ull 是 64 位整数别名。inf<int>、inf<ll> 有专门值；其他 T 的默认 inf<T> 是 T{}，不能当作通用无穷大。

复杂度：O(1)。

2. 声明 / 调用形式

```
cin >> a; cout << a
```

参数、效果与返回：a 为已定长非空 vector 或 array：输入逐项读取；输出带方括号和逗号，最多显示 50 项，适合调试，非题目常规输出格式。

复杂度：输入 O(n)；输出至多 50 项。

3. 声明 / 调用形式

```
debug(x1,...,x10); debugl(label,x1,...,x10)
```

参数、效果与返回：支持 1..10 个表达式，输出行号、名字和值；debugl 另加标签。编译 -DDEBUG 开启；否则参数不求值，cerr 也不输出。带逗号的表达式须加括号。

复杂度：与各表达式及输出相关。

4. 声明 / 调用形式

```
int last_bit(ull x); int first_bit(ull x); int popc(ull x)
```

参数、效果与返回：last_bit/first_bit 返回最高/最低 1 的位编号，0 起，x 必须非零；popc 返回 1 的个数，允许零。

复杂度：O(1)。

5. 声明 / 调用形式

```
bool chkmax(A &x,const B &y); bool chkmin(A &x,const B &y)
```

参数、效果与返回：把 x 更新为最大/最小值，发生改变返回 true；否则 false。

复杂度：O(1)。

6. 声明 / 调用形式

```
LOCAL; DEBUG; main()
```

参数、效果与返回：-DLOCAL 将 stdin/stdout 重定向到 Otomachi_Una.in/out；DEBUG 单独控制调试。以本文件为程序底稿，其他模板粘在 main 前，按题意改主流程。

复杂度：不适用。

注意事项

定义 LOCAL 时从 Otomachi_Una.in 读入并写入 Otomachi_Una.out，打开失败返回 1；否则使用标准输入输出。DEBUG 独立控制调试输出。未定义时通过宏屏蔽裸写的 cerr，不要写 std::cerr。向量输入前必须先分配非空长度；向量输出带方括号并截断长向量，正式答案应自己循环输出。

2.4 basic/vector.cpp

vector 常用操作与运算符

用途

生成区间、拼接、逐项运算及集合操作；多数接口返回新 vector，复合赋值才原地修改。

最小示例

```
auto a=vrange(1,4);
auto b=a*2;
assert((b==vector<int>{2,4,6,8}));
auto c=vmap(a,[](int x){return x*x;});
auto d=a&vector<int>{2,2,4,9};
assert((d==vector<int>{2,4}));
assert((a<3)==vector<bool>({true,true,false,false}));
for(int x:vrange(2,4))cout<<x<<' '; // 2 3 4
```

接口参考

1. 声明 / 调用形式

```
vector<T> vrange(T l,T r)
```

参数、效果与返回：生成整数闭区间 $[l,r]$ ，要求 $l \leq r$ ， T 为非 bool 的至多 64 位整数；`vrange(1,4)` 得 $\{1,2,3,4\}$ 。
复杂度： $O(r-l+1)$ 。

2. 声明 / 调用形式

```
vector<T> vcat(vector<T> a,const vector<T> &b)
```

参数、效果与返回：返回 `a` 后拼接 `b`，不修改传入的普通左值；可用 `move(a)` 转移 `a` 的存储。
复杂度： $O(a.size()+b.size())$ 。

3. 声明 / 调用形式

```
auto vmap(const vector<T,Alloc> &a,F f)
```

参数、效果与返回：对每项调用 `f(x)`，按原顺序返回新 vector；结果元素类型自动推导，空 vector 也可。例 `vmap(a,[](int x){return 1ll*x*x;})`。
复杂度： $O(n)$ 次 `f` 调用。

4. 声明 / 调用形式

```
auto vadd(const vector<T> &a,const U &x); auto vsub(const vector<T> &a,const U &x)
```

参数、效果与返回：分别返回逐项加 `x`、减 `x` 的新 vector，与 `a+x`、`a-x` 相同。
复杂度： $O(n)$ 。

5. 声明 / 调用形式

```
a+x; a-x; a*x; a/x; a%x; a&x; a|x; a^x; a<<x; a>>x
```

参数、效果与返回：`x` 在右侧，为标量；逐元素运算，返回新 vector，结果类型按元素表达式推导。除数非零，位移和溢出规则同 C++ 标量运算。
复杂度： $O(n)$ 。

6. 声明 / 调用形式

```
a==x; a!=x; a<x; a<=x; a>x; a>=x; a&& x; a||x
```

参数、效果与返回：右侧标量时返回 `vector<bool>`。&&、|| 不跳过右侧实参求值；两边都是 `vector` 的比较仍按 STL 字典序，返回 `bool`。

复杂度： $O(n)$ 。

7. 声明 / 调用形式

```
a+=x; a-=x; a*=x; a/=x; a%=x; a&=x; a|=x; a^=x; a<<=x; a>>=x
```

参数、效果与返回：原地逐项更新，保留 `a` 的元素类型，返回 `a` 的引用；`x` 按值保存，允许 `x` 来自 `a` 的某个元素。

复杂度： $O(n)$ 。

8. 声明 / 调用形式

```
+a; -a; ~a; !a
```

参数、效果与返回：此行 `+a` 是一元正号：返回逐项正值；其余为负值、按位取反、逻辑取反。均返回新 `vector`，`!a` 的元素类型为 `bool`。

复杂度： $O(n)$ 。

9. 声明 / 调用形式

```
++a; a++; --a; a--
```

参数、效果与返回：所有元素自增/自减；前置返回 `a` 引用，后置返回修改前副本。此组要求元素能绑定 `T&`，不适用于 `vector<bool>`。

复杂度： $O(n)$ 。

10. 声明 / 调用形式

```
vector<T> vfilter(const vector<T> &a, F f)
```

参数、效果与返回：保留 `f(x)` 为真的元素，顺序不变，返回新 `vector`。

复杂度： $O(n)$ 次 `f` 调用。

11. 声明 / 调用形式

```
vector<T> vsort(vector<T> a); vector<T> vunique(vector<T> a)
```

参数、效果与返回：`vsort` 返回排序结果；`vunique` 返回排序去重结果，均不保留原顺序。

复杂度： $O(n \log n)$ 。

12. 声明 / 调用形式

```
vector<T> vintersection(vector<T> a,vector<T> b); vector<T> vunion(vector<T> a,vector<T> b)
```

参数、效果与返回：集合交/并，输入可乱序、重复；输出升序且唯一。交集可写 `a&b`，并集可写 `a|b`；`a&=b`、`a|=b` 原地替换。不是按位置位运算。

复杂度： $O(n \log n + m \log m)$ 。

13. 声明 / 调用形式

```
vector<T> vreverse(vector<T> a); vector<T> vslice(const vector<T> &a,size_t l,size_t r)
```

参数、效果与返回：`vreverse` 返回逆序副本；`vslice` 复制 0 起闭区间 `[l,r]`，要求 `l<=r<a.size()`，不支持空区间。

复杂度： $O(n)$ / $O(r-l+1)$ 。

注意事项

标量右侧运算与 `vector` 对 `vector` 的集合运算不同。`for(int x:f())` 中 `f()` 只调用一次，返回的临时 `vector` 活到循环结束。

3 数据结构 / data-structure

3.1 data-structure/bitset.cpp

动态 Bitset

用途

运行时位数的按位集合，支持位移、区间计数、枚举及窗口位运算。

最小示例

```
Bitset a(130),b(130);
a.set(0);a.set(64);a.set(129);
assert(a.count(1,128)==1);
assert((a.positions()==vector<int>{0,64,129}));
a.window_xor(64,129,b,0);
assert((b.positions()==vector<int>{0,65}));
assert(a.next(64)==129 && a.prev(64)==0);
```

接口参考

1. 声明 / 调用形式

```
Bitset(); Bitset(int n); Bitset(const vector<T>&); Bitset(const string&); using BS=Bitset
```

参数、效果与返回：默认长度 0，整数构造为 n 个零；vector/string 只允许 0/1。输入第 i 项就是 bit i；字符串顺序与 std::bitset 字符串构造相反。

复杂度：O(n/64+1)；序列构造 O(n)。

2. 声明 / 调用形式

```
void setN(int n); void set(); void fill(bool v); int size() const
```

参数、效果与返回：setN 重设长度并清零；set 保留长度清零；fill 统一赋值；size 返回位数。空 bitset 合法。

复杂度：修改 O(n/64+1)，size O(1)。

3. 声明 / 调用形式

```
void set(int i,bool v=true); void reset(int i); void flip(int i)
```

参数、效果与返回：0≤i<n；分别赋值、清零、翻转一位。注意 set() 不是把所有位设为 1。

复杂度：O(1)。

4. 声明 / 调用形式

```
bool get(int i) const; bool operator[](int i) const; void flip()
```

参数、效果与返回：get 和 a[i] 只读，写入必须 set；无参 flip 翻转所有有效位。

复杂度：读 O(1)；全翻转 O(n/64+1)。

5. 声明 / 调用形式

```
int count() const; int count(int l,int r) const
```

参数、效果与返回：返回全部或非空闭区间 [l,r] 内的 1 数量，0≤l≤r<n。

复杂度：O(n/64+1) / O((r-l+1)/64+1)。

6. 声明 / 调用形式

```
vector<int> positions() const; string str() const
```

参数、效果与返回: positions 返回所有 1 的升序下标; str 返回长度 n 的 01 字符串, 下标 0 的字符对应最低位。
复杂度: $O(n/64+1 + \text{答案数}) / O(n)$ 。

7. 声明 / 调用形式

```
bool any() const; bool none() const; bool all() const
```

参数、效果与返回: 判断至少一位为 1、全为 0、全为 1。空对象的 any=false, none=true, all=true。
复杂度: 最坏 $O(n/64+1)$ 。

8. 声明 / 调用形式

```
int next(int p) const; int prev(int p) const
```

参数、效果与返回: next 找严格大于 p 的 1, 找不到返回 n; prev 找严格小于 p 的 1, 找不到返回 -1。next 允许 -1..n, prev 允许 0..n。
复杂度: 最坏 $O(n/64+1)$ 。

9. 声明 / 调用形式

```
int first() const; int last() const; int _Find_first() const; int _Find_next(int p) const
```

参数、效果与返回: first=next(-1), last=prev(n); 两个 _Find 接口分别同 first 和 next。
复杂度: 最坏 $O(n/64+1)$ 。

10. 声明 / 调用形式

```
a&b; a|b; a^b; a&=b; a|=b; a^=b; ~a
```

参数、效果与返回: 两个对象须等长, 按位运算; 普通形式返回新对象, 复合形式改 a 并返回引用。~a 只翻转有效 n 位。
复杂度: $O(n/64+1)$ 。

11. 声明 / 调用形式

```
a<<k; a>>k; a<=<k; a>=>k
```

参数、效果与返回: $k \geq 0$, 左移到较大下标, 右移到较小下标; 越界位丢弃, $k \geq n$ 清零, 长度不变。
复杂度: $O(n/64+1)$ 。

12. 声明 / 调用形式

```
a==b; a!=b; a<b
```

参数、效果与返回: == 同时比较长度与内容; < 要求等长, 把 bit 0 当最低位进行非负整数大小比较。没有 >、<=、>= 重载。
复杂度: 最坏 $O(n/64+1)$ 。

13. 声明 / 调用形式

```
void window_and(int l,int r, Bitset &b,int to) const; window_or(...); window_xor(...)
```

参数、效果与返回: a.window_xor(l,r,b,to) 把 a 的 [l,r] 异或进 b 的 [to,to+r-l]; 另两种同理。两段均须有效; 允许 a、b 同对象且区间重叠, 按更新前源区间计算。无返回值。
复杂度: $O((r-l+1)/64+1)$, 额外空间 $O(1)$ 。

注意事项

下标从 0 起; 所有区间为闭区间。不要直接修改底层 vector, 否则尾部无效位可能破坏 count/比较等接口。

3.2 data-structure/disjoint-set-rollback.cpp

可撤销并查集

用途

适合线段树分治或搜索中加入连边后回退。按大小合并、不做路径压缩，单次查找 $O(\log n)$ 。

最小示例

```
::set(); unionfind d; d.setN(3);
auto t=version(); d.merge(1,2);
assert(d.find(1)==d.find(2));
roll_back(t); assert(d.find(1)!=d.find(2));
```

接口参考

1. 声明 / 调用形式

```
void setN(int n)
```

参数、效果与返回：重建 1..n 单点集合；要求全局 `version()==0`。先回滚或清空保存器，避免旧地址跨过重新分配。

复杂度： $O(n)$ 。

2. 声明 / 调用形式

```
int find(int x); int merge(int x,int y)
```

参数、效果与返回：`find` 不压缩路径；`merge` 按大小合并并 `save` 修改过的父亲/大小。成功返回新根，同集合返回 0。

复杂度： $O(\log n)$ 。

3. 声明 / 调用形式

```
fa; siz; version(); roll_back(t)
```

参数、效果与返回：用 `siz[find(x)]` 读集合大小；用 `find(x)==find(y)` 判断连通。保存全局版本 `t` 后合并，再 `roll_back(t)` 撤销；该版本也包括其他对象的 `save`。

复杂度：撤销 $O(\text{保存条数})$ 。

注意事项

`setN` 前先清空旧历史，不能在旧历史仍引用数组时重新分配。不要自行加入路径压缩。

3.3 data-structure/disjoint-set.cpp

普通并查集

用途

维护集合连通关系，路径压缩配合按大小合并。操作均摊近似 $O(1)$ ，空间 $O(n)$ 。

最小示例

```
DSU d; d.setN(4);
d.merge(1,2); assert(d.same(1,2));
assert(d.siz[d.find(1)]==2 && !d.same(1,3));
```

接口参考

1. 声明 / 调用形式

```
void setN(int n)
```

参数、效果与返回：重建 $1..n$ 的 n 个单点集合，清除旧关系；允许 $n=0$ 。

复杂度： $O(n)$ 。

2. 声明 / 调用形式

```
int find(int x); bool same(int x,int y)
```

参数、效果与返回：find 返回代表元并压缩路径；same 判断同一集合，点必须在 $1..n$ 。

复杂度：均摊 $O(\alpha(n))$ 。

3. 声明 / 调用形式

```
int merge(int x,int y)
```

参数、效果与返回：按大小合并；成功返回合并后的根，原本同集合返回 0。不是返回合并后的大小。

复杂度：均摊 $O(\alpha(n))$ 。

4. 声明 / 调用形式

```
fa; siz
```

参数、效果与返回：fa[x] 为父结点；siz[find(x)] 为集合大小，非根的 siz 不保证代表当前集合。只读这些成员，修改会破坏不变量。

复杂度：查询大小需一次 find。

注意事项

再次 setN 会清空关系。普通版不能回滚；需要撤销时选择 unionfind。

3.4 data-structure/fast-set.cpp

有界整数集合

用途

用分层位集维护集合成员及前驱后继。修改和前驱后继 $O(\log_{64} V)$ ，成员判断 $O(1)$ 。

最小示例

```
FastSet s; s.setN(100);
s.insert(3); s.insert(100);
assert(s.next(3)==3 && s.prev(99)==3);
assert(s.next(101)==-1);
```

接口参考

1. 声明 / 调用形式

```
void setN(int V)
```

参数、效果与返回：清空并设允许元素为 $0..V$ ，包含 V ；不是 $0..V-1$ 。

复杂度： $O(V/64 + \log(V+1))$ 。

2. 声明 / 调用形式

```
void insert(int x); void erase(int x); bool operator[](int x) const
```

参数、效果与返回：插入、删除、查询存在性；重复插入和删除不存在元素无影响；要求 $0 \leq x \leq$ 初始化的 V 。

复杂度：修改 $O(\log_{64}(V+1))$ ，查询 $O(1)$ 。

3. 声明 / 调用形式

```
int prev(int x); int next(int x)
```

参数、效果与返回：prev 找 $\leq x$ 的最大元素；next 找 $\geq x$ 的最小元素，包含 x 自身。都以 -1 表示不存在，参数可超出集合域。

复杂度： $O(\log_{64}(V+1))$ 。

4. 声明 / 调用形式

```
int front(); int back(); vector<int> to_vector() const
```

参数、效果与返回：front/back 返回最小/最大元素，空集 -1；to_vector 返回全部元素的升序 vector。

复杂度：前两项 $O(\log_{64}(V+1))$ ；枚举 $O(V/64 + \text{答案数})$ 。

注意事项

值域太大时不适合使用，空间跟 V 相关而非当前元素个数。需要 src 的位运算和最值辅助函数。

3.5 data-structure/fenwick/fenwick.cpp

自定义下标的树状数组

用途

同时提供单点加的 BIT 和区间加的双 BIT。初始化 $O(n)$ ，单次操作 $O(\log n)$ ，空间 $O(n)$ 。

最小示例

```
BIT b; b.setRange(-2,2); b.add(0,7);
assert(b.ask(0)==7 && b.ask(-1,1)==7);
rars t; t.setRange(-2,2); t.add(-1,1,3);
assert(t.ask(-2,0)==6);
b.setN(3); assert(b.ask(3)==0);
```

接口参考

1. 声明 / 调用形式

```
BIT::set(); BIT::setRange(int L,int R); BIT::setN(int n)
```

参数、效果与返回：清空，或建立初值全零的闭区间； $\text{setN}(n)=\text{setRange}(1,n)$ 。

复杂度： $O(R-L+1)$ 。

2. 声明 / 调用形式

```
BIT::add(ll i,ll v); BIT::ask(ll r); BIT::ask(int l,int r)
```

参数、效果与返回：单点加、前缀和、闭区间和；允许 $i=R+1$ 哨兵；前缀只允许 $L-1..R$ ；双端查询要求 $L \leq l \leq r \leq R$ 。

复杂度： $O(\log(R-L+2))$ 。

3. 声明 / 调用形式

```
rars::set(); rars::setRange(int L,int R); rars::setN(int n)
```

参数、效果与返回：两棵 BIT 维护区间加区间和，初始化域与 BIT 相同，初值零。

复杂度： $O(R-L+1)$ 。

4. 声明 / 调用形式

```
rars::add(int l,int r,ll v); rars::ask(int l,int r)
```

参数、效果与返回：在有效闭区间整体加 v ，返回区间总和；必须 $L \leq l \leq r \leq R$ ，不接受空区间。

复杂度： $O(\log(R-L+2))$ 。

5. 声明 / 调用形式

```
ll rars::query(ll r) const
```

参数、效果与返回：返回前缀 $[L,r]$ 的总和，允许 $L-1 \leq r \leq R$ ； $r=L-1$ 返回 0。与 $\text{rars::ask}(l,r)$ 的双端闭区间区分。

复杂度： $O(\log(R-L+2))$ 。

注意事项

下标 0 或负数只要在设定区间内就合法；内部会转成正下标。 $\text{ask}(L-1)$ 返回零， $\text{add}(R+1,v)$ 是差分哨兵，不计入查询。极端坐标的 $L-1/R+1$ 用 ll 计算。 rars 的加权差分使用相对位置，不依赖坐标绝对值；值、加权积与答案须在 ll 范围内。 $\text{set}()$ 清空；重新 $\text{setRange}/\text{setN}$ 丢弃旧值。文件末尾保留全局 rars 对象 T 。

3.6 data-structure/fenwick/max.cpp

树状数组：最大值特化

用途

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

最小示例

```
Max_BIT t; t.setRange(-2,2); t.upd(0,5); t.upd(0,3);
assert(t.ask(1)==5);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setRange(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空域；setRange 设置闭区间 $[L,R]$ 并重置所有值为 `LLONG_MIN`；setN(n) 等价 setRange(1,n)，允许 $n=0$ 建空域。

复杂度： $O(R-L+1)$ 。

2. 声明 / 调用形式

```
void upd(ll i,ll v)
```

参数、效果与返回：将下标 i 的值取 $\max(\text{原值}, v)$ ，不是任意赋值。不支持撤回旧最值或反向修改。

复杂度： $O(\log(R-L+2))$ 。

3. 声明 / 调用形式

```
ll ask(ll r) const
```

参数、效果与返回：返回前缀 $[L,r]$ 的 $\max$ ；只允许 $L-1 \leq r \leq R$ ； $r=L-1$ 返回 `LLONG_MIN`；不自动截断越界参数。

复杂度： $O(\log(R-L+2))$ 。

注意事项

初值和空前缀返回 `LLONG_MIN`。只支持单点 chmax ；不能把已有大值减小。没有 $\text{ask}(l,r)$ ，最值不能像和一样用两个前缀相减；任意点赋值或一般区间最值请用线段树。三个类名不同，可组合粘贴；set() 清空。

3.7 data-structure/fenwick/min.cpp

树状数组：最小值特化

用途

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

最小示例

```
Min_BIT t; t.setRange(-2,2); t.upd(0,5); t.upd(0,3);
assert(t.ask(1)==3);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setRange(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空域；setRange 设置闭区间 $[L,R]$ 并重置所有值为 `LLONG_MAX`；setN(n) 等价 setRange(1,n)，允许 $n=0$ 建空域。

复杂度： $O(R-L+1)$ 。

2. 声明 / 调用形式

```
void upd(ll i,ll v)
```

参数、效果与返回：将下标 i 的值取 $\min(\text{原值}, v)$ ，不是任意赋值。不支持撤回旧最值或反向修改。

复杂度： $O(\log(R-L+2))$ 。

3. 声明 / 调用形式

```
ll ask(ll r) const
```

参数、效果与返回：返回前缀 $[L,r]$ 的 $\min$ ；只允许 $L-1 \leq r \leq R$ ； $r=L-1$ 返回 `LLONG_MAX`；不自动截断越界参数。

复杂度： $O(\log(R-L+2))$ 。

注意事项

初值和空前缀返回 `LLONG_MAX`。只支持单点 `chmin`；不能把已有小值增大。没有 `ask(l,r)`，最值不能像和一样用两个前缀相减；任意点赋值或一般区间最值请用线段树。三个类名不同，可组合粘贴；set() 清空。

3.8 data-structure/fenwick/sum.cpp

树状数组：和特化

用途

初始化 $O(n)$ ，单点修改和前缀查询 $O(\log n)$ ，空间 $O(n)$ 。

最小示例

```
Sum_BIT t; t.setRange(-2,2); t.add(0,5); t.add(1,-3);
assert(t.ask(0)==5 && t.ask(0,1)==2);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setRange(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空域；setRange 设置闭区间 $[L,R]$ 并重置所有值为 0；setN(n) 等价 setRange(1,n)，允许 $n=0$ 建空域。

复杂度： $O(R-L+1)$ 。

2. 声明 / 调用形式

```
void add(ll i,ll v)
```

参数、效果与返回：将下标 i 的值加上 v ，不是任意赋值。 $i=R+1$ 是允许的差分哨兵，不参与有效域查询。

复杂度： $O(\log(R-L+2))$ 。

3. 声明 / 调用形式

```
ll ask(ll r) const
```

参数、效果与返回：返回前缀 $[L,r]$ 的和；只允许 $L-1 \leq r \leq R$ ； $r=L-1$ 返回 0；不自动截断越界参数。

复杂度： $O(\log(R-L+2))$ 。

4. 声明 / 调用形式

```
ll ask(int l,int r) const
```

参数、效果与返回：返回闭区间和，用前缀相减；要求 $L \leq l \leq r \leq R$ ，不接受空区间。

复杂度： $O(\log(R-L+2))$ 。

注意事项

初始值全零；允许 add($R+1,v$) 差分哨兵及 ask($L-1$) 空前缀。支持负坐标，查询不会修改树。三个类名不同，可组合粘贴；set() 清空。

3.9 data-structure/fhq-treap.cpp

按键或按序列位置分裂的 Treap

用途

随机平衡二叉树。外部维护根编号 `rt`，空树为 0，普通 `split/merge` 期望 $O(\log n)$ 。

最小示例

```
FHQ t; t.set(); int rt=0;
for(int v:{2,5,8})rt=t.merge(rt,t.newnode(v));
int a,b; t.split_by_key(rt,5,a,b);
assert(t.siz[a]==2 && t.siz[b]==1);
rt=t.merge(a,b); assert(t.siz[rt]==3);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n); int newnode(int x)
```

参数、效果与返回：set 清空；setN 创建 n 个互不相连、`key=0` 的节点，不会建成一棵树；newnode 新增 `key=x` 节点并返回编号。根由调用者保存，0 表示空。

复杂度：setN $O(n)$ ，newnode 均摊 $O(1)$ 。

2. 声明 / 调用形式

```
int merge(int p,int q)
```

参数、效果与返回：把 p 中序序列接在 q 前，返回新根；两树节点不相交。作为有序集合时须 $\max(p) \leq \min(q)$ 。操作会修改原树。

复杂度：期望 $O(\log n)$ 。

3. 声明 / 调用形式

```
void split_by_key(int rt,int k,int &p,int &q)
```

参数、效果与返回：将 rt 拆成 `key` $\leq k$ 的 p 和 `key` $> k$ 的 q ，写入引用；原 rt 不再是独立有效的整树根。

复杂度：期望 $O(\log n)$ 。

4. 声明 / 调用形式

```
void split_by_size(int rt,int k,int &p,int &q)
```

参数、效果与返回：前 k 项归 p ，剩余归 q ，要求 $0 \leq k \leq \text{siz}[rt]$ ；可用于序列区间切分。

复杂度：期望 $O(\log n)$ 。

5. 声明 / 调用形式

```
int merge_it(int p,int q)
```

参数、效果与返回：合并任意两个有序树，允许键值域交错和重复值，保留所有节点；返回新根，消耗原树。两树节点不可共享。

复杂度：取决于两树交错程度，不能按普通 `merge` 的 $O(\log n)$ 估算。

6. 声明 / 调用形式

```
int leftmost(int rt); int rightmost(int rt)
```

参数、效果与返回：返回最左/最右节点编号，空树返回 0；沿途下传标记。取值再读 `key[id]`。

复杂度：期望 $O(\log n)$ 。

7. 声明 / 调用形式

```
void push(const int &id, const int &x); void alldown(int id)
```

参数、效果与返回：push 给整棵非空子树的 key 加 x；必须保持与外部树的排序关系。alldown 把全部标记下传，之后可直接读取所有 key。

复杂度： $O(1)$ / $O(\text{子树大小})$ 。

8. 声明 / 调用形式

```
ls; rs; key; siz; tag; val; void pushup(const int &id); void pushdown(const int &id)
```

参数、效果与返回：分别是左右儿子、键、大小、加法标记、随机堆优先级。手动改结构后 pushup 更新大小；读取儿子前 pushdown 下传。普通调用无需直接改这些成员。

复杂度：单次维护 $O(1)$ 。

注意事项

setN(n) 只是创建 n 个孤立节点，不会把它们连成树。按键分裂依赖排序不变量；区间加可能破坏该不变量。没有现成的区间翻转或持久化。全局 rnd 可能与别的模板重名。

3.10 data-structure/li-chao-tree-static.cpp

离散或连续整数域上的静态李超树

用途

把函数类型作为模板参数，返回最优值和对应函数。

最小示例

```
struct F{
    using value_type=ll;
    ll k,b;int id;
    ll operator()(ll x)const{return k*x+b;}
};
LiChao<F> t;
t.set(vector<ll>{1,3,5},F{0,LLONG_MAX,-1});
t.add(F{2,1,7});
t.add(F{0,4,8},3,6);
auto [y,f]=t.ask(3);
assert(y==4 && f.id==8);
```

接口参考

1. 声明 / 调用形式

```
LiChao<F,bool COMPRESS=true,bool MINIMIZE=true>
```

参数、效果与返回：F 提供 using value_type=T 和 T operator()(ll x) const。COMPRESS 选择离散点/连续整数域，MINIMIZE 选择最小/最大。任意两函数优劣在域内至多切换一次，直线满足条件。

复杂度：求值 $F(x)$ 视为 $O(1)$ 。

2. 声明 / 调用形式

```
void set(const vector<ll> &X,const F &f)
```

参数、效果与返回：仅 COMPRESS=true：复制、排序去重 X，不能为空；f 是空节点默认函数，须始终不优于真实答案。例求最小用常数 LLONG_MAX 函数。

复杂度： $O(n \log n)$ ，空间 $O(n)$ 。

3. 声明 / 调用形式

```
void setRange(ll l,ll r,const F &f)
```

参数、效果与返回：仅 COMPRESS=false：建立整数半开域 $[l,r)$ ，要求 $0 < r-l \leq 2^{29}$ ；f 的要求同上。不是动态开点，按域长度分配。

复杂度： $O(r-l)$ 时间/空间。

4. 声明 / 调用形式

```
void set(); void add(F f); void add(F f,ll l,ll r)
```

参数、效果与返回：set 只清空函数，保留坐标和默认函数；add(f) 全域插入；带区间重载只覆盖 $[l,r)$ ，空区间无效果。离散模式的区间端点无需在 X 内。

复杂度：清空 $O(n)$ ，全域 $O(\log n)$ ，区间 $O(\log^2 n)$ 。

5. 声明 / 调用形式

```
pair<T,F> ask(ll x) const
```

参数、效果与返回：返回最优值及产生该值的函数副本，可在 F 中自带 id。平局任取；无真实线返回默认函数。
离散模式 x 必须在 X 内；连续模式须 $l \leq x < r$ 。

复杂度： $O(\log n)$ 。

注意事项

此模板用半开区间 $[l,r)$ ，与动态李超树的闭区间不同。默认函数也会被求值，因此必须构造造成安全的常数函数。

3.11 data-structure/li-chao-tree.cpp

线段李超树

用途

插入只在一段整数横坐标上有效的直线，查询某点的最小值。

最小示例

```
lichao t; t.set();
t.add({2,3},0,10); t.add({-1,8},0,10);
assert(t.ask(2)==6);
```

接口参考

1. 声明 / 调用形式

```
line{ll k,ll b}; ll line::f(ll x); void lichao::set()
```

参数、效果与返回：line 表示 $k*x+b$ ，f 求值；set 删除所有直线/线段。域固定为 $[-V,V]$ ， $V=1e9$ 。

复杂度：求值 $O(1)$ ，清空与已分配空间相关。

2. 声明 / 调用形式

```
void add(line f,int l=-V,int r=V)
```

参数、效果与返回：将函数插入闭区间 $[l,r]$ ；省略区间则全域。斜率绝对值 $\leq V$ ，相关乘加须安全，有效答案须在哨兵 $\text{inf}=2e18$ 范围内。

复杂度：全线 $O(\log V)$ ，区间 $O(\log^2 V)$ 。

3. 声明 / 调用形式

```
ll ask(int x)
```

参数、效果与返回：返回 x 处最小函数值；没有覆盖 x 的线返回 inf 。不返回是哪条线。

复杂度： $O(\log V)$ 。

注意事项

横坐标及斜率限制在 $[-10^9,10^9]$ ，有效端点的纵坐标需在 $[-2e18,2e18]$ 。整条线插入 $O(\log V)$ ，线段插入 $O(\log^2 V)$ ，查询 $O(\log V)$ 。

3.12 data-structure/linear-rmq.cpp

四毛子 RMQ：线性预处理与空间

用途

静态数组最值。采用块内单调栈位掩码、块间 ST 表；在机器字位运算为 $O(1)$ 的模型下，预处理及空间 $O(n)$ ，查询 $O(1)$ 。

最小示例

```
vector<int> a{4,1,1,3};
Linear_RMQ<int> mn; mn.build(a);
assert(mn.ask(0,3)==1 && mn.pos(0,3)==1);
Linear_RMQ<int,true> mx; mx.build(a);
assert(mx.ask(1,3)==3);
```

接口参考

1. 声明 / 调用形式

```
Linear_RMQ<T,bool MAX=false>; void set(); void build(const vector<T> &a)
```

参数、效果与返回：默认最小值，MAX=true 最大值；set 清空，build 复制 0 起数组并预处理，可空数组但不能对其查询。

复杂度：build $O(n)$ 时间/空间。

2. 声明 / 调用形式

```
int pos(int l,int r) const; T ask(int l,int r) const
```

参数、效果与返回： $0 \leq l \leq r < n$ 的闭区间；pos 返回最优值下标，平局最左；ask 返回该元素值。

复杂度： $O(1)$ 。

注意事项

块长 $B = \max(1, \text{floor}(\log_2 n))$ ，不是固定常数；块数 $m = \text{ceil}(n/B)$ ，块间表 $O(m \log m) = O(n)$ 。每个元素存一份块前缀的单调栈掩码，一次清除低位和 ctz 即可查询块内最左最值。 $n < \text{INT_MAX}$ ，块长最多 30，使用 unsigned 位运算。可 build 空数组，但不能查询；修改数组后须重新 build。set() 清空。不需要普通 ST 表或笛卡尔树作为依赖。

3.13 data-structure/ordered-disjoint-interval-tree-fast.cpp

位集加速的区间覆盖

用途

借助 FastSet 存储区间右端点。下标 1..n，适合值域固定的区间覆盖。

最小示例

```
Fast_ODT t; t.setN(5); t.insert(1,5,0);  
t.insert(2,4,7); auto a=t.extract(2,3);  
assert(a.size()==1 && a[0][2]==7);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n)
```

参数、效果与返回：建立 1..n，初始全部颜色 -1， $n \geq 1$ ；依赖 FastSet。

复杂度： $O(n)$ 。

2. 声明 / 调用形式

```
vector<array<int,3>> extract(int l,int r)
```

参数、效果与返回：返回有效闭区间 $[l,r]$ 内各段 {左端, 右端, 颜色}，按位置排列；只切段，不改颜色。

复杂度： $O((\text{段数} + 1) * \log_{64}(n+1))$ 。

3. 声明 / 调用形式

```
vector<array<int,3>> insert(int l,int r,int c)
```

参数、效果与返回：把闭区间整体赋色 c；返回赋色前被覆盖的段，而不是新段。不会主动合并相邻同色段。

复杂度： $O((\text{覆盖段数} + 1) * \log_{64}(n+1))$ 。

注意事项

extract 会分裂边界。不保证任意操作序列的高效性，成本仍取决于访问的段数。

3.14 data-structure/ordered-disjoint-interval-tree.cpp

有序区间集合

用途

维护若干不相交的常值闭区间。适合区间覆盖以及依赖当前分段的统计。

最小示例

```
ODT t; t.set(); t.add(1,5,0);
t.erase(2,4); t.add(2,4,7);
auto a=t.extract(2,3);
assert(a.size()==1 && a[0].c==7);
```

接口参考

1. 声明 / 调用形式

```
seg(int l,int r,int c); void ODT::set()
```

参数、效果与返回：seg 存闭区间及颜色，可读取 l/r/c；set 清空全部段，初始没有覆盖。

复杂度：清空 $O(\text{段数})$ 。

2. 声明 / 调用形式

```
void add(int l,int r,int c)
```

参数、效果与返回：插入一段 [l,r]；先用 erase 清掉重叠段。不会自动覆盖，也不会合并相邻同色段。

复杂度： $O(\log \text{ 段数})$ 。

3. 声明 / 调用形式

```
vector<seg> erase(int l,int r); vector<seg> extract(int l,int r)
```

参数、效果与返回：都按边界切段并按位置返回交集内各段；erase 删除这些段，extract 保留。没有覆盖的位置不返回。要求 $l \leq r$ 。

复杂度： $O(\log \text{ 段数} + \text{返回段数})$ 。

注意事项

extract 会切开边界，可能增加内部段数；没有自动合并相邻同色段。一次操作可能遍历很多段，不保证对任意输入都是对数复杂度。

3.15 data-structure/priority-queue-two-stacks.cpp

把优先队列删除转为栈撤销

用途

适合只支持栈式加入/撤销，却需要删除最大权元素的离线或可撤销结构。模板只安排顺序，实际状态由调用方维护。

最小示例

```
pq2stack<int> q; q.set(); vector<int> st;
q.push(10,1); q.push(20,2);
for(int x:q.prep())st.push_back(x);
int k=q.pop(); while(k-->0)st.pop_back();
for(int x:q.prep())st.push_back(x);
assert(st.size()==1 && st[0]==10);
```

接口参考

1. 声明 / 调用形式

```
pq2stack<T>; void set(); void push(T x,int w)
```

参数、效果与返回：维护按 w 删除最大权元素的重排计划。set 清空计划；push 登记元素和权重，不会替你修改外部可撤销结构。

复杂度：push 均摊 $O(1)$ 。

2. 声明 / 调用形式

```
vector<T> prep()
```

参数、效果与返回：返回外部结构尚未执行、需要按顺序加入的元素。每次查询答案前遍历 prep() 的结果执行加入，并保存撤销信息。连续 prep 且无修改返回空。

复杂度： $O(\text{返回元素数})$ 。

3. 声明 / 调用形式

```
int pop()
```

参数、效果与返回：非空时删除最大 w 的元素，返回外部已执行操作中需要撤销的尾部个数；按该数量撤销，下一次 prep 重放剩余元素。返回值不是元素，也不是权重。

复杂度：重排代价与当前元素和重建段长相关。

注意事项

不能把 pop 的返回值当成被删元素。每次重加入都需要重新产生撤销记录；同权元素删除顺序不固定。单次可能重排大量元素，收益依赖整段操作序列。

3.16 data-structure/rollback-int.cpp

记录并撤销变量修改

用途

保存变量的地址和旧值，用版本号撤销一段修改。每次保存和撤销一次记录都是 $O(1)$ 。

最小示例

```
::set(); int x=7;
auto t=version(); save(x); x=20;
roll_back(t); assert(x==7);
```

接口参考

1. 声明 / 调用形式

```
void set(); int version()
```

参数、效果与返回：set 丢弃保存记录，不会恢复变量；version 返回记录条数，用 `int t=version()` 保存检查点。

复杂度：set $O(\text{记录数})$ ，version $O(1)$ 。

2. 声明 / 调用形式

```
void save(int &x)
```

参数、效果与返回：记录变量当前地址和值，之后自行修改 `x`。重复 save 同一地址合法；变量必须一直存活且地址稳定，不能跨 vector 扩容或对象销毁。

复杂度：均摊 $O(1)$ 。

3. 声明 / 调用形式

```
void roll_back(int t)
```

参数、效果与返回：按逆序恢复到版本 `t`，要求 $0 \leq t \leq \text{version}()$ ；恢复后记录被移除。两个保存器选一个，不可同时粘贴。

复杂度： $O(\text{version}()-t)$ 。

注意事项

保存期间变量地址必须保持有效，不能让包含它的 vector 扩容。每个版本号只能用于当前历史；两种 saver 不能同时粘贴。

3.17 data-structure/rollback-int64.cpp

记录并撤销变量修改

用途

保存变量的地址和旧值，用版本号撤销一段修改。每次保存和撤销一次记录都是 $O(1)$ 。

最小示例

```
::set(); ll x=7;
auto t=version(); save(x); x=20;
roll_back(t); assert(x==7);
```

接口参考

1. 声明 / 调用形式

```
void set(); int version()
```

参数、效果与返回：set 丢弃保存记录，不会恢复变量；version 返回记录条数，用 `int t=version()` 保存检查点。

复杂度：set $O(\text{记录数})$ ，version $O(1)$ 。

2. 声明 / 调用形式

```
void save(int 或 ll &x)
```

参数、效果与返回：记录变量当前地址和值，之后自行修改 `x`。重复 save 同一地址合法；变量必须一直存活且地址稳定，不能跨 vector 扩容或对象销毁。

复杂度：均摊 $O(1)$ 。

3. 声明 / 调用形式

```
void roll_back(int t)
```

参数、效果与返回：按逆序恢复到版本 `t`，要求 $0 \leq t \leq \text{version}()$ ；恢复后记录被移除。两个保存器选一个，不可同时粘贴。

复杂度： $O(\text{version}()-t)$ 。

注意事项

保存期间变量地址必须保持有效，不能让包含它的 vector 扩容。每个版本号只能用于当前历史；两种 saver 不能同时粘贴。

3.18 data-structure/segment-tree/dynamic.cpp

大值域稀疏单点统计

用途

在整数键 $-10^9..10^9$ 上做单点加、区间和；节点带键交换以压缩空间。单次 $O(\log V)$ 。

最小示例

```
dynamic_segt t; t.set();
t.add(-100,3); t.add(200,4);
assert(t.ask(-100,0)==3);
```

接口参考

1. 声明 / 调用形式

```
void set(int m=0)
```

参数、效果与返回：清空并预留 m 个节点的容量，不改变固定域 $[-1e9, 1e9]$ ； m 不是数组长度，也不是值域。

复杂度： $O(\text{旧节点数})$ ，预留空间 $O(m)$ 。

2. 声明 / 调用形式

```
void add(int p, ll v); ll ask(int l, int r)
```

参数、效果与返回：在 p 点加 v ；查询闭区间和，要求端点位于固定域。未出现位置值为零；每个不同插入位置只占一个节点。

复杂度： $O(\log \text{值域})$ ，空间 $O(\text{不同位置数})$ 。

注意事项

空键权值为零。与主席树不同：不保留历史版本，也不支持复用旧根。文件有全局对象 T 。

树上二分：max_right / min_left

```
pair<ll,info> max_right(ll l, F f);
pair<ll,info> min_left(ll r, F f);
```

参数、效果与返回：有效下标域为固定域 $[-1e9, 1e9]$ 。两者返回边界与对应闭区间的累计信息， info.s 为区间和。 $\text{max_right}(l, f)$ 从 l 向右扩展，返回最大的 r ，使 $f([l, r])$ 为真；第一个元素就不满足时返回 $l-1$ 和空信息。允许从域的右端点加一开始，此时结果为空。

$\text{min_left}(r, f)$ 从 r 向左扩展，返回最小的 l ，使 $f([l, r])$ 为真；第一个元素就不满足时返回 $r+1$ 和空信息。允许从域的左端点减一开始。返回边界使用 ll ，避免在 int 极值下标处表示空区间时溢出。

判定要求： f 接受累计信息，返回 bool ；空信息必须为真。沿对应方向扩展区间时只能由真变假，不能变回真。 f 不得修改树或累计信息，也不应依赖调用次数。合并始终保持从左到右的顺序，不要求 info 支持减法。正向判定单调不代表反向也单调。

复杂度： $O(\log \text{值域})$ ，不额外做一次区间查询。下面是独立示例。

```
dynamic_segt t; t.set(); t.add(-100,3); t.add(200,4);
auto [r,x]=t.max_right(-100, [](const auto &v){return v.s<=3;});
assert(r==199&&x.s==3);
```

用区间和不超过阈值作为判定时，需保证各点权值非负。

3.19 data-structure/segment-tree/iterative-prefix.cpp

非递归线段树框架

用途

单点赋值和闭区间聚合，当前 info 维护区间和与最大前缀和（允许空前缀）。单次 $O(\log n)$ 。

最小示例

```
zkw_segt t; t.setN(3);  
t.upd(0,-4,0); t.upd(1,9,9); t.upd(2,-2,0);  
assert(t.ask(0,2)==5);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n)
```

参数、效果与返回：建立 $0..n-1$ 的全零序列。

复杂度： $O(n)$ 时间/空间。

2. 声明 / 调用形式

```
void upd(int i,ll s,ll mx)
```

参数、效果与返回：赋值叶子信息，总和 s 、最大前缀 mx 。普通单元素用 $\text{upd}(i,v,\max(0ll,v))$ ，允许空前缀。也可把一个叶子当作已汇总的块。

复杂度： $O(\log n)$ 。

3. 声明 / 调用形式

```
ll ask(int l,int r)
```

参数、效果与返回：返回 0 起闭区间 $[l,r]$ 的最大前缀和；要求有效非空区间。不会返回总和或最大子段和。

复杂度： $O(\log n)$ 。

注意事项

普通数组的叶子设为 $s=a[i]$ 、 $mx=\max(0,a[i])$ 。若 s 全为零且 mx 非负，也可用于区间最大值。其他统计应重写 info 的合并和单位元；合并必须满足结合律。下标从 0 开始。

树上二分: `max_right` / `min_left`

```
pair<ll,info> max_right(ll l,F f);  
pair<ll,info> min_left(ll r,F f);
```

参数、效果与返回: 有效下标域为 $[0, n-1]$ 。两者返回边界与对应闭区间的累计信息, `info.s` 为区间和, `info.mx` 为允许空前缀的最大前缀和。

`max_right(l,f)` 从 l 向右扩展, 返回最大的 r , 使 $f([l,r])$ 为真; 第一个元素就不满足时返回 $l-1$ 和空信息。允许从域的右端点加一开始, 此时结果为空。

`min_left(r,f)` 从 r 向左扩展, 返回最小的 l , 使 $f([l,r])$ 为真; 第一个元素就不满足时返回 $r+1$ 和空信息。允许从域的左端点减一开始。返回边界使用 `ll`, 避免在 `int` 极值下标处表示空区间时溢出。

判定要求: `f` 接受累计信息, 返回 `bool`; 空信息必须为真。沿对应方向扩展区间时只能由真变假, 不能变回真。`f` 不得修改树或累计信息, 也不应依赖调用次数。合并始终保持从左到右的顺序, 不要求 `info` 支持减法。正向判定单调不代表反向也单调。

复杂度: $O(\log n)$, 不额外做一次区间查询。下面是独立示例。

```
zkw_segt t; t.setN(3);  
t.upd(0,-4,0); t.upd(1,9,9); t.upd(2,-2,0);  
auto [r,x]=t.max_right(0,[](const auto &v){return v.mx<=4;});  
assert(r==0&&x.s==-4);
```

3.20 data-structure/segment-tree/lazy.cpp

通用线段树（带懒标记）

用途

默认维护区间和，下标 $1..n$ ，初值全零。递归部分只处理 info，按题目改写信息与标记的运算。

最小示例

```
segt t; t.setN(3); t.upd(2,5);
t.add(1,2,2); assert(t.ask(1,3)==9);
```

接口参考

1. 声明 / 调用形式

```
void set(); void set(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空；set(L,R) 建非空闭区间，全部叶子初值 0；setN(n)=set(1,n)。这里 $n \geq 1$ 。

复杂度： $O(R-L+1)$ 时间/空间。

2. 声明 / 调用形式

```
void upd(int i,ll v); void add(int i,ll v)
```

参数、效果与返回：upd 把第 i 项赋成 v ；add 给它加 v 。 i 必须在当前域中。

复杂度： $O(\log n)$ 。

3. 声明 / 调用形式

```
ll ask(int l,int r); ll ask(int i)
```

参数、效果与返回：返回有效非空闭区间的和；单参数重载返回该点值。查询不会修改逻辑上的数组内容。

复杂度： $O(\log n)$ 。

4. 声明 / 调用形式

```
void add(int l,int r,ll v)
```

参数、效果与返回：给闭区间各项加 v ；普通用法保证区间有效且非空。

复杂度： $O(\log n)$ 。

5. 声明 / 调用形式

```
int find_first(ll x); int find_last(ll x)
```

参数、效果与返回：所有叶子非负且 $0 < x \leq$ 总和。find_first 找从左累加首次达到 x 的位置；find_last 从右累加首次达到 x 的位置。不存在答案会触发断言，没有失败哨兵。前两份通用结构由 USE_COMPARE 控制是否编译。

复杂度： $O(\log n)$ 。

6. 声明 / 调用形式

info / lazt

参数、效果与返回：改题时修改 info 的空值、叶子构造和合并，保持结合律；再改公开 upd/add/ask 的参数转换与答案提取。懒标记类型实际名为 lazt，须同步修改标记复合与其作用于 info 的逻辑。二分比较也须与新信息的单调性一致，不能只换 s 的含义。

复杂度：每次合并/标记按 $O(1)$ 设计。

注意事项

info + info 按左右顺序合并；upd 在叶子整体替换 info，单点 add 使用 info(delta,0)，通过 info += info 更新叶子。初值在 build 的 info(0,1) 处修改。参数转换、提取答案只改公开接口。USE_COMPARE 默认开启；自定义信息不支持减法与比较时，删除文件顶部的 define 行，对应运算和二分函数就不编译。默认二分要求所有点值非负、目标为正且不超过总和。set/setN 清空重建。两版同名 segt，只选一份。lazt += lazt 的顺序是旧标记后接新标记；info += lazt 负责作用标记，区间长度等元数据由 info 保存。不能仅因支持单点加，就直接认为相同统计能支持区间加。

树上二分：max_right / min_left

```
pair<ll,info> max_right(ll l,F f);
pair<ll,info> min_left(ll r,F f);
```

参数、效果与返回：有效下标域为初始化时指定的 [vl,vr]，setN(n) 对应 [1,n]。两者返回边界与对应闭区间的累计信息，info.s 为区间和，info.len 为长度。

max_right(l,f) 从 l 向右扩展，返回最大的 r，使 f([l,r]) 为真；第一个元素就不满足时返回 l-1 和空信息。允许从域的右端点加一开始，此时结果为空。

min_left(r,f) 从 r 向左扩展，返回最小的 l，使 f([l,r]) 为真；第一个元素就不满足时返回 r+1 和空信息。允许从域的左端点减一开始。返回边界使用 ll，避免在 int 极值下标处表示空区间时溢出。

判定要求：f 接受累计信息，返回 bool；空信息必须为真。沿对应方向扩展区间时只能由真变假，不能变回真。f 不得修改树或累计信息，也不应依赖调用次数。合并始终保持从左到右的顺序，不要求 info 支持减法。正向判定单调不代表反向也单调。

复杂度： $O(\log n)$ ，不额外做一次区间查询。下面是独立示例。

```
segt st; st.setN(5);
auto [r,x]=st.max_right(1,[](const auto &v){return v.s<=10;});
```

用区间和不超过阈值作为判定时，需保证各点权值非负。

3.21 data-structure/segment-tree/merge.cpp

可合并的多棵线段树

用途

维护多棵互不共享节点的树，值域 $0..n$ 。当前权值设计用于统计所有键 $y \geq x$ 的 $(y-x) \cdot \text{权值}$ 。

最小示例

```
segt_merge t; t.setN(10); int a=0,b=0;
t.add(a,5,2); t.add(b,8,1); t.merge(a,b);
assert(b==0 && t.ask(a,3)==9);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n)
```

参数、效果与返回：清空节点池，值域为 $0..n$ ，包含 n 。调用者为每棵树保存 $\text{int rt}=0$ ；所有旧根失效。

复杂度： $O(\text{旧节点数})$ 。

2. 声明 / 调用形式

```
void add(int &rt,int x,int o)
```

参数、效果与返回：在 x 处增加权重/次数 o ，并通过引用更新根。可用负 o 减少；内部维护权重和与 $x \cdot \text{权重}$ 和。

复杂度： $O(\log(n+1))$ 。

3. 声明 / 调用形式

```
ll ask(int &rt,int x)
```

参数、效果与返回：返回 $\sum_{v \geq x} (v-x) \cdot \text{cnt}[v]$ ，不是区间和。空树返回 0； x 在 $0..n$ 。

复杂度： $O(\log(n+1))$ 。

4. 声明 / 调用形式

```
void merge(int &x,int &y)
```

参数、效果与返回：把 y 合入 x ，逐位置权重相加，令 $y=0$ ；两个根须独立，不能共享节点，也不能传同一个根变量。合并后只用 x 。

复杂度： $O(\text{访问的重叠节点数} + 1)$ 。

注意事项

这是破坏性合并，不是可持久化。合并后不能再使用旧根副本。单点操作 $O(\log V)$ ，合并成本取决于重叠节点数；若想查询普通权值和，要改 `info/query` 的对外接口。

树上二分: `max_right` / `min_left`

```
pair<ll,info> max_right(int rt,ll l,F f);  
pair<ll,info> min_left(int rt,ll r,F f);
```

参数、效果与返回: 有效下标域为 $[0,n]$ 。两者返回边界与对应闭区间的累计信息, `info.s` 为权值和, `info.s1` 为下标乘权值之和。

`max_right(l,f)` 从 l 向右扩展, 返回最大的 r , 使 $f([l,r])$ 为真; 第一个元素就不满足时返回 $l-1$ 和空信息。允许从域的右端点加一开始, 此时结果为空。

`min_left(r,f)` 从 r 向左扩展, 返回最小的 l , 使 $f([l,r])$ 为真; 第一个元素就不满足时返回 $r+1$ 和空信息。允许从域的左端点减一开始。返回边界使用 `ll`, 避免在 `int` 极值下标处表示空区间时溢出。

判定要求: `f` 接受累计信息, 返回 `bool`; 空信息必须为真。沿对应方向扩展区间时只能由真变假, 不能变回真。`f` 不得修改树或累计信息, 也不应依赖调用次数。合并始终保持从左到右的顺序, 不要求 `info` 支持减法。正向判定单调不代表反向也单调。

复杂度: $O(\log n)$, 不额外做一次区间查询。下面是独立示例。

```
segt_merge sm; sm.setN(5); int rt=0; sm.add(rt,2,4);  
auto [r,x]=sm.max_right(rt,0,[](const auto &v){return v.s<=3;});
```

用区间和不超过阈值作为判定时, 需保证各点权值非负。`rt` 是要查询的树根, `0` 表示全零树; 不消耗或修改根。

3.22 data-structure/segment-tree/point.cpp

通用线段树（不带懒标记）

用途

默认维护区间和，下标 1..n，初值全零。递归部分只处理 info，按题目改写信息与标记的运算。

最小示例

```
segt t; t.setN(3); t.upd(2,5);
t.add(2,2); assert(t.ask(1,3)==7);
```

接口参考

1. 声明 / 调用形式

```
void set(); void set(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空；set(L,R) 建非空闭区间，全部叶子初值 0；setN(n)=set(1,n)。这里 $n \geq 1$ 。

复杂度： $O(R-L+1)$ 时间/空间。

2. 声明 / 调用形式

```
void upd(int i,ll v); void add(int i,ll v)
```

参数、效果与返回：upd 把第 i 项赋成 v；add 给它加 v。i 必须在当前域中。

复杂度： $O(\log n)$ 。

3. 声明 / 调用形式

```
ll ask(int l,int r); ll ask(int i)
```

参数、效果与返回：返回有效非空闭区间的和；单参数重载返回该点值。查询不会修改逻辑上的数组内容。

复杂度： $O(\log n)$ 。

4. 声明 / 调用形式

```
int find_first(ll x); int find_last(ll x)
```

参数、效果与返回：所有叶子非负且 $0 < x \leq$ 总和。find_first 找从左累加首次达到 x 的位置；find_last 从右累加首次达到 x 的位置。不存在答案会触发断言，没有失败哨兵。前两份通用结构由 USE_COMPARE 控制是否编译。

复杂度： $O(\log n)$ 。

5. 声明 / 调用形式

```
info
```

参数、效果与返回：改题时修改 info 的空值、叶子构造和合并，保持结合律；再改公开 upd/add/ask 的参数转换与答案提取。二分比较也须与新信息的单调性一致，不能只换 s 的含义。

复杂度：每次合并/标记按 $O(1)$ 设计。

注意事项

info + info 按左右顺序合并；upd 在叶子整体替换 info，单点 add 使用 info(delta,0)，通过 info += info 更新叶子。初值在 build 的 info(0,1) 处修改。参数转换、提取答案只改公开接口。USE_COMPARE 默认开启；自定义信息不支持减法与比较时，删除文件顶部的 define 行，对应运算和二分函数就不编译。默认二分要求所有点值非负、目标为正且不超过总和。set/setN 清空重建。两版同名 segt，只选一份。

改写示例：单点加、区间最大子段和（非空）删除顶部的 `#define USE_COMPARE`，把 `info` 替换为下面这段；区间 `ask` 的返回字段从 `s` 改为 `mx`，其他部分不改。`len=0` 的参数表示单点增量，`len=1` 表示叶子；合并的左右区间均非空。

```
struct info{
    ll s,pre,suf,mx,len;
    info(ll x=0,ll n=0):s(x),pre(x),suf(x),mx(x),len(n){}
    inline info& operator +=(const info &y){
        if(!y.len){ // point add: y is info(delta,0)
            s+=y.s;pre=suf=mx=s;return *this;
        }
        mx=max({mx,y.mx,suf+y.pre});
        pre=max(pre,s+y.pre);suf=max(y.suf,y.s+suf);
        s+=y.s;len+=y.len;return *this;
    }
    inline info operator +(const info &y)const{return info(*this)+=y;}
};
```

树上二分： `max_right` / `min_left`

```
pair<ll,info> max_right(ll l,F f);
pair<ll,info> min_left(ll r,F f);
```

参数、效果与返回：有效下标域为初始化时指定的 `[vl,vr]`，`setN(n)` 对应 `[1,n]`。两者返回边界与对应闭区间的累计信息，`info.s` 为区间和，`info.len` 为长度。

`max_right(l,f)` 从 `l` 向右扩展，返回最大的 `r`，使 `f([l,r])` 为真；第一个元素就不满足时返回 `l-1` 和空信息。允许从域的右端点加一开始，此时结果为空。

`min_left(r,f)` 从 `r` 向左扩展，返回最小的 `l`，使 `f([l,r])` 为真；第一个元素就不满足时返回 `r+1` 和空信息。允许从域的左端点减一开始。返回边界使用 `ll`，避免在 `int` 极值下标处表示空区间时溢出。

判定要求：`f` 接受累计信息，返回 `bool`；空信息必须为真。沿对应方向扩展区间时只能由真变假，不能变回真。`f` 不得修改树或累计信息，也不应依赖调用次数。合并始终保持从左到右的顺序，不要求 `info` 支持减法。正向判定单调不代表反向也单调。

复杂度： $O(\log n)$ ，不额外做一次区间查询。下面是独立示例。

```
segt st; st.setN(5);
auto [r,x]=st.max_right(1,[](const auto &v){return v.s<=10;});
```

用区间和不超过阈值作为判定时，需保证各点权值非负。

3.23 data-structure/segment-tree/range-add-max.cpp

区间加、区间最大值

用途

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

最小示例

```
MaxAdd_segt t; t.setN(3);
t.upd(2,5); t.add(1,2,2);
assert(t.ask(1,3)==7);
```

接口参考

1. 声明 / 调用形式

```
void set(); void set(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空；set(L,R) 建非空闭区间，全部叶子初值 0；setN(n)=set(1,n)。这里 $n \geq 1$ 。

复杂度： $O(R-L+1)$ 时间/空间。

2. 声明 / 调用形式

```
void upd(int i,ll v); void add(int i,ll v)
```

参数、效果与返回：upd 把第 i 项赋成 v ；add 给它加 v 。 i 必须在当前域中。

复杂度： $O(\log n)$ 。

3. 声明 / 调用形式

```
ll ask(int l,int r); ll ask(int i)
```

参数、效果与返回：返回有效非空闭区间的最大值；单参数重载返回该点值。查询不会修改逻辑上的数组内容。

复杂度： $O(\log n)$ 。

4. 声明 / 调用形式

```
void add(int l,int r,ll v)
```

参数、效果与返回：给闭区间各项加 v ；普通用法保证区间有效且非空。

复杂度： $O(\log n)$ 。

注意事项

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。set/setN 直接初始化全零节点，不递归建树。若初始数组不是全零，应逐点 upd；负数数组不需要改初始化，只要把所有位置正确赋值。三个类名不同，可以组合粘贴。

树上二分: max_right / min_left

```
pair<ll,ll> max_right(ll l,F f);
pair<ll,ll> min_left(ll r,F f);
```

参数、效果与返回: 有效下标域为初始化时指定的 $[vl, vr]$, $setN(n)$ 对应 $[1, n]$ 。两者返回边界与对应闭区间的累计信息, ll 为区间最大值, 空区间为 `LLONG_MIN`。

max_right(l,f) 从 l 向右扩展, 返回最大的 r , 使 $f([l, r])$ 为真; 第一个元素就不满足时返回 $l-1$ 和空信息。允许从域的右端点加一开始, 此时结果为空。

min_left(r,f) 从 r 向左扩展, 返回最小的 l , 使 $f([l, r])$ 为真; 第一个元素就不满足时返回 $r+1$ 和空信息。允许从域的左端点减一开始。返回边界使用 ll , 避免在 `int` 极值下标处表示空区间时溢出。

判定要求: f 接受累计信息, 返回 `bool`; 空信息必须为真。沿对应方向扩展区间时只能由真变假, 不能变回真。 f 不得修改树或累计信息, 也不应依赖调用次数。合并始终保持从左到右的顺序, 不要求 `info` 支持减法。正向判定单调不代表反向也单调。

复杂度: $O(\log n)$, 不额外做一次区间查询。下面是独立示例。

```
MaxAdd_segt st; st.setN(5);
auto [r,mx]=st.max_right(1,[])(ll x){return x<10;});
// r+1 is the first position with value >=10, if it exists.
```

3.24 data-structure/segment-tree/range-add-min.cpp

区间加、区间最小值

用途

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

最小示例

```
MinAdd_segt t; t.setN(3);
t.upd(2,5); t.add(1,2,2);
assert(t.ask(1,3)==0);
```

接口参考

1. 声明 / 调用形式

```
void set(); void set(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空；set(L,R) 建非空闭区间，全部叶子初值 0；setN(n)=set(1,n)。这里 $n \geq 1$ 。

复杂度： $O(R-L+1)$ 时间/空间。

2. 声明 / 调用形式

```
void upd(int i,ll v); void add(int i,ll v)
```

参数、效果与返回：upd 把第 i 项赋成 v ；add 给它加 v 。 i 必须在当前域中。

复杂度： $O(\log n)$ 。

3. 声明 / 调用形式

```
ll ask(int l,int r); ll ask(int i)
```

参数、效果与返回：返回有效非空闭区间的最小值；单参数重载返回该点值。查询不会修改逻辑上的数组内容。

复杂度： $O(\log n)$ 。

4. 声明 / 调用形式

```
void add(int l,int r,ll v)
```

参数、效果与返回：给闭区间各项加 v ；普通用法保证区间有效且非空。

复杂度： $O(\log n)$ 。

注意事项

这是区间加的特化实现；要更换信息或标记类型，使用外面的两份通用模板。set/setN 直接初始化全零节点，不递归建树。若初始数组不是全零，应逐点 upd；负数数组不需要改初始化，只要把所有位置正确赋值。三个类名不同，可以组合粘贴。

树上二分: max_right / min_left

```
pair<ll,ll> max_right(ll l,F f);
pair<ll,ll> min_left(ll r,F f);
```

参数、效果与返回: 有效下标域为初始化时指定的 $[vl, vr]$, $setN(n)$ 对应 $[1, n]$ 。两者返回边界与对应闭区间的累计信息, ll 为区间最小值, 空区间为 `LLONG_MAX`。

max_right(l,f) 从 l 向右扩展, 返回最大的 r , 使 $f([l, r])$ 为真; 第一个元素就不满足时返回 $l-1$ 和空信息。允许从域的右端点加一开始, 此时结果为空。

min_left(r,f) 从 r 向左扩展, 返回最小的 l , 使 $f([l, r])$ 为真; 第一个元素就不满足时返回 $r+1$ 和空信息。允许从域的左端点减一开始。返回边界使用 ll , 避免在 `int` 极值下标处表示空区间时溢出。

判定要求: f 接受累计信息, 返回 `bool`; 空信息必须为真。沿对应方向扩展区间时只能由真变假, 不能变回真。 f 不得修改树或累计信息, 也不应依赖调用次数。合并始终保持从左到右的顺序, 不要求 `info` 支持减法。正向判定单调不代表反向也单调。

复杂度: $O(\log n)$, 不额外做一次区间查询。下面是独立示例。

```
MinAdd_segt st; st.setN(5);
auto [l,mn]=st.min_left(5, [](ll x){return x>0;});
// l-1 is the last position <=5 with value <=0, if it exists.
```

3.25 data-structure/segment-tree/range-add-sum.cpp

区间加、区间和

用途

默认闭区间下标 1..n、初值全零，初始化 $O(n)$ ，单次修改查询 $O(\log n)$ 。

最小示例

```
AddSum_segt t; t.setN(3);
t.upd(2,5); t.add(1,2,2);
assert(t.ask(1,3)==9);
```

接口参考

1. 声明 / 调用形式

```
void set(); void set(int L,int R); void setN(int n)
```

参数、效果与返回：set 清空；set(L,R) 建非空闭区间，全部叶子初值 0；setN(n)=set(1,n)。这里 $n \geq 1$ 。

复杂度： $O(R-L+1)$ 时间/空间。

2. 声明 / 调用形式

```
void upd(int i,ll v); void add(int i,ll v)
```

参数、效果与返回：upd 把第 i 项赋成 v ；add 给它加 v 。 i 必须在当前域中。

复杂度： $O(\log n)$ 。

3. 声明 / 调用形式

```
ll ask(int l,int r); ll ask(int i)
```

参数、效果与返回：返回有效非空闭区间的和；单参数重载返回该点值。查询不会修改逻辑上的数组内容。

复杂度： $O(\log n)$ 。

4. 声明 / 调用形式

```
void add(int l,int r,ll v)
```

参数、效果与返回：给闭区间各项加 v ；普通用法保证区间有效且非空。

复杂度： $O(\log n)$ 。

5. 声明 / 调用形式

```
int find_first(ll x); int find_last(ll x)
```

参数、效果与返回：所有叶子非负且 $0 < x \leq$ 总和。find_first 找从左累加首次达到 x 的位置；find_last 从右累加首次达到 x 的位置。不存在答案会触发断言，没有失败哨兵。

复杂度： $O(\log n)$ 。

注意事项

这是区间加的特化实现：要更换信息或标记类型，使用外面的两份通用模板。set/setN 直接初始化全零节点，不递归建树。区间长度由递归参数计算，不存入节点；ask 和二分携带祖先标记，不下传、不改树。使用二分查询时所有点值必须非负，且 $1 \leq k \leq$ 总和； k 参数为 ll。三个类名不同，可以组合粘贴。

树上二分: max_right / min_left

```
pair<ll,ll> max_right(ll l,F f);  
pair<ll,ll> min_left(ll r,F f);
```

参数、效果与返回: 有效下标域为初始化时指定的 $[vl, vr]$, $setN(n)$ 对应 $[1, n]$ 。两者返回边界与对应闭区间的累计信息, ll 为区间和, 空区间为 0。

max_right(l,f) 从 l 向右扩展, 返回最大的 r , 使 $f([l, r])$ 为真; 第一个元素就不满足时返回 $l-1$ 和空信息。允许从域的右端点加一开始, 此时结果为空。

min_left(r,f) 从 r 向左扩展, 返回最小的 l , 使 $f([l, r])$ 为真; 第一个元素就不满足时返回 $r+1$ 和空信息。允许从域的左端点减一开始。返回边界使用 ll , 避免在 int 极值下标处表示空区间时溢出。

判定要求: f 接受累计信息, 返回 $bool$; 空信息必须为真。沿对应方向扩展区间时只能由真变假, 不能变回真。 f 不得修改树或累计信息, 也不应依赖调用次数。合并始终保持从左到右的顺序, 不要求 $info$ 支持减法。正向判定单调不代表反向也单调。

复杂度: $O(\log n)$, 不额外做一次区间查询。下面是独立示例。

```
AddSum_segt st; st.setN(5);  
auto [r, sum]=st.max_right(1, [] (ll x){return x<=10;});
```

用区间和不超过阈值作为判定时, 需保证各点权值非负。

3.26 data-structure/sparse-table.cpp

静态区间最值

用途

数组不修改、需要多次查询闭区间最大值时使用。预处理 $O(n \log n)$ ，查询 $O(1)$ ，空间 $O(n \log n)$ 。

最小示例

```
ST_table t; t.setN(3);
t.a[0][1]=4; t.a[0][2]=1; t.a[0][3]=8;
t.build(); assert(t.ask(1,2)==4);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); a[0][i]; void build()
```

参数、效果与返回: $n \geq 1$ ，先 setN，再填写 $a[0][1..n]$ ，最后 build。再次修改底层值后须重建。自带全局对象 S。

复杂度: $O(n \log n)$ 时间/空间。

2. 声明 / 调用形式

```
int chk(const int &x, const int &y); int ask(int l, int r)
```

参数、效果与返回: chk 默认 max，可改成 min 等幂等合并。ask 返回 1 起有效闭区间答案；普通求和不满足幂等条件，不能直接换成加法。

复杂度: ask $O(1)$ 。

注意事项

修改原数组后必须重新 build。文件末尾已有全局对象 S，可以直接用它，也可以另建局部对象。

4 图论 / graph

4.1 graph/block-cut-tree.cpp

点双连通分量与圆方树

用途

无向图的点双分解，支持非连通图和重边，不支持自环。构建时间 $O(n+m)$ 。

最小示例

```
block_cut g; g.setN(3);
g.add_edge(1,2); g.add_edge(2,3); g.build();
assert(g.scc==5 && g.tr[2].size()==2);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); void add_edge(int u,int v)
```

参数、效果与返回：清空并建 $1..n$ 无向图；逐条加边，自环禁止，允许重边。

复杂度：初始化 $O(n)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
void build()
```

参数、效果与返回：重建整个圆方森林，支持原图不连通，可重复调用；不需要自行调用 tarjan/dfs。

复杂度： $O(n+m)$ 。

3. 声明 / 调用形式

```
tr; ec; scc; ff; dep
```

参数、效果与返回：圆点为 $1..n$ ，方点为 $n+1..scc$ 。tr 是森林邻接表；ec[k] 为第 k 个点双的原图边，对应方点 $n+k$ 。孤立点仍是单独圆点。ff/dep 是森林父亲及深度；点双数量 $=scc-n$ 。

复杂度：读取 $O(1)$ ，遍历 $O(n+m)$ 。

注意事项

scc 是最终最大节点编号，不是分量个数，方点数应为 $scc-n$ 。割点可以属于多个点双，不能像 SCC 一样给每个原点分配唯一块号。当前构建使用显式栈；后续对圆方森林的遍历仍须考虑长链。

4.2 graph/chordal.cpp

弦图判定与完美消除序列

用途

判断无向图中是否不存在长度至少为 4 的无弦诱导环。如果是弦图，返回一个完美消除序列。

最小示例

```
Chordal g; g.setN(4);
g.add_edge(1,2); g.add_edge(2,3);
g.add_edge(3,4); g.add_edge(4,1);
assert(!g.solve() && g.peo.empty());
g.add_edge(1,3); assert(g.solve());
assert(g.peo.size()==4);
for(int i=0;i<4;i++)assert(g.rk[g.peo[i]]==i+1);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n); void add_edge(int u,int v)
```

参数、效果与返回：清空、建立 1..n 无向图、加边。自环忽略，重复边由 solve 归并。

复杂度：初始化 $O(n)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
bool solve(); peo; rk
```

参数、效果与返回：返回是否弦图；成功 `peo[0..n-1]` 为完美消除序列，`rk[u]` 是 1 起位置，满足 `peo[rk[u]-1]=u`。失败清空两数组；空图成功。

复杂度： $O(n+m)$ 。

注意事项

时间、空间均为 $O(n+m)$ ， m 为输入边数。MCS 用桶维护已选邻居数；验证时把结点按最靠前的后继邻居分组，统一检查后继邻居的包含关系，避免对每个邻域两两枚举。返回 false 时只给出判定，不构造无弦环证据。

4.3 graph/dijkstra.cpp

非负权单源最短路

用途

堆优化 Dijkstra，有向图，复杂度 $O((n+m) \log n)$ 。

最小示例

```
graph_SSSP g; g.setN(3); g.setS(1);  
g.add_edge(1,2,4); g.add_edge(2,3,6);  
auto d=g.SSSP(); assert(d[3]==10);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); int new_node(); void setS(int s)
```

参数、效果与返回：setN 清图并取消源点；new_node 保留图并返回新点编号；setS 指定已有源点。

复杂度：setN $O(n)$ ，加点均摊 $O(1)$ 。

2. 声明 / 调用形式

```
void add_edge(int u,int v,ll w)
```

参数、效果与返回：加有向边 $u \rightarrow v$ ， $w \geq 0$ ；无向边需调用两次。

复杂度：均摊 $O(1)$ 。

3. 声明 / 调用形式

```
vector<ll> SSSP()
```

参数、效果与返回：返回 $d[0..n]$ ， $d[s]=0$ ， $1..n$ 为顶点答案，不可达为 $1e18$ ；所有有限距离必须小于 $1e18$ 。不提供路径恢复。

复杂度： $O((n+m) \log(n+m+1))$ ，空间 $O(n+m)$ 。

注意事项

有限最短路必须小于不可达哨兵 10^{18} ，边权相加不能溢出。不支持负权；不会返回具体路径。

4.4 graph/directed-mst.cpp

指定根的最小树形图

用途

在带权有向图中，以 S 为根，选取 $n-1$ 条边，使 S 能沿选边到达每个点，且总权最小。每个非根点恰有一条入边。

最小示例

```
Directed_MST g; g.setN(4);
g.add_edge(1,2,5); g.add_edge(1,3,8);
g.add_edge(2,3,-2); g.add_edge(3,2,-1);
g.add_edge(3,4,1);
auto a=g.solve(1);
assert(a.ok && a.cost==4 && a.id[1]==0);
assert(a.id[2]==1 && a.id[3]==3 && a.id[4]==5);
assert(!g.solve(4).ok);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n); int add_edge(int u,int v,ll w)
```

参数、效果与返回：清空或建 $1..n$ 有向图；加边返回 1 起边 id，可有负权、重边和自环。目标是从根向外到达所有点的树形图。

复杂度：初始化 $O(n)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
result solve(int S) const
```

参数、效果与返回：返回 ok、cost、id。ok=false 表示无解；成功 cost 是 `__int128` 总权，id[v] 是 v 的入边 id，id[S]=0。不会修改原图，可换根求解。

复杂度： $O((n+m) \log(n+1))$ 。

3. 声明 / 调用形式

```
e; result::id
```

参数、效果与返回：e[id] 读取该边 u/v/w；通过结果入边编号可恢复树。无解时不要读取 cost/id 作为答案。

复杂度： $O(n)$ 恢复。

注意事项

设 n 为点数、 m 为输入边数，时间 $O((n+m) \log(n+1))$ ，空间 $O(n+m)$ ；任意显式图至少需要 $O(m)$ 读边，只有稀疏图才能写成 n 乘多对数。使用左偏堆、可回退并查集缩环并恢复选边。中间权值和 cost 都是 `__int128`；输出总权时先抄 basic/fast-io.cpp，不要直接强转 ll。solve 不修改原图，可更换根重新求解。

4.5 graph/edge-biconnected.cpp

边双连通分量及桥森林

用途

无向图删除桥后缩点，支持重边和非连通图。时间 $O(n+m)$ 。

最小示例

```
bi_edge g; g.setN(3);
g.add_edge(1,2); g.add_edge(1,2); g.add_edge(2,3);
auto e=g.build();
assert(g.bel[1]==g.bel[2] && g.bel[2]!=g.bel[3]);
assert(e.size()==1);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); void add_edge(int u,int v)
```

参数、效果与返回：清空并建 $1..n$ 无向图，允许不连通、自环、重边；每条无向边只调用一次。

复杂度：初始化 $O(n)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
vector<array<int,2>> build()
```

参数、效果与返回：分解边双，返回缩点森林边 {分量编号, 分量编号}，不是原点端点，也不是边 id。可重复调用。

复杂度： $O(n+m)$ 。

3. 声明 / 调用形式

```
bel; scnt
```

参数、效果与返回：bel[u] 为顶点 u 所属边双编号 $1..scnt$ ；原边两端 bel 不同就是桥。build 后使用。

复杂度： $O(1)$ 。

注意事项

桥森林的顶点是分量，不是原图点。与圆方树分别处理边双、点双，不能互换。

4.6 graph/flow/max-flow-hlpp.cpp

HLPP 最大流

用途

有向容量图的最大流。使用 HLPP：优先处理最高标号的活跃点，配合反向 BFS 全局重标号、当前弧和 gap 剪枝。邻接边编号连续存储，整个算法不使用递归。不能到达汇点的多余预流会退回源点，因此返回的边流量满足流量守恒。

最小示例

```
Flow_Graph g; g.setN(3); g.setST(1,3);
int id=g.add_edge(1,2,5); g.add_edge(2,3,3);
assert(g.max_flow()==3 && g.edge_flow(id)==3);
assert(g.max_flow()==3);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n)
```

参数、效果与返回：set 清空图、源汇及答案；setN 再建立 1..n。多个测试用例必须重置。

复杂度： $O(n+m)$ 。

2. 声明 / 调用形式

```
void setST(int S,int T); int new_node()
```

参数、效果与返回：S、T 为不同正编号；setST 必要时扩展点数。new_node 保留旧边，返回新增点编号。改变源汇会恢复原容量；源汇不变可继续增广。

复杂度：setST 最坏 $O(n+m)$ ，加点均摊 $O(1)$ 。

3. 声明 / 调用形式

```
int add_edge(int u,int v,ll cap)
```

参数、效果与返回：加有向容量边， $cap \geq 0$ ，返回正向边 id，保存用于 edge_flow。允许重边/自环；无向独立容量须加两次。

复杂度：均摊 $O(1)$ 。

4. 声明 / 调用形式

```
ll edge_flow(int id)
```

参数、效果与返回：查询最近求解后该原边的流量；参数必须用 add_edge 的返回值。图修改后先重新求解再读最终方案。

复杂度： $O(1)$ 。

5. 声明 / 调用形式

```
ll max_flow()
```

参数、效果与返回：返回总最大流，不是本次新增流量。未修改图时重复调用返回缓存；加边后从已有可行流继续求解。容量及总流量须能存入 ll。

复杂度：推送重标号；保守最坏 $O(n^2*m)$ ，空间 $O(n+m)$ 。

注意事项

重复求流返回总流量，不是新增流量；添加边后继续增广。setN 清空图，new_node 保留旧图添加点。无向容量边需要根据建模显式加边。每条边容量和总流量须能用 ll 表示，预流余量使用 `__int128`，源点出边容量之和可以超过 ll。更换源汇后从原始容量重算；设置相同源汇保留已有流。不同图上不保证比 Dinic 更快。

4.7 graph/flow/min-cost-flow-isap.cpp

最小费用最大流：Dijkstra + ISAP 备份

用途

作为网络单纯形版的替代实现。初始 SPFA 求势能，每轮 Dijkstra 求最短路，再在零约化费用子图上用非递归 ISAP 批量增广，包含反向 BFS、当前弧和 gap 剪枝。两份模板均使用 `Cost_Flow_Graph`，接口相同，选一份粘贴即可。

最小示例

```
Cost_Flow_Graph g; g.setN(3); g.setST(1,3);
int id=g.add_edge(1,2,2,-1); g.add_edge(2,3,2,3);
auto [f,c]=g.min_cost_flow();
assert(f==2 && c==4 && g.edge_flow(id)==2);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n)
```

参数、效果与返回：set 清空图、源汇及答案；setN 再建立 1..n。多个测试用例必须重置。

复杂度： $O(n+m)$ 。

2. 声明 / 调用形式

```
void setST(int S,int T); int new_node()
```

参数、效果与返回：S、T 为不同正编号；setST 必要时扩展点数。new_node 保留旧边，返回新增点编号。修改源汇后下次从头求解。

复杂度：setST 最坏 $O(n+m)$ ，加点均摊 $O(1)$ 。

3. 声明 / 调用形式

```
int add_edge(int u,int v,ll cap,ll cost)
```

参数、效果与返回：加有向容量边， $cap \geq 0$ ，返回正向边 id，保存用于 edge_flow。允许重边/自环；无向独立容量须加两次。费用可负但不能 `LLONG_MIN`，要求输入无负费用环。

复杂度：均摊 $O(1)$ 。

4. 声明 / 调用形式

```
ll edge_flow(int id)
```

参数、效果与返回：查询最近求解后该原边的流量；参数必须用 add_edge 的返回值。图修改后先重新求解再读最终方案。

复杂度： $O(1)$ 。

5. 声明 / 调用形式

```
pair<ll,ll> min_cost_flow()
```

参数、效果与返回：返回 {最大流, 其最小总费用}；加边、加点或设置源汇后，下次从原容量重新计算。无改动重复调用返回缓存。流量/答案须在 ll 范围内。

复杂度：首次势能最坏 $O(n*m)$ ；每个费用阶段保守 $O(n^2*m+(n+m)\log(n+m+2))$ 。

注意事项

允许负费用边，但整个输入图不能有负费用环。容量、总流量及增广过程中的总费用须能用 ll 表示；距离使用 `__int128`。初始 SPFA 最坏 $O(nm)$ ；设最短路费用阶段数为 R ，每阶段保守上界为 $O(n^2*m+(n+m)\log(n+m+2))$ ，总耗时还需乘阶段数并加初始化开销，不是总计 $O(nm)$ 。空间 $O(n+m)$ 。

4.8 graph/flow/min-cost-flow-simplex.cpp

最小费用最大流

用途

先求最大可行流量，再在最大流中最小化费用。使用网络单纯形：先建立残量生成森林，再通过环增广和生成树换基调整流量，复用未受换基影响的势能缓存。直接在原图残量边上计算，不设强制重启阈值，不使用递归。人工边费用取原图边权绝对值之和加一；证明势能运算不会溢出时使用 `ll`，否则使用 `__int128`。

最小示例

```
Cost_Flow_Graph g; g.setN(3); g.setST(1,3);
int id=g.add_edge(1,2,2,-1); g.add_edge(2,3,2,3);
auto [f,c]=g.min_cost_flow();
assert(f==2 && c==4 && g.edge_flow(id)==2);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n)
```

参数、效果与返回：set 清空图、源汇及答案；setN 再建立 1..n。多个测试用例必须重置。

复杂度： $O(n+m)$ 。

2. 声明 / 调用形式

```
void setST(int S,int T); int new_node()
```

参数、效果与返回：S、T 为不同正编号；setST 必要时扩展点数。new_node 保留旧边，返回新增点编号。修改源汇后下次从头求解。

复杂度：setST 最坏 $O(n+m)$ ，加点均摊 $O(1)$ 。

3. 声明 / 调用形式

```
int add_edge(int u,int v,ll cap,ll cost)
```

参数、效果与返回：加有向容量边， $cap \geq 0$ ，返回正向边 id，保存用于 edge_flow。允许重边/自环；无向独立容量须加两次。费用可负但不能 `LLONG_MIN`，要求输入无负费用环。

复杂度：均摊 $O(1)$ 。

4. 声明 / 调用形式

```
ll edge_flow(int id)
```

参数、效果与返回：查询最近求解后该原边的流量；参数必须用 add_edge 的返回值。图修改后先重新求解再读最终方案。

复杂度： $O(1)$ 。

5. 声明 / 调用形式

```
pair<ll,ll> min_cost_flow()
```

参数、效果与返回：返回 {最大流, 其最小总费用}；加边、加点或设置源汇后，下次从原容量重新计算。无改动重复调用返回缓存。流量/答案须在 ll 范围内。

复杂度：网络单纯形，最坏不保证多项式时间；空间 $O(n+m)$ 。

注意事项

允许负费用边，但输入图不能有负费用环。没有指定流量参数；要限制流量，增加一个容量为上限的新源边。更改图后从原始容量重算，不改图返回缓存。容量、总流量和费用答案须能用 ll 表示；只汇总原图边费用，避免人工边的大费用参与乘法累加。每条边流量均可查询。运行时间取决于换基次数，退化数据仍可能较慢；空间为 $O(n + m)$ 。

4.9 graph/matching/bipartite.cpp

二分图最大匹配

用途

给定左右两部，多次加边后求最大基数匹配，返回匹配的点对。先贪心匹配，再用多源 BFS 记录起点与前驱，遇到未匹配右点就立即增广；整个过程不递归。

最小示例

```
Bipartite_Matching g; g.setN(3,3);
g.add_edge(1,1); g.add_edge(2,1);
g.add_edge(2,2); g.add_edge(3,2);
auto a=g.solve(); assert(a.size()==2);
g.add_edge(3,3); a=g.solve(); assert(a.size()==3);
for(auto [u,v]:a){
    assert(g.matchL[u]==v && g.matchR[v]==u);
}
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int L,int R); void add_edge(int u,int v)
```

参数、效果与返回：清空，或建左右两部 1..L、1..R；只从左向右登记边，允许重边。

复杂度：初始化 $O(L+R)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
vector<pair<int,int>> solve()
```

参数、效果与返回：重新求最大匹配，返回匹配端点 {左点, 右点}，按左点升序。返回 vector 大小就是匹配数，空图可用。

复杂度：每轮多源 BFS $O(L+R+m)$ ，最多 $\min(L,R)$ 个增广轮次。

3. 声明 / 调用形式

```
matchL; matchR
```

参数、效果与返回：solve 后双方匹配点，0 表示未匹配。新增边后要再 solve；该接口不承诺增量复杂度。

复杂度：单次读取 $O(1)$ 。

注意事项

邻接表在 solve 内按左点压成连续存储，保持各点的加边顺序。rt[u] 记录搜索树的未匹配起点，pre[u] 记录增广前驱；某起点完成增广后，跳过该搜索树剩余队列项。一轮允许处理不同长度的增广路。设总点数 $V=n+m$ 、边数 E ，空间 $O(V+E)$ ，每轮 $O(n+E)$ ；保守最坏时间界为 $O((V+E)(\min(n,m)+1))$ ，不沿用分层 HK 的复杂度保证。空部、空边集允许，返回空匹配。每次 solve 都重新计算；可继续加边再调用。

4.10 graph/matching/general.cpp

无权一般图最大匹配

用途

带花树，允许奇环，返回匹配边数，最坏 $O(n^3)$ 。

最小示例

```
blossom g; g.setN(4);
g.add_edge(1,2); g.add_edge(2,3);
g.add_edge(3,1); g.add_edge(3,4);
assert(g.solve()==2);
assert(g.match[g.match[1]]==1);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); void add_edge(int u,int v)
```

参数、效果与返回：建立 $1..n$ 无向图；每条边加一次。

复杂度：初始化 $O(n)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
int solve(); match
```

参数、效果与返回：返回最大匹配边数；`match[u]` 是配对顶点，0 未匹配。输出边集只取 $u < \text{match}[u]$ 。完美匹配条件为 $2 * \text{solve()} == n$ 。

复杂度：保守 $O(n^3)$ 。

注意事项

这是无权匹配，不会最小化或最大化边权。点数为奇数时不可能完全匹配。重复 `solve` 会重新求解。

4.11 graph/matching/regular-bipartite.cpp

正则二分图的完美匹配分解

用途

两部各 n 个点，每个点度数均为 d ，将全部 nd 条边恰好分成 d 组完美匹配。支持多重边，同一对点之间的边按重数计入分解。

最小示例

```
Regular_Bipartite g; g.setN(2);
g.add_edge(1,1); g.add_edge(1,2);
g.add_edge(2,1); g.add_edge(2,2);
auto a=g.solve(); assert(a.size()==2);
for(auto &match:a){
    assert(match.size()==3 && match[0]==0);
    assert(1<=match[1] && match[1]<=2);
    assert(1<=match[2] && match[2]<=2);
    assert(match[1]!=match[2]);
}
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n); void add_edge(int u,int v)
```

参数、效果与返回：建左右各 n 点的二分多重图，两侧顶点都从 1 起。每次 `add_edge` 代表独立一条边，重复边保留。

复杂度：初始化 $O(n)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
vector<vector<int>>> solve() const
```

参数、效果与返回：前提：两侧每点度数都是同一 d 。返回 d 行，每行 $n+1$ 项， $a[c][u]$ 是第 c 组中左点 u 的右配点； c 从 0 起，第 0 列不用。边按重数恰好用完； $d=0$ 返回空。

复杂度： $O(n+nd \log(nd+1))$ 时间， $O(n+nd)$ 空间。

注意事项

确定性算法，时间 $O(n + nd \log(nd + 1))$ ，包括结果在内空间 $O(n + nd)$ 。偶数度用欧拉分边；奇数度通过整数重数缩减找完美匹配，再借用已分解的匹配，把另一分支凑成二的幂次度数。无随机失败概率。非正则图不属于此接口，不要关闭断言后直接输入；重复 `solve` 不修改原边集。

4.12 graph/steiner-tree.cpp

最小斯坦纳树

用途

无向非负权图中，求连接给定关键点的最小费用子图，允许使用其他点。设关键点去重后有 k 个，子集 DP 加多源 Dijkstra 的时间为 $O(n3^k + m \log(m+2)2^k)$ ，空间为 $O(n2^k + n + m)$ 。固定一个关键点作为根，实际只为剩余 $k-1$ 个点开子集状态。

最小示例

```
Steiner_Tree g; g.setN(4);
g.add_edge(1,4,2); g.add_edge(2,4,2); g.add_edge(3,4,2);
assert(g.solve({1,2,3})==6); // use vertex 4 as a Steiner point
assert(g.solve({1,1})==0);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n); void add_edge(int u,int v,ll w)
```

参数、效果与返回：建立 1..n 无向图；每条边加一次， $0 \leq w < \text{LLONG_MAX}$ 。

复杂度：初始化 $O(n)$ ，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
ll solve(vector<int> terminals) const
```

参数、效果与返回：连通所有终端的最小总边权，允许经过其他点。终端自动去重；0/1 个终端返回 0；无法连通返回 -1。只返回费用，不恢复边集。

复杂度： k 为不同终端数： $O(n \cdot 3^k + m \log(m+2) \cdot 2^k)$ ，空间 $O(n \cdot 2^k + n + m)$ 。

注意事项

空关键点集或只有一个不同关键点时返回 0，重复关键点自动去重。支持零权、重边和自环，不支持负权。边权和有限答案必须小于 LLONG_MAX ；中间状态超过范围时会被截断，不会有整数加法溢出。只返回费用，不恢复边集。每次 solve 独立计算，可更换关键点或加边后重算； k 较大时，先估算指数级时间和内存。

4.13 graph/two-sat.cpp

布尔约束求解

用途

求满足所有二元或子句的一组赋值，时间与空间 $O(n+m)$ 。

最小示例

```
two_sat s; s.setN(2);
s.add(1,1,1,1); // force x1=true
s.add(1,0,2,1); // x1 implies x2
auto ans=s.solve();
assert(!ans.empty() && ans[1] && ans[2]);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n)
```

参数、效果与返回：清空或建立 1..n 个 bool 变量，删除旧子句；不是增量扩容。

复杂度： $O(n)$ 。

2. 声明 / 调用形式

```
void add(int x1,int o1,int x2,int o2)
```

参数、效果与返回：加子句 $(x1==o1) \text{ OR } (x2==o2)$ ， $o1/o2$ 取 0 或 1。强制 $x=v$ 可 $\text{add}(x,v,x,v)$ 。

复杂度：均摊 $O(1)$ 。

3. 声明 / 调用形式

```
vector<int> solve()
```

参数、效果与返回：无解返回空；有解返回 $n+1$ 项， $\text{ans}[i]$ 是变量 i 的 0/1 值，第 0 项不用。0 个变量有解返回长度 1。

复杂度： $O(n + \text{子句数})$ 。

注意事项

只有“或”子句接口；例如强制 $x=a$ 可重复同一文字。setN 清空约束。不要把 add 当作直接添加两条蕴含边。

5 树 / tree

5.1 tree/cartesian-tree.cpp

小根堆笛卡尔树

用途

$O(n)$ 构建，同时满足下标的中序顺序与值的小根堆序，空间 $O(n)$ 。

最小示例

```
vector<int> a{3,1,2,1};
auto [ls,rs]=cartesian(a);
int rt=min_element(a.begin(),a.end())-a.begin();
assert(rt==1 && ls[rt]==0 && rs[rt]==3 && ls[3]==2);
```

接口参考

1. 声明 / 调用形式

```
pair<vector<int>,vector<int>> cartesian(const vector<T> &a)
```

参数、效果与返回：对 0 起数组建小根堆笛卡尔树，返回 ls、rs，缺儿子用 -1；相等时较早下标在上方。根是第一个最小值下标；空数组返回两个空 vector。

复杂度： $O(n)$ 时间/空间。

注意事项

空输入返回两个空 vector，此时没有根。构造无递归，单调数组形成长链也可直接构建；后续遍历要自行避免递归爆栈。类型只需支持小于比较。返回值不含原数组或额外哨兵。

5.2 tree/centroid-decomposition.cpp

点分治框架

用途

提供寻找重心、递归处理剩余连通块的框架。与 HLD 查询共用 Tree，题目需要的统计尚未实现。

最小示例

```
Tree t; t.setN(3);
t.add_edge(1,2); t.add_edge(2,3);
t.HLD_build(); assert(t.dist(1,3)==2);
t.DC_build(); // traversal only, no problem answer
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); void add_edge(int u,int v); int rt
```

参数、效果与返回：重置后添加无向树边；根默认 1，可直接设 rt。输入须为连通树。与 HLD 文件都叫 Tree，只选一份或改名。

复杂度：初始化 $O(n)$ 。

2. 声明 / 调用形式

```
void HLD_build(); int lca(int u,int v); int dist(int u,int v); int kth_anc(int u,int k)
```

参数、效果与返回：HLD_build 在当前根下预处理；LCA/距离/祖先同 HLD，距离为边数， $0 \leq k < \text{dep}[u]$ 。

复杂度：建表 $O(n \log n)$ ，LCA/距离 $O(1)$ ，祖先 $O(\log n)$ 。

3. 声明 / 调用形式

```
void DC_build()
```

参数、效果与返回：清除分治访问状态，从整棵树开始点分治；调用你填写的 DC/get 逻辑，不直接返回题目答案。

复杂度：框架 $O(n \log n)$ ，另加题目处理代价。

4. 声明 / 调用形式

```
void get(int u,int fa); void DC(int u)
```

参数、效果与返回：改写入口：get 收集未删除连通块信息；DC 处理以当前重心为中心的贡献，再递归。按题意做子树贡献排重，不能把空框架当成求解器。

复杂度：由改写逻辑决定。

5. 声明 / 调用形式

```
vis; cen; csiz; msiz; tsiz; void output(); void solve()
```

参数、效果与返回：vis 标记已选重心，cen 当前重心，csiz/msiz 是找重心的工作数组，tsiz 当前块大小。output 调试；solve 只是读取父亲列表的示范流程，提交前按题意改写。

复杂度：不作为独立答案接口。

5.3 tree/divide-combine-tree.cpp

析合树：返回完整结构

用途

对 $1..n$ 的排列构建析合树。构建时间 $O(n \log n)$ ，包含四毛子 RMQ、辅助线段树和返回结构在内的空间均为 $O(n)$ 。

最小示例

```
xihe_tree tr; auto t=tr.build(vector<int>{3,1,2});
int u=t.rt;
assert(t.L[u]==1 && t.R[u]==3);
assert(t.mn[u]==1 && t.mx[u]==3);
assert(t.typ[u]==xihe_tree::DEC && t.son[u].size()==2);
assert(t.mn[t.id[2]]==1);
```

接口参考

1. 声明 / 调用形式

```
void set(); void setN(int n); vector<int> a
```

参数、效果与返回：set 清空；setN 分配 $a[0..n]$ ，填写 $a[1..n]$ 为 $1..n$ 的排列。 $a[0]$ 不用；允许 $n=0$ 。

复杂度： $O(n)$ 。

2. 声明 / 调用形式

```
result build() const; result build(const vector<int> &v)
```

参数、效果与返回：无参版本使用 $a[1..n]$ ；带参版本接受没有哨兵的 v ，重设 n 后建树。返回值独立拥有结果，外部自行做树上 DP。

复杂度： $O(n \log n)$ 时间， $O(n)$ 空间。

3. 声明 / 调用形式

```
result::rt; cnt; son; typ; L; R; mn; mx; id
```

参数、效果与返回：rt 为根，空输入 $rt=0$ ；节点编号 $1..cnt$ 。son[u] 按原位置排序；typ 为 LEAF/INC/DEC/PRIME。L/R 为 1 起闭区间，mn/mx 为值域端点；id[i] 为原位置 i 对应的叶子。

复杂度：结果总大小 $O(n)$ 。

注意事项

输入必须恰为 $1..n$ 的排列，不接受重复值；必要时在外部离散化，但需确认题目是否只关心排名连续。返回的结点编号、下标区间、id 均从 1 开始，0 不使用。 $n=0$ 时 $rt=cnt=0$ 。INC/DEC 是儿子值域顺序递增/递减的合点，PRIME 是析点。不要假定父编号大于儿子编号；在返回结果上自行遍历、DP，构建器不含 solve。再次 build 或修改构建器不会改变已返回结果。

5.4 tree/heavy-light-decomposition.cpp

LCA、祖先和路径分段

用途

对一棵连通无向树预处理，支持 $O(1)$ LCA 和 $O(\log n)$ 路径分段。

最小示例

```
Tree t; t.setN(3);
t.add_edge(1,2); t.add_edge(1,3); t.build();
assert(t.lca(2,3)==1 && t.dist(2,3)==2);
int cnt=0;
for(auto [l,r]:t.index_of_path(2,3))cnt+=r-l+1;
assert(cnt==3);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); void setRt(int r); void add_edge(int u,int v)
```

参数、效果与返回：重置树，默认根为 1；setRt 改根；加无向边。输入必须是一棵连通树， $n \geq 1$ 。

复杂度： $O(n)$ 初始化，加边均摊 $O(1)$ 。

2. 声明 / 调用形式

```
void build()
```

参数、效果与返回：加完边、设置根后调用；计算重链、DFS 序及 LCA 表。改变根或树结构后重建。

复杂度： $O(n \log n)$ 时间/空间。

3. 声明 / 调用形式

```
int lca(int u,int v); int dist(int u,int v); int kth_anc(int u,int k)
```

参数、效果与返回：LCA 返回最近公共祖先；dist 返回边数；kth_anc 返回第 k 级祖先， $k=0$ 返回自己，要求 $0 \leq k < \text{dep}[u]$ 。

复杂度：lca/dist $O(1)$ ，祖先 $O(\log n)$ 。

4. 声明 / 调用形式

```
vector<array<int,2>> index_of_path(int u,int v)
```

参数、效果与返回：返回覆盖点路径（含两端）的若干 DFS 序闭区间，区间内左端 $\leq$ 右端；这些段不保证沿 $u \rightarrow v$ 的方向排序，不能直接用于非交换路径合并。

复杂度： $O(\log n)$ 段及时间。

5. 声明 / 调用形式

```
in; out; dfn; ff; dep; siz; hson; top; ed; void output()
```

参数、效果与返回：in 为点到 DFS 序，dfn 是逆映射；子树为 $[\text{in}[u], \text{out}[u]]$ ；ff 父亲、dep 深度（根为 1）、siz 子树大小、hson 重儿子、ed[u] 是 u 所在重链向下延伸的末点、top 链顶。output 向 cerr 输出调试信息。

复杂度：读取 $O(1)$ ，output $O(n)$ 。

注意事项

配线段树时，把点 u 的值放在位置 $\text{in}[u]$ ，不要直接用 u 。路径段没有保证从 u 到 v 的方向顺序，只能直接用于可交换聚合。按边统计时要另行排除 LCA。与 centroid-decomposition.cpp 都定义 Tree。

6 字符串 / string

6.1 string/aho-corasick.cpp

多模式串 AC 自动机

用途

一次加入多个模式串，再统计它们在文本中的出现次数。允许重叠出现；构建 $O(26 \times \text{节点数})$ ，每次 count 为 $O(\text{文本长度} + \text{节点数})$ 。

最小示例

```
AC ac; int a=ac.insert(" aba"),b=ac.insert(" ba");
ac.build(); auto cnt=ac.count(" ababa");
assert(cnt[a]==2 && cnt[b]==2);
```

接口参考

1. 声明 / 调用形式

```
AC(); void set(int m=0)
```

参数、效果与返回：创建空自动机；set 清空，m 为预留节点数量而非字符串长度上限。根为 0。

复杂度：与旧节点数和预留空间相关。

2. 声明 / 调用形式

```
int insert(const string &s)
```

参数、效果与返回：s 带前导空格，其后为小写字母；返回该模式串末节点 id，相同模式可能返回同一个 id。必须在 build 前插入。

复杂度： $O(|s|)$ 。

3. 声明 / 调用形式

```
void build()
```

参数、效果与返回：建 fail 并补全转移。此后 ch 是自动机转移，不再只是 Trie 儿子；要新增模式请 set 后重建。

复杂度： $O(26 \times \text{节点数})$ 。

4. 声明 / 调用形式

```
vector<ll> count(const string &s) const
```

参数、效果与返回：文本带前导空格、小写；返回按节点编号的出现次数，模式答案取 cnt[insert 返回的 id]，计重叠匹配。

复杂度： $O(|s| + \text{节点数})$ 。

5. 声明 / 调用形式

```
ch; fail; tot
```

参数、效果与返回：ch[u][c-'a'] 为转移；扫描文本时从根 0 沿 ch 前进，可结合 fail 树自写处理。tot 为最大节点编号，数组按 0..tot 使用。

复杂度：单次转移 $O(1)$ 。

注意事项

模式和文本都只支持小写字母，前导空格不参加匹配。重复模式串返回同一编号，但它们各自的答案仍是该模式在文本中的出现次数。空模式返回根编号 0，次数为文本长度 +1。不同 count 之间不会累计。

6.2 string/generalized-suffix-automaton.cpp

广义后缀自动机

用途

维护多个小写串的所有子串；出现次数是这些串中次数之和，重复插入的串也重复计数，不允许跨串边界匹配。

最小示例

```
GSAM t; t.insert(" aba"); t.insert(" bab");
t.build(); assert(t.count(" ab")==2);
assert(t.count(" abab")==0 && t.distinct()==6);
t.insert(" aba"); t.build(); assert(t.count(" aba")==2);
```

接口参考

1. 声明 / 调用形式

```
GSAM(); void set(int m=0); int insert(const string &s)
```

参数、效果与返回：set 清空并可预留节点；insert 加入一条带前导空格的小写串，返回末状态。重复插入会重复计数；全部插完再 build。

复杂度：与总串长、自动机状态数相关。

2. 声明 / 调用形式

```
void build()
```

参数、效果与返回：沿 suffix link 汇总出现次数，允许重复 build；新插串后计数需重新 build。

复杂度： $O(\text{状态数} + \text{最长串长})$ 。

3. 声明 / 调用形式

```
int find(const string &s) const; ll count(const string &s) const
```

参数、效果与返回：find 返回匹配状态，找不到 -1；count 返回所有已插串中的总出现次数，找不到 0，须先 build。空模式写成单个空格，count 等于各串的长度 +1 之和。

复杂度： $O(|s|)$ 。

4. 声明 / 调用形式

```
ll distinct() const
```

参数、效果与返回：所有已插串中不同的非空子串数量，不按出现次数重复计算。

复杂度： $O(\text{状态数})$ 。

5. 声明 / 调用形式

```
ch; len; fail; siz; tot
```

参数、效果与返回：根 0；状态 u 代表长度 $(\text{len}[\text{fail}[u]], \text{len}[u])$ 的同类子串；siz 为 build 后次数。不要直接调用 clone/new_node 改结构。

复杂度：读取 $O(1)$ 。

注意事项

count 不是“包含该子串的字符串个数”。根为 0，fail[0]=-1；别把普通 SAM 简单重置 lst 的做法当成这里的插入实现。set(m) 的 m 用于预留总字符量级的空间，build 的计数按当前全部插入内容重新计算。

6.3 string/kmp-z.cpp

前缀函数和 Z 函数

用途

线性时间计算字符串的 border 与各后缀和全串的公共前缀长度。

最小示例

```
auto pi=border(" ababa"); auto z=z_function(" ababa");
assert(pi[5]==3 && z[3]==3);
```

接口参考

1. 声明 / 调用形式

```
vector<int> border(string s)
```

参数、效果与返回：s 带前导空格。返回 1 起前缀函数，b[i] 为 s[1..i] 的最长真 border 长度。沿 j=b[j] 枚举更短 border。

复杂度： $O(n)$ 时间/空间。

2. 声明 / 调用形式

```
vector<int> z_function(string s)
```

参数、效果与返回：返回 $z[i]=LCP(s[1..n],s[i..n])$ ，下标从 1 起；实现保留 $z[1]=0$ 占位；需要自身 LCP 时自行取 n，第 0 项不用。空内容写成” ”。

复杂度： $O(n)$ 时间/空间。

注意事项

这个实现 $z[1]=0$ ，而不是全串长度。没有封装“模式串在文本的所有位置”：可自行 KMP 扫描，或拼接 pattern+分隔符+text 后用 Z；分隔符不能出现在两个原串里。

6.4 string/lyndon.cpp

Lyndon 分解

用途

用 Duval 将串唯一分解为字典序不增的 Lyndon 串。Lyndon 串严格小于自己的每个非空真后缀，相同因子可以连续出现。

最小示例

```
string s=" ababab"; auto a=lyndon(s);
assert((a==vector<pair<int,int>>{{1,2},{3,4},{5,6}}));
assert((lyndon(" cba")==vector<pair<int,int>>{{1,1},{2,2},{3,3}}));
assert(lyndon(" ").empty());
```

接口参考

1. 声明 / 调用形式

```
vector<pair<int,int>> lyndon(const string &s)
```

参数、效果与返回：带前导空格：返回从左到右的 Lyndon 因子闭区间 $\{l,r\}$ ，覆盖 $1..n$ ，因子按字典序非增。用 `s.substr(l,r-l+1)` 取内容。空内容返回空。

复杂度： $O(n)$ 时间，除输出外 $O(1)$ 空间。

注意事项

时间 $O(n)$ ，除返回区间外额外空间 $O(1)$ 。比较按 `unsigned char` 的字节顺序，可处理任意字节；不要求小写字母。同样的因子不会合并，`ababab` 分为三个 `ab`，而 `aaaa` 分为四个 `a`。空串返回空向量。本接口是整串分解，不是每个位置的最长 Lyndon 前缀数组。

6.5 string/manacher.cpp

Manacher 回文半径

用途

线性求出所有奇数、偶数长度回文的最大半径，并以 $O(1)$ 判断某个子串是否回文。

最小示例

```
Manacher t; t.build(" abba");
assert(t.d2[3]==2 && t.d1[2]==1);
assert(t.ask(1,4) && t.ask(2,3));
assert(!t.ask(1,3));
long long cnt=0;
for(int i=1;i<=t.n;i++)cnt+=t.d1[i]+t.d2[i];
assert(cnt==6);
```

接口参考

1. 声明 / 调用形式

```
void set(); void build(const string &s)
```

参数、效果与返回：set 清空；build 接受前导空格字符串，重建两种回文半径。内容可为空。

复杂度： $O(n)$ 时间/空间。

2. 声明 / 调用形式

```
d1; d2
```

参数、效果与返回：奇回文中心 i 的区间 $[i-d1[i]+1, i+d1[i]-1]$ ；偶回文中心在 $i-1$ 与 i 之间，区间 $[i-d2[i], i+d2[i]-1]$ 。点编号 $1..n$ 。

复杂度：读取 $O(1)$ 。

3. 声明 / 调用形式

```
bool ask(int l, int r) const
```

参数、效果与返回：判断非空有效闭区间 $[l, r]$ 是否回文；须先 build。

复杂度： $O(1)$ 。

注意事项

时间、空间均为 $O(n)$ ；不插入分隔字符，可处理任意字节。奇偶半径含义不同，不能直接把 $d1$ 当成回文长度。空串允许 build，但没有可查询的非空区间。求最长回文取所有 $2*d1[i]-1$ 与 $2*d2[i]$ 的最大值，空串答案为 0。

6.6 string/palindromic-automaton.cpp

回文自动机

用途

在线维护不同回文子串、最长回文后缀及回文出现次数。字符集为小写字母，建树与空间均为 $O(n)$ 。

最小示例

```
PAM p; p.build(" ababa");
auto cnt=p.count();
assert(p.tot-1==5 && p.len[p.lst]==5);
assert(p.num[p.lst]==3 && cnt[p.lst]==1);
int u=p.ch[1]['a'-'a']; assert(cnt[u]==3);
p.extend('b'); cnt=p.count(); // still safe after count()
assert(p.len[p.lst]==5 && cnt[u]==3);
```

接口参考

1. 声明 / 调用形式

```
PAM(); void set(int m=0); void build(string s)
```

参数、效果与返回：set 清空、预留容量；build 自动重置并逐字符加入，s 为带前导空格的小写串。

复杂度： $O(n)$ 时间/空间。

2. 声明 / 调用形式

```
int extend(char c)
```

参数、效果与返回：在当前串尾追加一个小写字母，返回当前最长回文后缀节点；可 set 后在线逐个调用。不是任意位置插入。

复杂度：总计线性，单次可能沿 fail 跳多次。

3. 声明 / 调用形式

```
vector<int> count() const
```

参数、效果与返回：返回每个真实回文节点的出现次数，包含重叠出现；可反复调用，不会重复累加内部计数。

复杂度： $O(\text{节点数})$ 。

4. 声明 / 调用形式

```
len; fail; num; pos; lst; tot; ch
```

参数、效果与返回：0/1 是虚根，真实节点 2..tot；len 是回文长度，fail 是最长真回文后缀，num 是不同回文后缀数量，pos 是首次出现右端点。lst 当前最长后缀；共有 tot-1 种非空回文。

复杂度：单次读取 $O(1)$ 。

注意事项

siz 只记录结点作为最长后缀的次数，不能直接作为总出现次数。count 在副本上沿 fail 累加，单次 $O(n)$ ，可以重复调用，也能继续 extend；不要每加入一个字符都调用 count。所有回文出现次数之和用 ll 保存。空串写作一个空格。

6.7 string/rolling-hash.cpp

双模前缀哈希

用途

提供子串指纹与拼接，不是无碰撞的字符串判等。构建前缀数组 $O(n)$ ，常规子串取值 $O(1)$ 。

最小示例

```
auto h=Hash_of(" ababa");
auto a=h[3]-h[0],b=h[5]-h[2];
assert(a.l==b.l && a.v()==b.v());
```

接口参考

1. 声明 / 调用形式

```
void init_hash(int n=0); vector<Hash> Hash_of(const string &s)
```

参数、效果与返回：init_hash 扩展幂表，可预处理最大长度；Hash_of 接受前导空格串，返回 $H[0..n]$ ， $H[0]$ 为空串。底数在程序启动时随机选定，不随 init_hash 改变。

复杂度：预处理 $O(n)$ ，建前缀 $O(n)$ 。

2. 声明 / 调用形式

```
Hash{}; Hash operator+(const Hash &a,const Hash &b); Hash operator-(const Hash &a,const Hash &b)
```

参数、效果与返回：默认空串； $a+b$ 拼接； $a-b$ 删除 a 的前缀 b ，调用者须保证 b 确为前缀。子串 $[l,r]$ 为 $H[r]-H[l-1]$ 。

复杂度：幂表就绪后 $O(1)$ 。

3. 声明 / 调用形式

```
h+x; h+=x
```

参数、效果与返回： x 为字符/非负整数编码时追加一个符号； x 为 Hash 时拼接整段。保持编码可在两个模数下安全计算，普通字符串已按 unsigned char 处理。

复杂度：单符号 $O(1)$ ，拼接可能扩展幂表。

4. 声明 / 调用形式

```
int Hash::l; ll Hash::v() const; h1; h2
```

参数、效果与返回： l 为长度， v 打包双模哈希。比较内容先比较长度，再比较 $v()$ ；哈希可能碰撞，不是确定性相等判定。

复杂度： $O(1)$ 。

注意事项

比较子串时同时比较长度和哈希。底数每个进程随机，不能把数值用于跨进程永久存储。减法只表示删去前缀，不是任意删除；Hash 对象应使用 {} 零初始化。

6.8 string/runs.cpp

Runs 极大周期子串

用途

枚举所有长度至少为最小周期两倍、且不能保持该周期向左或向右扩展的子串。返回若干三元组 (l, r, p) ，其中 p 必须是最小周期。

最小示例

```
Runs t; auto a=t.build(" ababab");
assert((a==vector<array<int,3>>{{1,6,2}}));
a=t.build(" aabaa");
assert((a==vector<array<int,3>>{{1,2,1},{4,5,1}}));
assert(t.build(" abc").empty());
```

接口参考

1. 声明 / 调用形式

```
vector<array<int,3>> Runs::build(const string &s) const
```

参数、效果与返回： s 有前导空格，字符编码 1..255。返回排序去重后的 $\{l, r, p\}$ ：1 起闭区间 $[l, r]$ ，最小周期 p ，长度 $\geq 2p$ ，左右都不能保持该周期继续扩展。空/单字符串返回空。

复杂度： $O(n \log n)$ 时间， $O(n)$ 空间。

注意事项

实际字符限字节 1..255，0 留给 SA 哨兵。采用两种字母序下的最长 Lyndon 前缀，加 SA_LCP、SA_LCS 向两侧扩展；LCP 使用四毛子，不使用哈希。源码版时间 $O(n \log n)$ （最后排序），空间 $O(n)$ ；考场倍增 SA 也满足此界。Runs 不包括所有周期区间，只给出极大区间；长度不必是 p 的倍数，例如 ababa 返回 (1,5,2)。

6.9 string/suffix-array.cpp

后缀数组、LCP 与 LCS

用途

先粘贴 data-structure/linear-rmq.cpp。SA-IS 构建后缀排序，SA_LCP 用 height 和线性 RMQ 查询后缀公共前缀；SA_LCS 在反串上查询两个前缀的公共后缀。

最小示例

```
SA sa; sa.build(" banana");
assert(sa.sa[1]==6);
SA_LCP l; l.build(sa);
assert(l.lcp(2,4)==3); // anana / ana
assert(l.lcp(3,3)==4);
SA_LCS r; r.build(sa);
assert(r.lcs(4,6)==3); // bana / banana
assert(r.lcs(0,6)==0);
```

接口参考

1. 声明 / 调用形式

```
void SA::set(); void SA::build(string s,int M=128)
```

参数、效果与返回：清空或构建 SA；s 必须有前导空格，正文每个字节属于 [1,M)，一般字节串可传 M=256。正文为空合法；0 字节不能出现。

复杂度：O(n+M) 时间/空间。

2. 声明 / 调用形式

```
SA::sa; rk; n; void output()
```

参数、效果与返回：sa[1..n] 为按字典序排列的后缀起点；rk[i] 是 suf_i 排名。满足 sa[rk[i]]=i。output 输出调试数组；solve 为示例流程而非额外算法接口。

复杂度：读取 O(1)，output O(n)。

3. 声明 / 调用形式

```
void SA_LCP::build(const SA &sa); void SA_LCP::build(const string &s,int M=128)
```

参数、效果与返回：从已建 SA 或字符串建 height 与线性 RMQ；对象保存自己的查询数据，不依赖传入 SA 后续存活。

复杂度：O(n+M) 时间/空间。

4. 声明 / 调用形式

```
int SA_LCP::lcp(int i,int j) const; int ask(int i,int j) const; height
```

参数、效果与返回：返回 suf_i 与 suf_j 的最长公共前缀，i/j 是原串位置 1..n，不是排名；ask 同义。height[r] 是排名 r-1/r 两后缀的 LCP，height[1]=0。

复杂度：O(1)。

5. 声明 / 调用形式

```
void SA_LCS::build(const SA &sa); void SA_LCS::build(string s,int M=128)
```

参数、效果与返回：反转原内容后建查询结构。前缀下标仍按原串，不需要调用者自行反转。

复杂度：O(n+M) 时间/空间。

6. 声明 / 调用形式

```
int SA_LCS::lcs(int i,int j) const; int ask(int i,int j) const
```

参数、效果与返回：返回原串前缀 $s[1..i]$ 与 $s[1..j]$ 的最长公共后缀； $0 \leq i, j \leq n$ ，0 表示空前缀。ask 同义。

复杂度： $O(1)$ 。

注意事项

SA、LCP、LCS 均 $O(n)$ 构建和空间，查询 $O(1)$ 。LCS 参数是原串前缀的右端点。不要把后缀排名传给 lcp。

默认字符编码范围 1..127，任意非零字节传 $M=256$ ；不支持串内零字节。修改原串后须重建相关结构。

6.10 string/suffix-automaton.cpp

单串后缀自动机

用途

构建单个小写字母串的子串状态图，并统计每个状态对应子串的出现次数。时间及空间 $O(n)$ ，字母表固定 26。

最小示例

```
SAM t; t.build(" ababa");
int u=0; for(char c:string("aba"))u=t.ch[u][c-'a'];
assert(u && t.siz[u]==2);
ll distinct=0;
for(int i=1;i<=t.tot;i++)distinct+=t.len[i]-t.len[t.fail[i]];
assert(distinct==9);
```

接口参考

1. 声明 / 调用形式

```
SAM(); void set(int m=0); void extend(char c)
```

参数、效果与返回：set 清空并预留容量，根 0；extend 追加小写字母，构建状态和 fail。仅 extend 尚未得到汇总出现次数。

复杂度：总体 $O(n)$ ，单次可能多次跳 fail。

2. 声明 / 调用形式

```
void build(string s)
```

参数、效果与返回：s 是带前导空格的小写串；自动重置、构建并沿 fail 树汇总 siz。需要统计出现次数时用完整 build。

复杂度： $O(n)$ 时间/空间。

3. 声明 / 调用形式

```
ch; len; fail; siz; tot; lst
```

参数、效果与返回：ch 转移，len 最大长度，fail 后缀链接，siz 为完整 build 后的出现次数；tot 最大状态编号，lst 整串末状态。状态 u 对应长度 $(len[fail[u]], len[u])$ 。

复杂度：读取 $O(1)$ 。

4. 声明 / 调用形式

```
void output(); void solve()
```

参数、效果与返回：output 调试；solve 是文件内演示入口，需要按题意改写。new_node/clone/dfs 是构建内部步骤，不应在 build 之后手动再汇总 siz。

复杂度：output $O(\text{状态数})$ 。

注意事项

匹配模式串时遇到不存在转移应立即判定不出现。build 内已累计 siz，不能再次累计。裸调用 extend 不会自动维护整套出现次数；多串应用请用 GSAM。solve 是“重复子串长度乘次数最大值”的特定示例，不是通用查询。

7 数学 / math

7.1 math/big-int.cpp

高精度有符号整数

用途

十进制大整数，vector 按 10^9 分块。加减 $O(n)$ ，普通乘法及长除法最坏 $O(nm)$ ， n 、 m 是分块数。

最小示例

```
BigInt a,b;
istringstream in("-12345678901234567890 100");
in >> a >> b;
auto [q,r]=a.divmod(b);
assert(q*b+r==a);
ostringstream out; out << q << ' ' << r;
assert(out.str()=="-123456789012345678 -90");
```

接口参考

1. 声明 / 调用形式

```
BigInt(ll x=0); cin>>a; cout<<a
```

参数、效果与返回：任意精度有符号整数，默认 0；用流读取/输出十进制，字符串允许正负号。非法输入置 failbit。若超过 ll，不能先用 ll 保存再构造。

复杂度： $O(\text{十进制位数})$ 。

2. 声明 / 调用形式

```
a+b; a-b; a*b; a/b; a%b; -a
```

参数、效果与返回：返回新的 BigInt。除法向零截断，余数与被除数同号，满足 $a=(a/b)*b+a\%b$ ；除数不得为零。

复杂度： d/e 为内部位数：加减 $O(d+e)$ ，乘除最坏二次。

3. 声明 / 调用形式

```
a+=b; a-=b; a*=b; a/=b; a%=b
```

参数、效果与返回：原地四则/取余，返回 a 的引用；允许整数隐式转换为 BigInt。

复杂度：同对应非原地运算。

4. 声明 / 调用形式

```
a==b; a!=b; a<b; a<=b; a>b; a>=b; bool(a)
```

参数、效果与返回：比较有符号数值，bool 检查非零；不要把内部 base=1e9 分块数组当十进制位。

复杂度：比较最坏 $O(d+e)$ ，bool $O(1)$ 。

5. 声明 / 调用形式

```
pair<BigInt,BigInt> divmod(const BigInt &b) const
```

参数、效果与返回：同时返回 {商, 余数}，避免分别 / 和 % 重复计算。

复杂度：最坏二次。

6. 声明 / 调用形式

```
int cmp(const BigInt &b) const; int cmp_abs(const BigInt &b) const
```

参数、效果与返回：比较数值/绝对值，返回 -1、0、1；无需先构造两个绝对值副本。

复杂度： $O(d+e)$ 。

7. 声明 / 调用形式

```
BigInt mul(int x) const; BigInt operator+() const; void trim()
```

参数、效果与返回：mul 返回乘以非负小整数 x 的结果， $0 \leq x < 1e9$ ；一元 $+$ 返回副本；trim 删除最高零分块并规范零的符号，正常运算已自动调用。

复杂度： $O(d)$ 。

注意事项

除法向零取整，余数与被除数同号；除数不能为零。这不是 FFT 高精度乘法，也不支持小数。

7.2 math/dynamic-mod-int.cpp

运行时模数及组合数

用途

同一份程序需要选定不同模数时使用。设置模数后再创建该轮使用的值。

最小示例

```
mint::setM(101); init_fact(10);
assert(C(2,5).x==10);
mint a=100; assert((a+2).x==1);
```

接口参考

1. 声明 / 调用形式

```
static void mint::setM(unsigned M)
```

参数、效果与返回：设运行时模数， $2 \leq M \leq \text{INT_MAX}$ ；在构造值之前调用，改模后旧值失效。不会自动重建阶乘表。

复杂度： $O(1)$ 。

2. 声明 / 调用形式

```
mint(); mint(int); mint(unsigned); mint(ll); mint(ull); x
```

参数、效果与返回：默认 0；整数构造自动归一化，可处理负数。x 为 $[0, \text{模数})$ 的值，只读，不要直接写越界数。

复杂度： $O(1)$ 。

3. 声明 / 调用形式

```
a+b; a-b; a*b; a/b; a+=b; a-=b; a*=b; a/=b
```

参数、效果与返回：模运算，支持左右两侧混合整数；复合运算改左值并返回引用。除法等价乘逆元，除数必须与模数互质。

复杂度：加减乘 $O(1)$ ，除 $O(\log \text{模数})$ 。

4. 声明 / 调用形式

```
+a; -a; ++a; --a; a==b; a!=b; bool(a); !a
```

参数、效果与返回：一元正负、前置增减、相等/非等及非零判断；没有数值大小比较。前置增减返回自身引用。

复杂度： $O(1)$ 。

5. 声明 / 调用形式

```
mint pow(ll k) const; mint inv() const
```

参数、效果与返回：a.pow(k) 返回 a^k ； $k < 0$ 时先求逆，要求 a 可逆。a.inv() 同样要求 $\text{gcd}(a, \text{模数}) = 1$ ；0 不可逆。

复杂度： $O(\log(|k|+1))$ / $O(\log \text{模数})$ 。

6. 声明 / 调用形式

```
cin>>a; cout<<a
```

参数、效果与返回：按 ll 范围读取十进制后归一化，输出非负标准余数；流式运算返回流引用。

复杂度： $O(\text{字符数})$ 。

7. 声明 / 调用形式

```
mint C(int k,int n); mint binom(int n,int k)
```

参数、效果与返回：返回从 n 个中选 k 个，即通常写的 $C(n,k)$ 。本库 C 的参数顺序相反： $k < 0$ 或 $k > n$ 返回 0。

复杂度：表已就绪时 $O(1)$ 。

8. 声明 / 调用形式

```
fac; ifac; inv
```

参数、效果与返回： $\text{fac}[i]=i!$, $\text{ifac}[i]=1/i!$, $\text{inv}[i]=1/i$, $\text{inv}[0]$ 仅占位。只能读预处理范围内的项。

复杂度： $O(1)$ 。

9. 声明 / 调用形式

```
void init_fact(int n=0)
```

参数、效果与返回：完全重建 $0..n$ 的 $\text{fac}/\text{ifac}/\text{inv}$ ； $0 \leq n < M$ 且 $n!$ 与 M 互质。调用 C 前须显式预处理到 n ，改模后必须重做。

复杂度： $O(n+\log M)$ 时间， $O(n)$ 空间。

注意事项

修改模数不会转换已有 mint 变量；丢弃旧值并重建阶乘。与固定模数版、各多项式版本的 mint / fac 等全局名字冲突。

7.3 math/gaussian-elimination.cpp

线性方程组与行列式

用途

用消元求 $Ax=b$ 的一组解，或计算方阵行列式。支持质数模数下的 mint 以及浮点类型；不能用 int/ll 实例化。

最小示例

```
Gauss<long double> g;
// x+y=3, 2x-y=0
int res=g.solve({{1,1,3},{2,-1,0}},2);
assert(res==1 && abs(g.x[0]-1)<1e-9L);
assert(abs(g.x[1]-2)<1e-9L);
assert(abs(g.det({{1,2},{3,4}})+2)<1e-9L);
```

接口参考

1. 声明 / 调用形式

```
Gauss<T>; long double eps
```

参数、效果与返回：T 取 mint 等域类型，或 double/long double；不接受整数。浮点判零为 $\text{abs}(v) \leq \text{eps}$ ，默认 1e-12，可在求解前改；模数应为质数。

复杂度：判零 $O(1)$ 。

2. 声明 / 调用形式

```
int solve(vector<vector<T>> a,int m)
```

参数、效果与返回：a 是 n 行 m+1 列增广矩阵，最后一列常数项；按值传入，不改调用者矩阵。返回 -1 无解、0 多解、1 唯一解。

复杂度： $O(n*m*\min(n,m))$ ，另含拷贝 $O(n*m)$ 。

3. 声明 / 调用形式

```
x; rank; where
```

参数、效果与返回：有解时 $x[0..m-1]$ 为一组解，自由变量取 0；无解 x 为空。rank 为系数矩阵秩；where[j] 为该列主元行，-1 表示自由列。

复杂度：读取 $O(1)$ 。

4. 声明 / 调用形式

```
T det(vector<vector<T>> a) const
```

参数、效果与返回：输入方阵，返回行列式，空矩阵返回 1；不改变上一次 solve 的 x/rank/where。浮点结果受 eps 与数值误差影响。

复杂度： $O(n^3)$ 。

5. 声明 / 调用形式

```
bool zero(const T &v) const
```

参数、效果与返回：按本对象的判零规则测试 v ；浮点 $\text{abs}(v) \leq \text{eps}$ ，精确域判断是否为零，可用于核对残差。

复杂度： $O(1)$ 。

注意事项

Gauss<mint> 需要先粘贴 ModInt 或一个内置 mint 的多项式文件。浮点使用最大绝对值选主元和绝对阈值 eps（默认 $1e-12$ ），尺度差异大或病态方程需要自行缩放并检查残差。方阵时间 $O(n^3)$ ，复制矩阵后原输入保持不变；多个 RHS 要自行扩展。

7.4 math/linear-basis/field-prefix.cpp

前缀模意义线性基

用途

最小示例

```
Prefix_Linear_Basis<mint> B; B.setN(2);
assert(B.insert({1,0},1)==0);
assert(B.insert({0,1},2)==0);
assert(B.insert({0,2},3)==2);
assert(B.insert({1,2},4)==1);
assert(B.get_rank(3)==2 && B.contains({1,0},3));
assert(!B.contains({1,0},4));
assert(B.insert({0,0},5)==5);
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); rank
```

参数、效果与返回：设向量维数并清空，坐标 $0..n-1$ ；rank 为当前秩。T 必须是精确域，如质数模数 mint；不接受浮点与内置整数。

复杂度：清空 $O(\text{旧存储} + n)$ ，最多 $O(n^2)$ 空间。

2. 声明 / 调用形式

```
int insert(vector<T> x, int id)
```

参数、效果与返回：id 为严格递增正整数，x 长度 n。增秩返回 0；否则返回可剔除的最早编号，可能就是当前 id。内部留下适合后缀查询的基，不保存原始向量副本。

复杂度： $O(n^2)$ 次域运算。

3. 声明 / 调用形式

```
int get_rank(int l=1) const; bool contains(vector<T> x, int l=1) const
```

参数、效果与返回：只用已插入且编号 $\geq l$ 的向量，求秩/是否张成 x。查询历史区间 $[l,r]$ 须在只插到 r 时调用，或自己保存快照；不是任意 r 查询。

复杂度：get_rank $O(n)$ ，contains $O(n^2)$ 。

注意事项

处理到编号 r 时，get_rank(l) 查询 $[l,r]$ 的秩，contains(x,l) 判断 x 是否在该区间张成空间中。l 默认 1，超出 r 时为空基。插入和可表示性查询 $O(n^2)$ ，秩查询 $O(n)$ ，空间 $O(n^2)$ 。只用于精确域，通常用质数模数 mint；setN 清空所有状态。pos 保留原编号，a 是消元后的向量。不是任意删除或持久化结构。

7.5 math/linear-basis/field.cpp

模意义线性基

用途

最小示例

```
Linear_Basis<mint> B; B.setN(2);
assert(B.insert({1,0}) && B.insert({0,1}));
assert(!B.insert({2,3}) && B.rank==2);
assert(B.contains({7,9}));
B.setN(2); assert(!B.contains({1,0}));
```

接口参考

1. 声明 / 调用形式

```
void setN(int n); rank
```

参数、效果与返回：设向量维数并清空，坐标 $0..n-1$ ；rank 为当前秩。T 必须是精确域，如质数模数 mint；不接受浮点与内置整数。

复杂度：清空 $O(\text{旧存储} + n)$ ，最多 $O(n^2)$ 空间。

2. 声明 / 调用形式

```
bool insert(vector<T> x); bool contains(vector<T> x) const
```

参数、效果与返回：输入长度必须恰好 n；insert 增秩返回 true，否则 false；contains 判断是否在张成空间，不修改基。

复杂度： $O(n^2)$ 次域运算。

注意事项

T 要求精确域上的运算，通常使用质数模数 mint；不支持普通 int/ll、合数模数或浮点数。插入和查询 $O(n^2)$ ，空间 $O(n^2)$ ，占用的主元行按实际长度存储。n=0 允许，空向量不增秩。只维护张成空间，不返回具体线性组合系数。

7.6 math/linear-basis/xor-prefix.cpp

前缀异或线性基

用途

最小示例

```
Prefix_Xor_Basis B; B.setM(2); // bits 0..1
assert(B.insert(1,1)==0 && B.insert(2,2)==0);
assert(B.insert(2,3)==2); // removing id=1 would lower rank
assert(B.insert(3,4)==1);
assert(B.get_rank(3)==2 && B.contains(1,3));
assert(!B.contains(1,4) && B.ask(0,4)==3);
assert(B.insert(0,5)==5);
```

接口参考

1. 声明 / 调用形式

```
void setM(int m); void set(); rank
```

参数、效果与返回：默认 64 位，setM 设置 0..64 位并清空，使用位 0..m-1；set 保留 m 清空。rank 是当前秩；插入值不得有 m 以上的 1。

复杂度：O(m)。

2. 声明 / 调用形式

```
int insert(ull x,int id)
```

参数、效果与返回：id 正且严格递增。增秩返回 0，否则返回此次相关关系中被剔除的最早可替换编号；x=0 返回本次 id。set/setM 也清空编号进度。

复杂度：O(m)。

3. 声明 / 调用形式

```
int get_rank(int l=1) const; bool contains(ull x,int l=1) const; ull ask(ull x=0,int l=1) const
```

参数、效果与返回：只考虑已插入且编号 $\geq l$ 的元素，分别求秩、能否表示 x、最大 x xor 子集值。查询 [l,r] 时处理到 r 即查询，继续插入后不能直接指定过去的 r。

复杂度：O(m)。

注意事项

处理到右端点 r 后，get_rank(l)、contains(x,l)、ask(x,l) 分别查询编号 [l,r] 的秩、可表示性和最大异或值。l 默认 1；l 大于 r 时视为空基。rank 是全部已插入元素的秩。pos 中的正编号是保留的原元素编号，a 是消元后的基。插入值必须满足 $x < 2^m$ (m=64 时允许任意 ull)。单次 O(m)，空间 O(m)；查询历史右端点须离线按 r 扫描或自己保存副本，没有持久化。contains(x,l) 对超范围高位返回 false，ask(x,l) 保留 x 的高位。

7.7 math/linear-basis/xor.cpp

异或线性基

用途

最小示例

```
Xor_Basis B; B.setM(3); // bits 0..2
assert(B.insert(1) && B.insert(2));
assert(!B.insert(3) && B.rank==2);
assert(B.contains(3) && !B.contains(4));
assert(B.ask()==3 && B.ask(4)==7);
```

接口参考

1. 声明 / 调用形式

```
void setM(int m); void set(); rank
```

参数、效果与返回：默认 64 位，setM 设置 0..64 位并清空，使用位 0..m-1；set 保留 m 清空。rank 是当前秩；插入值不得有 m 以上的 1。

复杂度： $O(m)$ 。

2. 声明 / 调用形式

```
bool insert(ull x); bool contains(ull x) const; ull ask(ull x=0) const
```

参数、效果与返回：insert 增秩返回 true；contains 判断是否能异或得到 x；ask 最大化 x xor 子集异或和，允许空集。ask 的高于 m 位保持原样。

复杂度： $O(m)$ 。

注意事项

插入、查询均 $O(m)$ ，空间 $O(m)$ 。插入值必须满足 $x < 2^m$ ，m=64 时允许任意 ull。contains(x) 对带有超范围高位的 x 返回 false；ask(x) 允许这样的 x，高位保持不变。输入按无符号大小比较；m=64 时第 63 位可用，m=0 时只能插入零。零向量不增秩。a 中存的是消元后的向量，不是原输入元素。

7.8 math/mod-int.cpp

固定模数及组合数

用途

默认模数 998244353。模数写在源码中的 MOD；阶乘表按需扩容。

最小示例

```
init(10);
assert(C(2,5).x==10 && binom(5,2).x==10);
mint a=3; assert((a*a.inv()).x==1);
assert(a.pow(4).x==81);
```

接口参考

1. 声明 / 调用形式

```
ModInt<unsigned M>; using mint=ModInt<MOD>
```

参数、效果与返回：编译期模数类型，默认 MOD=998244353。四则可用合数模，但阶乘表的递推按质数模数使用；换模要改 MOD。

复杂度：O(1)。

2. 声明 / 调用形式

```
mint(); mint(int); mint(unsigned); mint(ll); mint(ull); x
```

参数、效果与返回：默认 0；整数构造自动归一化，可处理负数。x 为 [0, 模数) 的值，只读，不要直接写越界数。

复杂度：O(1)。

3. 声明 / 调用形式

```
a+b; a-b; a*b; a/b; a+=b; a-=b; a*=b; a/=b
```

参数、效果与返回：模运算，支持左右两侧混合整数；复合运算改左值并返回引用。除法等价乘逆元，除数必须与模数互质。

复杂度：加减乘 O(1)，除 O(log 模数)。

4. 声明 / 调用形式

```
+a; -a; ++a; --a; a==b; a!=b; bool(a); !a
```

参数、效果与返回：一元正负、前置增减、相等/非等及非零判断；没有数值大小比较。前置增减返回自身引用。

复杂度：O(1)。

5. 声明 / 调用形式

```
mint pow(ll k) const; mint inv() const
```

参数、效果与返回：a.pow(k) 返回 a^k ；k<0 时先求逆，要求 a 可逆。a.inv() 同样要求 gcd(a,x, 模数)=1；0 不可逆。

复杂度：O(log(|k|+1)) / O(log 模数)。

6. 声明 / 调用形式

```
cin>>a; cout<<a
```

参数、效果与返回：按 ll 范围读取十进制后归一化，输出非负标准余数；流式运算返回流引用。

复杂度：O(字符数)。

7. 声明 / 调用形式

```
mint C(int k,int n); mint binom(int n,int k)
```

参数、效果与返回：返回从 n 个中选 k 个，即通常写的 $C(n,k)$ 。本库 C 的参数顺序相反； $k < 0$ 或 $k > n$ 返回 0。
复杂度：表已就绪时 $O(1)$ 。

8. 声明 / 调用形式

```
fac; ifac; inv
```

参数、效果与返回： $fac[i]=i!$, $ifac[i]=1/i!$, $inv[i]=1/i$, $inv[0]$ 仅占位。只能读预处理范围内的项。
复杂度： $O(1)$ 。

9. 声明 / 调用形式

```
mint operator++(int); mint operator--(int)
```

参数、效果与返回： $a++/a--$ 返回修改前副本；此重载是本文件的接口，动态模数版未提供后置形式。
复杂度： $O(1)$ 。

10. 声明 / 调用形式

```
void init(int n=0)
```

参数、效果与返回：扩展阶乘/逆阶乘/逆元表到至少 n ； $0 \leq n < \text{MOD}$ ，要求 MOD 为质数。 C 会自动调用，重复小范围 init 无额外扩展。
复杂度：扩表 $O(n)$ ，空间 $O(n)$ 。

11. 声明 / 调用形式

```
mint Apple_in_Box(int n,int m); mint Apple_in_Box(int n,int m,int k)
```

参数、效果与返回： n 个相同球放入 m 个有标号盒；两参要求每盒非空，三参要求前 k 盒非空，其余可空， $0 \leq k \leq m$ 。使用 $m \geq 1$ ， n 及组合数上界须可预处理。
复杂度：扩表后 $O(1)$ 。

注意事项

阶乘表要求质数模数且 $n < \text{MOD}$ 。当前实现支持 $==/!=$ 、负整数输入归一化与 $\text{pow}(\text{LLONG_MIN})$ ，负幂仍要求底数可逆。 $C(k,n)$ 的参数顺序不要用反。多项式文件已内置 mint ，不要重复粘贴。

7.9 math/rational.cpp

有理数

用途

用 `ll` 分子 `p`、分母 `q` 表示精确有理数，适合需要精确比较和运算的斜率、比例等。构造和运算后自动约分，分母保持正数，零统一为 `0/1`。比较 $O(1)$ ，运算的约分需要一次辗转相除。

最小示例

```
Rat a(1,2),b(2,3);
assert(a+b==Rat(7,6) && a*b==Rat(1,3));
assert(a<b && 2*a==1);
a+=b;
ostringstream out; out<<a;
assert(out.str()=="7/6");
assert(Rat(2,-4)==Rat(-1,2));
```

接口参考

1. 声明 / 调用形式

```
Rat(); Rat(ll p,ll q=1); static Rat make(__int128 p,__int128 q)
```

参数、效果与返回：构造最简有理数，默认 `0/1`；分母不可为零，符号归到分子。`make` 允许 128 位中间参数，但约分后的 `p/q` 必须能用 `ll` 存储且 `q>0`。

复杂度： $O(\log \text{ 数值范围})$ 。

2. 声明 / 调用形式

```
p; q; a+b; a-b; a*b; a/b; -a
```

参数、效果与返回：`p/q` 可读取，不要直接改；四则和取负返回约分后的 `Rat`，允许与整数混用。除数不可为 0；约分后的结果须在 `ll` 范围内。

复杂度：每次 $O(\log \text{ 数值范围})$ 。

3. 声明 / 调用形式

```
a+=b; a-=b; a*=b; a/=b
```

参数、效果与返回：原地运算，返回 `a` 引用，其余条件同四则。

复杂度： $O(\log \text{ 数值范围})$ 。

4. 声明 / 调用形式

```
a==b; a!=b; a<b; a<=b; a>b; a>=b; cout<<a
```

参数、效果与返回：比较使用精确交叉乘积；输出始终为 `p/q`，包括整数如 `2/1`。没有输入 » 重载，先读 `p`、`q` 再构造。

复杂度：比较 $O(1)$ ，输出 $O(\text{位数})$ 。

注意事项

依赖公共底稿中的 `ll`。分母及除数不能为零；初始分子、分母和约分后的分子、正分母必须能存入 `ll`。中间交叉乘法及加减使用 `__int128`，因此允许中间值超出 `ll` 后约分回来，但最终溢出会触发 `assert`。没有浮点转换，也不表示无穷大。

7.10 math/xor-equations.cpp

XOR 方程组

用途

解若干条形如 $x_0 \oplus x_2 = k$ 的方程。每个未知数是一个 `ll`，系数只决定该未知数是否参加异或；对系数 `bitset` 做消元时，同时异或整个右端 `ll`，一次处理全部 64 位。支持负数右端，按其位模式参与异或。

最小示例

```
Xor_Gauss<3> g;
g.add(bitset<3>(0b011),5); // x[0] ^ x[1] = 5
g.add(bitset<3>(0b110),3); // x[1] ^ x[2] = 3
assert(g.solve()==0);      // free x[2] = 0
g.add(bitset<3>(0b001),7); // x[0] = 7
assert(g.solve()==1);
assert(g.x==vector<ll>({7,2,1}));
assert(!g.add(bitset<3>(),1)); // 0 = 1
assert(g.solve()==-1);
```

接口参考

1. 声明 / 调用形式

```
Xor_Gauss<N>; void set(); void setN(int n)
```

参数、效果与返回：N 是编译期 `bitset` 容量，默认 `n=N`；`set` 清空方程保留 `n`，`setN` 设 $0 \leq n \leq N$ 再清空。变量从 0 起，值为 `ll`。

复杂度： $O(n * \text{ceil}(N/64))$ 。

2. 声明 / 调用形式

```
bool add(bitset<N> v, ll w)
```

参数、效果与返回：添加 $\text{XOR}_{\{i:v[i]=1\}} x[i]=w$ ，`n` 以上的系数必须为零。返回加入后整个系统是否相容，不是是否增秩。不相容后持续 `false`，需 `set` 才重置。

复杂度： $O((n+1) * (\text{ceil}(N/64)+1))$ 。

3. 声明 / 调用形式

```
int solve(); x; rank; ok
```

参数、效果与返回：返回 -1 无解、0 多解、1 唯一；有解 `x[0..n-1]` 是一组 `ll` 解，自由变量取零。`rank` 是系数秩，`ok` 是相容标志。可继续加方程后再次 `solve`。

复杂度： $O(n * (n + \text{ceil}(N/64) + 1))$ 。

注意事项

`bitset` 的最低位对应 `x[0]`；同一个变量在一条方程中出现两次会抵消，构造时可用 `flip(i)`。零系数行右端非零即矛盾。本模板使用 GNU `bitset` 的置位遍历接口。设 $W = \lceil N/64 \rceil$ ，每次插入最坏 $O((n+1)(W+1))$ ，回代 $O(n(n+W+1))$ ，存储上界 $O(N(W+1))$ 。N 与 `n` 同阶时可按常规 `bitset` 消元估算开销。

8 数论 / number-theory

8.1 number-theory/dujiao-sieve.cpp

μ 、 φ 前缀和

用途

预筛小范围并缓存整除分块递归，分别查询 $\sum \mu(i)$ 、 $\sum \varphi(i)$ ， $i=1..n$ 。

最小示例

```
du_jiao d; d.init_mu(3);
assert(d.sum_mu(10)==-1);
d.init_phi(3); assert(d.sum_phi(10)==32);
assert(d.sum_mu(0)==0 && d.sum_phi(0)==0);
```

接口参考

1. 声明 / 调用形式

```
void init_mu(int B); void init_phi(int B)
```

参数、效果与返回： $B \geq 1$ ；分别预处理 μ/φ 的前缀数组并清空对应记忆化缓存。两个独立，只初始化需要的一种即可；数组 $\mu[i]/\varphi[i]$ 已是前缀和，不是单点函数值。

复杂度： $O(B)$ 时间/空间。

2. 声明 / 调用形式

```
ll sum_mu(ll n); __int128 sum_phi(ll n)
```

参数、效果与返回： $n \geq 0$ ，分别返回 $[1, n]$ 上的函数和， $n=0$ 返回 0；调用前须完成对应 init。区间 $[l, r]$ 用 $\text{sum}(r) - \text{sum}(l-1)$ 。 φ 结果不要转回 ll。

复杂度： B 取约 $n^{2/3}$ 时整体 $O(n^{2/3})$ ；缓存命中 $O(1)$ 期望。

注意事项

必须先初始化对应函数。 B 是预处理上界而非查询上限，常从 $n^{2/3}$ 附近结合内存选择；预筛 $O(B)$ ，递归性能随 B 和查询集改变。重复初始化会清空该项缓存。

8.2 number-theory/floor-sum.cpp

类欧几里得整除求和

用途

计算闭区间 $i=l..r$ 上 $\text{floor}((a*i+b)/c)$ 的和，时间 $O(\log c)$ ，返回 `__int128`。

最小示例

```
auto ans=floor_sum(1,4,3,1,2);
assert(ans==16); // 2+3+5+6
```

接口参考

1. 声明 / 调用形式

```
__int128 floor_sum(ll l,ll r,ll a,ll b,ll c)
```

参数、效果与返回：返回 $\sum_{i=l}^r \text{floor}((a*i+b)/c)$ 。要求 $l,r,a,b \geq 0, c > 0$ ；中间计算和答案须在 `__int128` 内。取模前先用此类型接返回值；可用 FastIO 输出。 $l > r$ 时返回 0。

复杂度：类欧几里得 $O(\log(c+1))$ 轮。

注意事项

不能直接处理负系数或负下标。输出可配 basic/fast-io.cpp；不要先在 ll 中计算 $a*l$ 。答案和内部累计值须在有符号 128 位范围内。

8.3 number-theory/linear-sieve.cpp

线性筛素数

用途

$O(n)$ 时间和空间，适合需要枚举范围内全部素数。

最小示例

```
init(20);  
assert(m==8 && prime[1]==2 && prime[m]==19);  
assert(is_prime[17] && !is_prime[1]);
```

接口参考

1. 声明 / 调用形式

```
void init(int n); is_prime; prime; m
```

参数、效果与返回： $n \geq 1$ 。重建 $0..n$ 素数表，`is_prime[x]` 判断素数；`prime[1..m]` 是递增素数，第 0 项占位，`m` 为素数数目。超出 `n` 不能直接查表。

复杂度： $O(n)$ 时间/空间。

注意事项

不是任意大整数判素接口，不能访问 `n` 以外。全局 `init`、`m`、`is_prime` 名字较通用，与其他模板组合时应检查冲突。

8.4 number-theory/modular-arithmetic.cpp

模逆元和扩展 CRT

用途

提供 gcd、lcm、逆元与不要求模数互素的同余方程合并。

最小示例

```
COE a(2,3),b(3,5); auto c=a+b;
assert(!c.empty() && c.r==8 && c.M==15);
assert(c.accept(23) && inv(3,11)==4);
assert((COE(0,2)+COE(1,2)).empty());
```

接口参考

1. 声明 / 调用形式

```
ull gcd(ull x,ull y); ull lcm(ull x,ull y)
```

参数、效果与返回：非负整数最大公约/最小公倍数，gcd(0,0)=0，任一为零则 lcm=0；lcm 结果须在 ull 范围内。

复杂度： $O(\log(\max(x,y)+1))$ 。

2. 声明 / 调用形式

```
ll inv(ll x,ll M)
```

参数、效果与返回：返回标准余数形式的 x 逆元， $M>0$ ，允许 x 为负；要求 gcd(x,M)=1，否则断言失败。

复杂度： $O(\log M)$ 。

3. 声明 / 调用形式

```
COE(); COE(ll r,ll M); r; M
```

参数、效果与返回：表示同余 $x=r \pmod{M}$ ， $M>0$ ；构造归一化 r。默认 $0 \pmod{1}$ 表示全部整数。结果 r 是最小非负解，M 为合并后模数。

复杂度： $O(1)$ 。

4. 声明 / 调用形式

```
COE a+b; COE& a+=b; bool empty() const; bool accept(ll x) const
```

参数、效果与返回：+ 合并两同余，可非互质；无解时 $M=-1$ ，empty 为 true。accept 判断整数是否满足当前同余，无解恒 false。合并后 lcm 必须能存 ll。

复杂度：合并 $O(\log \text{模数})$ ，判定 $O(1)$ 。

注意事项

合并后的模数须能放入 ll，内部乘法虽提升到 `__int128`，但结果不是任意精度。inv 函数会与部分模数模板的全局 inv 数组重名，需要按实际组合改名。

8.5 number-theory/pollard-rho.cpp

大整数判素与质因数分解

用途

Miller-Rabin 配合 Pollard-Rho, 适用于范围内不适合筛到上界的整数。随机算法的运行时间依赖输入。

最小示例

```
auto f=factor(360);
assert((f==vector<ull>{2,2,2,3,3,5}));
assert(is_prime(1000000007ULL));
```

接口参考

1. 声明 / 调用形式

```
bool is_prime(ull n)
```

参数、效果与返回: 对模板允许的 n 范围作素性判定, 小于 2 返回 false。使用固定 Miller-Rabin 底数; 本实现要求 $n < 2^{62}$, 不能按完整 ull 范围调用。

复杂度: $O(\log n)$ 次模乘 (固定底数个数)。

2. 声明 / 调用形式

```
ull pollard_rho(ull n); vector<ull> factor(ull n)
```

参数、效果与返回: factor 要求 $n \geq 1$, pollard_rho 要求 $n \geq 2$; 统一限制 $n < 2^{62}$ 。pollard_rho 找因子, 素数返回自身; factor 返回递增质因子列表, 按重数重复, factor(1) 为空。不允许 factor(0)。

复杂度: 随机化算法, 运行时间与因子大小有关, 不保证固定多项式最坏界。

3. 声明 / 调用形式

```
ull rho_gcd(ull x, ull y)
```

参数、效果与返回: 非负整数二进制 gcd, 任一为零时返回另一项; 用于需要避免与其他 gcd 重名的调用。

复杂度: $O(\log(\max(x,y)+1))$ 。

注意事项

factor 的输入要求 $1 \leq n < 2^{62}$, 不能传 0。不是完整 uint64 范围版本。判素函数与线性筛的 is_prime 数组同名, 不能原样同贴。

8.6 number-theory/stern-brocot.cpp

有理数二分

用途

在 $0 \leq p \leq M$, $1 \leq q \leq M$ 的最简分数中寻找目标或目标两侧相邻分数。先粘贴 math/rational.cpp。沿 Stern-Brocot 树倍增跳步，再二分跳步长度；询问次数及本地计算量均为 $O(\log(M+1))$ ，额外空间 $O(1)$ ，不计判定函数内部开销。

最小示例

```
SBT t; t.setM(100); Rat target(2,3);
while(!t.done()){
    Rat x=t.ask();
    t.tell(x<target?-1:x>target?1:0);
}
assert(t.found() && t.l==make_pair(211,311));
// Bracket sqrt(2), which is not a rational candidate.
t.setM(10);
while(!t.done()){
    Rat x=t.ask();
    t.tell((__int128)x.p*x.p<2*(__int128)x.q*x.q?-1:1);
}
assert(!t.found());
assert(t.l==make_pair(711,511) && t.r==make_pair(1011,711));
```

接口参考

1. 声明 / 调用形式

```
void setM(ll M)
```

参数、效果与返回：M>=1，重置搜索域：最简分数 $0 \leq p \leq M$ 、 $1 \leq q \leq M$ 。只搜非负有理数；先粘贴 Rat。
复杂度：O(1)。

2. 声明 / 调用形式

```
bool done() const; Rat ask() const; void tell(int res)
```

参数、效果与返回：循环 while(!done()){Rat x=ask();tell(cmp(x));}。res=-1 表示刚问的 x 太小，+1 太大，0 相等；问后反馈一次，反馈必须与数轴顺序一致。done 后不可再 ask/tell。

复杂度：整次搜索 $O(\log(M+1))$ 交互，空间 $O(1)$ 。

3. 声明 / 调用形式

```
bool found() const; pair<ll,ll> l,r
```

参数、效果与返回：结束后 found=true 表示反馈了相等，l==r 是答案；否则 l/r 是阈值左右相邻候选。l=(-1,0) 表示无下界候选，r=(1,0) 表示无上界候选，不能构造造成 Rat。

复杂度：O(1)。

4. 声明 / 调用形式

```
tell(x<target?-1:1)
```

参数、效果与返回：只有 $<$ 与 $\geq$ 时这样反馈：结束得到最大的 $<\text{target}$ 候选 l 、最小的 $\geq\text{target}$ 候选 r ，`found` 不会为真。若目标可能不在候选集中，这仍是边界搜索而非精确等值定位。

复杂度：同上。

注意事项

反馈必须对应同一个目标或单调分界，符号描述的是当前询问值，不能反过来。目标可以不在候选集合，也可以小于零或大于 M 。若只提供 -1 和 1 ，则搜索最终返回分界两侧候选，不会报告 `found`；对“ $<$ 与 $\geq$ ”判定， l 是最大的 $<$ 候选， r 是最小的 $\geq$ 候选， r 可以恰好等于目标。没有有限上界时 r 为 $1/0$ ；目标小于零时 l 为 $-1/0$ ， r 为 $0/1$ 。

9 多项式 / polynomial

9.1 polynomial/fwt.cpp

按位 AND / OR / XOR 卷积

用途

求 $c_k = \sum_{i \oplus j = k} a_i b_j$, 其中 op 为按位 AND、OR 或 XOR。长度 $n = 2^k$, 每次变换 $O(n \log n)$, 额外空间 $O(1)$ 。

最小示例

```
vector<int> a{1,2},b{3,4};
auto c=convolution_and(a,b); assert((c==vector<int>{13,8}));
c=convolution_or(a,b); assert((c==vector<int>{3,18}));
c=convolution_xor(a,b); assert((c==vector<int>{11,10}));
// vector<mint> works with the same interfaces.
```

接口参考

1. 声明 / 调用形式

```
void fwt_and(vector<T> &a, bool inverse=false); fwt_or(...); fwt_xor(...)
```

参数、效果与返回: T 可为 int、ll 或 mint。原地变换, 长度必须为正的 2 次幂; false 正变换, true 逆变换。不改变 vector 长度。

复杂度: $O(n \log n)$, 额外空间 $O(1)$ 。

2. 声明 / 调用形式

```
vector<T> convolution_and(vector<T> a, vector<T> b); convolution_or(...); convolution_xor(...)
```

参数、效果与返回: 返回 $c[k] = \sum_{i \oplus j = k} a[i] * b[j]$, OP 对应 &、竖线或 ^。自动把两输入补到不小于 max 长度的 2 次幂; 任一输入空则返回空。调用者原 vector 不改。

复杂度: $O(n \log n)$ 时间, $O(n)$ 空间。

3. 声明 / 调用形式

XOR 逆变换的除法

参数、效果与返回: 整数版本需输入确实来自整数变换/卷积, 使除以 2 精确; mint 要求 2 可逆。所有整数中间结果也须不溢出, 不能只看最终答案范围。

复杂度: 每层 $O(n)$ 。

注意事项

下标从 0 开始。int/ll 表示普通整数运算, 不会自动取模, 正变换和逐点乘法的中间值也必须不溢出。整数 XOR 逆变换逐层除以 2, 要求每步可整除, 不能把任意整数数组当成合法频域数据。XOR 模运算要求 2 可逆, 通常使用奇数模数; AND/OR 无此限制。不需要归一化第二次。

9.2 polynomial/int64.cpp

模 2^{64} 卷积与递推

用途

使用 ull 自然溢出，所有加减乘均按模 2^{64} 计算。卷积拆为四个 16 位段，在两个 NTT 质数下精确恢复各段的贡献，再合并为 ull；不依赖浮点精度。

最小示例

```
poly f{1,0ull-2,3},g{4,5};
assert((f*g==poly{4,0ull-3,2,15}));
assert((poly{~0ull}*poly{~0ull}==poly{1}));
assert(recurrence({0,1},{1,1},10)==55);
assert(FSPE({0,1},{1,~0ull,~0ull},10)==55);
assert(RSPE({0,2,2,4,6,10,16,26},10)==110);
```

接口参考

1. 声明 / 调用形式

```
using poly=vector<ull>; using Poly=poly;
poly operator*(const poly &f,const poly &g)
```

参数、效果与返回：0 起下标为次数；非空结果长 $n+m-1$ ，须不超过 2^{23} 。空输入返回空。负数写成 $0ull-x$ ，结果始终以无符号整数表示。

复杂度： $O(N \log N)$ 时间、 $O(N)$ 空间， N 为补齐长度。

2. 声明 / 调用形式

```
ull FSPE(poly f,poly g,ll k)
```

参数、效果与返回：返回 $[x^k]f/g$ ； $k \geq 0$ ， g 非空且 $g[0]$ 为奇数（无需等于 1）。空 f 返回 0。

复杂度： $O(d \log(d+1) \log(k+1))$ ， d 为输入规模。

3. 声明 / 调用形式

```
ull recurrence(const poly &a,const poly &c,ll k)
```

参数、效果与返回： a 、 c 等长 d ，给出 d 个初值及递推系数； $a[t]=\sum c[j]*a[t-1-j]$ ，返回 0 起第 k 项， $k \geq 0$ 。计算中和返回时都允许自然溢出。已知递推式时优先使用。

复杂度： $O(d \log(d+1) \log(k+1))$ 。

4. 声明 / 调用形式

```
ull RSPE(const poly &a,ll k)
```

参数、效果与返回：从样本猜最短递推，再求 0 起第 k 项。必须已知这是某个 d 阶递推的前缀，并给至少 $2d$ 项；有限样本不能证明任意数列的后续规律。内部按 2 的幂次选择主元消元，允许全偶数样本；不使用有限域 BM。

复杂度： s 个样本的推断最坏 $O(s^3 \log(s+1))$ 时间、 $O(s^2)$ 空间，再加推项代价。这比质数模数的 BM 慢，只建议小规模猜递推。

注意事项

本文件不是有符号精确整数版本；输出是模 2^{64} 的无符号代表元。FSPE 的分母常数必须是奇数。每次内部卷积结果长度不超过 2^{23} ；已知递推式时用 recurrence 避免消元。不依赖 mint，不与其他 poly 实现同时粘贴。

9.3 polynomial/mtt-fast.cpp

运行时模数快速多项式

用途

先调用 `mint::setM(mod)`。奇素数模数在精度预算内使用二维格映射与 AVX2/FMA 复数 FFT；合数、偶数及超预算规模自动使用三模 NTT。保留 `mtt.cpp` 的所有公开运算及 FPS 接口。需要支持 AVX2/FMA 的 CPU、GNU C++17/20，并先包含 `<immintrin.h>`。

最小示例

```
mint::setM(1000000007);
poly f{1,1,0,0};
auto g=Inv(f); auto h=f*g; h.resize(4);
assert(f*g==mtt_exact(f,g));
assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
auto e=Exp(Ln(f));
for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
assert(FSPE({0,1},{1,-1,-1},10).x==55);
auto p=lagrange({0,1,2},{1,4,9});
assert(p[0].x==1 && p[1].x==2 && p[2].x==1);
```

接口参考

1. 声明 / 调用形式

```
static void mint::setM(unsigned mod)
```

参数、效果与返回：2<=mod<=INT_MAX；先设置再创建值，改模使旧值失效并清空 `fac/ifac/inv`。卷积支持合数；FPS/递推/插值中的所有除数必须是单位元。卷积长度 <=2²³，FPS 输入 <=2²²。

复杂度：O(旧表大小)。

2. 声明 / 调用形式

```
poly; Poly; f[i]; f.resize(n)
```

参数、效果与返回：系数类型 `mint`，0 起下标就是次数；`Poly` 是 `poly` 别名。FPS 截断精度由输入长度决定，`resize(n)` 补高位零后再调用；不要提前去掉要保留的零。

复杂度：`resize` 与新增项数相关。

3. 声明 / 调用形式

```
poly operator*(const poly &f,const poly &g); f+g; f-g; f+=g; f-=g
```

参数、效果与返回：`*` 为卷积；`+/-` 按次数相加减并扩展至较长长度；复合加减原地修改。空输入的卷积为空。

复杂度：乘 O(N log N)，加减 O(n+m)。

4. 声明 / 调用形式

```
poly operator*(poly f,mint x); poly operator*(mint x,poly f); poly operator/(poly f,poly g)
```

参数、效果与返回：数乘逐项乘；`f/g` 是相关运算，结果第 `i` 项 =sum_j f[i+j]*g[j]，不是多项式长除或 FPS 相除。

复杂度：数乘 O(n)，相关 O(N log N)。

5. 声明 / 调用形式

```
mint value(const poly &f, mint x)
```

参数、效果与返回: Horner 法求 $f(x)$, 空多项式返回 0; 不修改 f 。

复杂度: $O(n)$ 。

6. 声明 / 调用形式

```
poly Inv(const poly &f)
```

参数、效果与返回: 返回长度 n 的 $1/f \bmod x^n$; 要求非空且 $f[0]$ 可逆。FPS 相除用 $f*Inv(g)$ 后 `resize` 到目标长度。

复杂度: $O(n \log n)$ 。

7. 声明 / 调用形式

```
poly diff(poly f); poly integ(poly f)
```

参数、效果与返回: 求导返回 $n-1$ 项 (空输入返回空); 积分返回 $n+1$ 项, 常数为零。integ 要求 $1..n$ 的逆元存在。

复杂度: $O(n)$, 积分可能扩展逆元表。

8. 声明 / 调用形式

```
poly Ln(const poly &f); poly Exp(const poly &f)
```

参数、效果与返回: Ln 要求 $f[0]=1$, Exp 要求 $f[0]=0$; 输入非空, 返回同长度。计算涉及的整数分母均须可逆。

复杂度: $O(n \log n)$ 。

多项式复合与复合逆

```
poly Compose(poly f, const poly &g, int n=-1);
poly Compose_Inv(poly f, int n=-1);
auto h=Compose({1,2,3},{4,1},4); // {57,26,3,0}
auto r=Compose_Inv({0,1,1},5); // {0,1,-1,2,-5}
// Compose({0,1,1},r,5) == poly({0,1,0,0,0})
```

Compose: 返回 $f(g(x)) \bmod x^n$ 的 n 个系数, n 默认取 $f.size()$ 。允许 $g[0]$ 非零, 空 g 按零多项式处理。 $n=0$ 返回空, 空 f 返回 n 个零。 $g[0]=0$ 时自动忽略 f 的 n 次及以上项; 否则完整保留 f 的系数。显式 n 不能改变输入 f 所表示的多项式, 缺少的高次系数均视为零。

Compose_Inv: 返回 $f(g(x)) = x \bmod x^n$ 且 $g[0]=0$ 的 g , n 默认取 $f.size()$; $n=0$ 返回空, $n=1$ 返回 $\{0\}$ 。要求 $f[0]=0$, $n \geq 2$ 时还要求 $f[1]$ 可逆, 计算中的整数 $1, \dots, n-1$ 也须可逆。只读取 f 的前 n 项, 缺少的系数补零。

规模与复杂度: 取 $k = \max(n, |f|)$, $g[0]=0$ 时先截断 f 。本文件要求 $k \leq 2^{20}$, 复合逆要求 $n \leq 2^{20}$ 。复合时间 $O(k \log^2 k)$ 、空间 $O(k \log k)$; 复合逆时间 $O(n \log^2 n)$ 、空间 $O(n)$ 。任意模数版本中, $g[0]$ 非零时还要求 $(2 \lceil k \rceil_2 - 1)!$ 可逆, $\lceil k \rceil_2$ 为不小于 k 的最小二次幂。使用大于该界的质数模数即可。

9. 声明 / 调用形式

```
poly BM(const poly &a)
```

参数、效果与返回: 返回 $C[0]=1$ 的最短递推分母, 阶数 $d=C.size()-1$, 满足 $a_t + \sum_{i=1}^d C_i a_{t-i} = 0$ ($t \geq d$)。实际递推系数是 $-C[i]$; 只保证拟合已给样本。空序列与全零序列返回 $\{1\}$ 。末尾零系数也表示阶数, 不能删去。长序列使用 Half-GCD, 短序列直接 BM。要求在域上计算, 任意模数版本请使用质数模数。样本不足时最短递推可能不唯一, 返回系数可能与旧版不同, 但阶数最短且满足全部已有样本。

复杂度: 时间 $O(n \log^2 n)$, 额外空间 $O(n)$ 。NTT 版本合并矩阵运算中的变换, 复用频域结果。

10. 声明 / 调用形式

```
mint FSPE(poly F,poly G,ll k)
```

参数、效果与返回：返回 $[x^k]F/G$, $k \geq 0$, G 非空、 $G[0]$ 可逆； F 可为空，返回 0。

复杂度： $O(d \log(d+1) \log(k+1))$ 。

11. 声明 / 调用形式

```
mint RSPE(poly samples,ll k)
```

参数、效果与返回：推断递推后求 0 起第 k 项，要求 $k \geq 0$ ； k 在输入范围内时直接返回。真实递推阶数 d 时通常至少给 $2d$ 项，不能任意外推。分子由截断卷积构造，不再使用二重循环。

复杂度：时间 $O(n \log^2 n + d \log(d+1) \log(k+1))$ 。

12. 声明 / 调用形式

```
poly lagrange(const vector<mint> &xs,const vector<mint> &ys)
```

参数、效果与返回：等长 n 项，输入任意点值 (x_i, y_i) ，返回长度恰为 n 的系数数组 f ，满足 $f(x_i) = y_i$ 且次数不超过 $n-1$ ，末尾零系数保留。 xs 不要求连续，但模意义下须两两不同；合数模数下还要求所有差值可逆。空输入返回空。要求 $n < 2^{23}$ 。

复杂度：时间 $O(n \log^2 n)$ ，空间 $O(n \log n)$ 。乘积树复用于导数的多点求值和插值合并，权重用一次求逆批量计算；沿用所选模板的卷积内核。

```
poly f=lagrange({2,5,9},{9,36,100});
assert(f==poly({1,2,1})); // f(X)=(X+1)^2
```

13. 声明 / 调用形式

```
mint; mint::pow(ll k); mint::inv(); x; cin>>a; cout<<a
```

参数、效果与返回：内置 `mint`，不再粘贴其他模整数文件。构造、四则/复合四则、正负号、前置增减、`==/!=`、`bool/!`、输入输出的用法同模整数；负指数要求可逆。这里未提供后置 `++/--`。

复杂度：基本运算 $O(1)$ ，逆元/幂 $O(\log \text{参数})$ 。

14. 声明 / 调用形式

```
void init(int n=0); mint C(int k,int n); mint binom(int n,int k); fac; ifac; inv
```

参数、效果与返回：预处理阶乘等表， $C(k,n)$ 与 $\text{binom}(n,k)$ 返回 n 选 k ，越界 k 返回 0； C 会扩表。直接读表前先 `init`。

复杂度：扩表 $O(n)$ ，表就绪查询 $O(1)$ 。

精确卷积 / 调用形式

```
poly mtt_exact(const poly &f,const poly &g)
```

参数、效果与返回：直接使用三模 NTT 和 CRT，返回当前 MOD 下的精确卷积。允许合数模数，空输入返回空。与 $f*g$ 相同，非空结果长 $n+m-1$ ，须不超过 2^{23} 。本接口不经过浮点 FFT；需要确定性精确卷积时显式调用，或直接选择普通 `mtt.cpp`。

复杂度： $O(N \log N)$ 时间、 $O(N)$ 空间， N 为补齐长度。

注意事项

这是浮点加速版。二维格坐标采用固定种子的随机舍入来减小偏差，精度预算是保守经验条件，不是对任意输入的数学保证。预算为 $(n+m-1)d \text{MOD}^2 \leq 2^{84}$ ， d 由模数自动计算；不满足时转到精确 CRT。FPS 中的卷积同样可能走 FFT。要求确定性精确答案时选 `mtt.cpp`，或卷积直接用 `mtt_exact`。改模后旧 `mint/poly` 值失效； Ln/Exp /积分涉及的分母必须可逆。需要 AVX2/FMA 与 `<immintrin.h>`，不提供运行时 CPU 分派。

固定点的重复求值与插值

```
Poly_Tree t;           // Empty tree.
t.set(xs);             // Replace points and clear cached inverse.
Poly_Tree t2(xs);      // Same as set(xs).
auto ys=t.eval(f);     // Also available as Eval(f,xs).
auto a=t.interpolate(ys); // Also available as lagrange(xs,ys).
t.set();               // Clear points and cache.
```

参数、效果与返回：xs 为 n 个取值点。构造/set 建立共享乘积树，只存 $2n - 1$ 个节点； n 为零时不建树。eval 返回与 xs 等长的点值，允许重复点；空多项式在各点的值均为零。interpolate 要求 ys 长度为 n ，返回 n 个系数，点差必须可逆；重复点不允许用于插值。

同一批 xs 反复求值时，直接保留对象调用 eval，复用乘积树与根部逆多项式；传入更长的 f 时会扩充逆多项式缓存。单次调用仍可用 Eval(f,xs) 或 lagrange(xs,ys)，无需修改旧代码。底层模数改变后必须重新 set，旧的点值与缓存都不能继续使用。

复杂度：建树、插值均为 $O(n \log^2 n)$ 。eval 时间为 $O(M(m) + M(n) \log(n + 1))$ ， $m=f.size()$ ； $M(k)$ 表示多项式乘法代价。树占用 $O(n \log n)$ 空间，另缓存已使用的根部逆多项式。小规模直接求值；超出根部相关乘积长度限制时采用余数树回退。

9.4 polynomial/mtt.cpp

非 NTT 模数下的多项式

用途

目标模数运行时设置，三模 NTT 后 CRT 还原。卷积支持 2 至 INT_MAX 之间的任意模数，包括合数；常用 FPS 功能与 ntt-fast 对齐。

最小示例

```
mint::setM(1000000007);
poly f{1,1,0,0};
auto g=Inv(f); auto h=f*g; h.resize(4);
assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
auto e=Exp(Ln(f));
for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
assert(FSPE({0,1},{1,-1,-1},10).x==55);
auto p=lagrange({0,1,2},{1,4,9});
assert(p[0].x==1 && p[1].x==2 && p[2].x==1);
```

接口参考

1. 声明 / 调用形式

```
static void mint::setM(unsigned mod)
```

参数、效果与返回：2<=mod<=INT_MAX；先设置再创建值，改模使旧值失效并清空 fac/ifac/inv。卷积支持合数；FPS/递推/插值中的所有除数必须是单位元。卷积长度 <=2²³，FPS 输入 <=2²²。

复杂度：O(旧表大小)。

2. 声明 / 调用形式

```
poly; Poly; f[i]; f.resize(n)
```

参数、效果与返回：系数类型 mint，0 起下标就是次数；Poly 是 poly 别名。FPS 截断精度由输入长度决定，resize(n) 补高位零后再调用；不要提前去掉要保留的零。

复杂度：resize 与新增项数相关。

3. 声明 / 调用形式

```
poly operator*(const poly &f,const poly &g); f+g; f-g; f+=g; f-=g
```

参数、效果与返回：* 为卷积；+/- 按次数相加减并扩展至较长长度；复合加减原地修改。空输入的卷积为空。

复杂度：乘 O(N log N)，加减 O(n+m)。

4. 声明 / 调用形式

```
poly operator*(poly f,mint x); poly operator*(mint x,poly f); poly operator/(poly f,poly g)
```

参数、效果与返回：数乘逐项乘；f/g 是相关运算，结果第 i 项 =sum_j f[i+j]*g[j]，不是多项式长除或 FPS 相除。

复杂度：数乘 O(n)，相关 O(N log N)。

5. 声明 / 调用形式

```
mint value(const poly &f,mint x)
```

参数、效果与返回：Horner 法求 f(x)，空多项式返回 0；不修改 f。

复杂度：O(n)。

6. 声明 / 调用形式

```
poly Inv(const poly &f)
```

参数、效果与返回：返回长度 n 的 $1/f \bmod x^n$ ；要求非空且 $f[0]$ 可逆。FPS 相除用 $f*Inv(g)$ 后 `resize` 到目标长度。

复杂度： $O(n \log n)$ 。

7. 声明 / 调用形式

```
poly diff(poly f); poly integ(poly f)
```

参数、效果与返回：求导返回 $n-1$ 项（空输入返回空）；积分返回 $n+1$ 项，常数为零。`integ` 要求 $1..n$ 的逆元存在。

复杂度： $O(n)$ ，积分可能扩展逆元表。

8. 声明 / 调用形式

```
poly Ln(const poly &f); poly Exp(const poly &f)
```

参数、效果与返回：`Ln` 要求 $f[0]=1$ ，`Exp` 要求 $f[0]=0$ ；输入非空，返回同长度。计算涉及的整数分母均须可逆。

复杂度： $O(n \log n)$ 。

多项式复合与复合逆

```
poly Compose(poly f, const poly &g, int n=-1);
poly Compose_Inv(poly f, int n=-1);
auto h=Compose({1,2,3},{4,1},4); // {57,26,3,0}
auto r=Compose_Inv({0,1,1},5); // {0,1,-1,2,-5}
// Compose({0,1,1},r,5) == poly({0,1,0,0,0})
```

Compose：返回 $f(g(x)) \bmod x^n$ 的 n 个系数， n 默认取 `f.size()`。允许 $g[0]$ 非零，空 g 按零多项式处理。 $n=0$ 返回空，空 f 返回 n 个零。 $g[0]=0$ 时自动忽略 f 的 n 次及以上项；否则完整保留 f 的系数。显式 n 不能改变输入 f 所表示的多项式，缺少的高次系数均视为零。

Compose_Inv：返回 $f(g(x)) = x \bmod x^n$ 且 $g[0]=0$ 的 g ， n 默认取 `f.size()`； $n=0$ 返回空， $n=1$ 返回 $\{0\}$ 。要求 $f[0]=0$ ， $n \geq 2$ 时还要求 $f[1]$ 可逆，计算中的整数 $1, \dots, n-1$ 也须可逆。只读取 f 的前 n 项，缺少的系数补零。

规模与复杂度：取 $k = \max(n, |f|)$ ， $g[0]=0$ 时先截断 f 。本文件要求 $k \leq 2^{20}$ ，复合逆要求 $n \leq 2^{20}$ 。复合时间 $O(k \log^2 k)$ 、空间 $O(k \log k)$ ；复合逆时间 $O(n \log^2 n)$ 、空间 $O(n)$ 。任意模数版本中， $g[0]$ 非零时还要求 $(2\lceil k \rceil_2 - 1)!$ 可逆， $\lceil k \rceil_2$ 为不小于 k 的最小二次幂。使用大于该界的质数模数即可。

9. 声明 / 调用形式

```
poly BM(const poly &a)
```

参数、效果与返回：返回 $C[0]=1$ 的最短递推分母，阶数 $d=C.size()-1$ ，满足 $a_t + \sum_{i=1}^d C_i a_{t-i} = 0$ ($t \geq d$)。实际递推系数是 $-C[i]$ ；只保证拟合已给样本。空序列与全零序列返回 $\{1\}$ 。末尾零系数也表示阶数，不能删去。长序列使用 Half-GCD，短序列直接 BM。要求在域上计算，任意模数版本请使用质数模数。样本不足时最短递推可能不唯一，返回系数可能与旧版不同，但阶数最短且满足全部已有样本。

复杂度：时间 $O(n \log^2 n)$ ，额外空间 $O(n)$ 。NTT 版本合并矩阵运算中的变换，复用频域结果。

10. 声明 / 调用形式

```
mint FSPE(poly F, poly G, ll k)
```

参数、效果与返回：返回 $[x^k]F/G$ ， $k \geq 0$ ， G 非空、 $G[0]$ 可逆； F 可为空，返回 0。

复杂度： $O(d \log(d+1) \log(k+1))$ 。

11. 声明 / 调用形式

```
mint RSPE(poly samples,ll k)
```

参数、效果与返回：推断递推后求 0 起第 k 项，要求 $k \geq 0$ ； k 在输入范围内时直接返回。真实递推阶数 d 时通常至少给 $2d$ 项，不能任意外推。分子由截断卷积构造，不再使用二重循环。

复杂度：时间 $O(n \log^2 n + d \log(d+1) \log(k+1))$ 。

12. 声明 / 调用形式

```
poly lagrange(const vector<mint> &xs,const vector<mint> &ys)
```

参数、效果与返回：等长 n 项，输入任意点值 (x_i, y_i) ，返回长度恰为 n 的系数数组 f ，满足 $f(x_i) = y_i$ 且次数不超过 $n-1$ ，末尾零系数保留。 xs 不要求连续，但模意义下须两两不同；合数模数下还要求所有差值可逆。空输入返回空。要求 $n < 2^{23}$ 。

复杂度：时间 $O(n \log^2 n)$ ，空间 $O(n \log n)$ 。乘积树复用于导数的多点求值和插值合并，权重用一次求逆批量计算；沿用所选模板的卷积内核。

```
poly f=lagrange({2,5,9},{9,36,100});
assert(f==poly({1,2,1})); // f(X)=(X+1)^2
```

13. 声明 / 调用形式

```
mint; mint::pow(ll k); mint::inv(); x; cin>>a; cout<<a
```

参数、效果与返回：内置 `mint`，不再粘贴其他模整数文件。构造、四则/复合四则、正负号、前置增减、`==`/`!=`、`bool`/`!`、输入输出的用法同模整数；负指数要求可逆。这里未提供后置 `++`/`--`。

复杂度：基本运算 $O(1)$ ，逆元/幂 $O(\log \text{参数})$ 。

14. 声明 / 调用形式

```
void init(int n=0); mint C(int k,int n); mint binom(int n,int k); fac; ifac; inv
```

参数、效果与返回：预处理阶乘等表， $C(k,n)$ 与 $\text{binom}(n,k)$ 返回 n 选 k ，越界 k 返回 0； C 会扩表。直接读表前先 `init`。

复杂度：扩表 $O(n)$ ，表就绪查询 $O(1)$ 。

注意事项

精度由输入 `vector` 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i] = \sum_j f[i+j] * g[j]$ ，不是多项式除法。形式幂级数相除要 `f*Inv(g)` 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。卷积结果长度最多 2^{23} ，FPS 输入长度最多 2^{22} ，Ln/Exp 长度 n 还要小于模数，且 $1..n-1$ 均可逆。`init(n)` 要求 $n!$ 可逆，`integ(f)` 要求 $1..f.size()$ 可逆；使用质数模数且次数小于模数即可满足这些条件。BM、递推求值和插值中的除数也必须可逆；合数模数下仅仅非零或取值点互异不足以保证这一点。没有 `ntt` 的 `Sqrt/Div`；`Eval` 与 `Poly_Tree` 已提供。RSPE 需要输入确实足以确定递推，通常至少给 2 倍阶数的样本，不适用于任意序列外推。

固定点的重复求值与插值

```
Poly_Tree t;           // Empty tree.
t.set(xs);             // Replace points and clear cached inverse.
Poly_Tree t2(xs);      // Same as set(xs).
auto ys=t.eval(f);     // Also available as Eval(f,xs).
auto a=t.interpolate(ys); // Also available as lagrange(xs,ys).
t.set();               // Clear points and cache.
```

参数、效果与返回：xs 为 n 个取值点。构造/set 建立共享乘积树，只存 $2n - 1$ 个节点； n 为零时不建树。eval 返回与 xs 等长的点值，允许重复点；空多项式在各点的值均为零。interpolate 要求 ys 长度为 n ，返回 n 个系数，点差必须可逆；重复点不允许用于插值。

同一批 xs 反复求值时，直接保留对象调用 eval，复用乘积树与根部逆多项式；传入更长的 f 时会扩充逆多项式缓存。单次调用仍可用 Eval(f,xs) 或 lagrange(xs,ys)，无需修改旧代码。底层模数改变后必须重新 set，旧的点值与缓存都不能继续使用。

复杂度：建树、插值均为 $O(n \log^2 n)$ 。eval 时间为 $O(M(m) + M(n) \log(n + 1))$ ， $m=f.size()$ ； $M(k)$ 表示多项式乘法代价。树占用 $O(n \log n)$ 空间，另缓存已使用的根部逆多项式。小规模直接求值；超出根部相关乘积长度限制时采用余数树回退。

9.5 polynomial/ntt-fast.cpp

998244353 快速多项式

用途

固定模数 998244353; 面向极限卡常, 使用 AVX2 Montgomery 模乘、32 字节对齐、基 4 蝶形及分块 Ln/Exp/Sqrt。保留 poly、mint.x 等接口。普通卷积在支持 AVX-512F 时使用不完全 NTT 与 16 点局部卷积, 否则自动走 AVX2; 小输入仍使用朴素乘法。Inv/Ln/Exp/Sqrt 保留 AVX2 分块内核。需要支持 AVX2 的 CPU、GNU C++17/20, 并在模板前包含 <immintrin.h>。参考内核作者为 QedDust413 与 Killer_joke。

最小示例

```
poly f{1,1,0,0};
auto g=Inv(f); auto h=f*g; h.resize(4);
assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
auto e=Exp(Ln(f));
for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
assert(FSPE({0,1},{1,-1,-1},10).x==55);
poly q,r; Div({1,2,1},{1,1},q,r);
assert(q.size()==2 && q[0].x==1 && q[1].x==1);
auto v=Eval({1,2,1},{0,1,2});
assert(v[0].x==1 && v[1].x==4 && v[2].x==9);
```

接口参考

1. 声明 / 调用形式

```
MOD=998244353; FFT_MAX=23
```

参数、效果与返回：固定模数；实际变换长度必须是 $\leq 2^{23}$ 的 2 次幂，FPS 的中间卷积也算在内。不要只根据最终返回长度判断可用范围。

复杂度：不适用。

2. 声明 / 调用形式

```
poly; Poly; f[i]; f.resize(n)
```

参数、效果与返回：系数类型 mint, 0 起下标就是次数；Poly 是 poly 的别名。FPS 截断精度由输入长度决定，resize(n) 补高位零后再调用；不要提前去掉要保留的零。

复杂度：resize 与新增项数相关。

3. 声明 / 调用形式

```
poly operator*(const poly &f,const poly &g); f+g; f-g; f+=g; f-=g
```

参数、效果与返回：* 为卷积；+/- 按次数相加减并扩展至较长长度；复合加减原地修改。空输入的卷积为空。

复杂度：乘 $O(N \log N)$ ，加减 $O(n+m)$ 。

4. 声明 / 调用形式

```
poly operator*(poly f,mint x); poly operator*(mint x,poly f); poly operator/(poly f,poly g)
```

参数、效果与返回：数乘逐项乘；f/g 是相关运算，结果第 i 项 $= \sum_j f[i+j]*g[j]$ ，不是多项式长除或 FPS 相除。

复杂度：数乘 $O(n)$ ，相关 $O(N \log N)$ 。

5. 声明 / 调用形式

```
mint value(const poly &f, mint x)
```

参数、效果与返回：Horner 法求 $f(x)$ ，空多项式返回 0；不修改 f 。

复杂度： $O(n)$ 。

6. 声明 / 调用形式

```
poly Inv(poly f)
```

参数、效果与返回：返回长度 n 的 $1/f \bmod x^n$ ；要求非空且 $f[0]$ 可逆。FPS 相除用 $f*Inv(g)$ 后 `resize` 到目标长度。

复杂度： $O(n \log n)$ 。

7. 声明 / 调用形式

```
poly diff(poly f); poly integ(poly f)
```

参数、效果与返回：求导返回 $n-1$ 项（空输入返回空）；积分返回 $n+1$ 项，常数为零。`integ` 要求 $1..n$ 的逆元存在。

复杂度： $O(n)$ ，积分可能扩展逆元表。

8. 声明 / 调用形式

```
poly Ln(poly f); poly Exp(poly f)
```

参数、效果与返回：`Ln` 要求 $f[0]=1$ ，`Exp` 要求 $f[0]=0$ ；输入非空，返回同长度。计算涉及的整数分母均须可逆。

复杂度： $O(n \log n)$ 。

多项式复合与复合逆

```
poly Compose(poly f, const poly &g, int n=-1);
poly Compose_Inv(poly f, int n=-1);
auto h=Compose({1,2,3},{4,1},4); // {57,26,3,0}
auto r=Compose_Inv({0,1,1},5); // {0,1,-1,2,-5}
// Compose({0,1,1},r,5) == poly({0,1,0,0,0})
```

Compose: 返回 $f(g(x)) \bmod x^n$ 的 n 个系数， n 默认取 $f.size()$ 。允许 $g[0]$ 非零，空 g 按零多项式处理。 $n=0$ 返回空，空 f 返回 n 个零。 $g[0]=0$ 时自动忽略 f 的 n 次及以上项；否则完整保留 f 的系数。显式 n 不能改变输入 f 所表示的多项式，缺少的高次系数均视为零。

Compose_Inv: 返回 $f(g(x)) = x \bmod x^n$ 且 $g[0]=0$ 的 g ， n 默认取 $f.size()$ ； $n=0$ 返回空， $n=1$ 返回 $\{0\}$ 。要求 $f[0]=0$ ， $n \geq 2$ 时还要求 $f[1]$ 可逆，计算中的整数 $1, \dots, n-1$ 也须可逆。只读取 f 的前 n 项，缺少的系数补零。

规模与复杂度: 取 $k = \max(n, |f|)$ ， $g[0]=0$ 时先截断 f 。本文件要求 $k \leq 2^{21}$ ，复合逆要求 $n \leq 2^{21}$ 。复合时间 $O(k \log^2 k)$ 、空间 $O(k \log k)$ ；复合逆时间 $O(n \log^2 n)$ 、空间 $O(n)$ 。NTT 版本使用成对频域乘积，避免每层重复进行完整卷积。

9. 声明 / 调用形式

```
poly BM(const poly &a)
```

参数、效果与返回：返回 $C[0]=1$ 的最短递推分母，阶数 $d=C.size()-1$ ，满足 $a_t + \sum_{i=1}^d C_i a_{t-i} = 0$ ($t \geq d$)。实际递推系数是 $-C[i]$ ；只保证拟合已给样本。空序列与全零序列返回 $\{1\}$ 。末尾零系数也表示阶数，不能删去。长序列使用 Half-GCD，短序列直接 BM。要求在域上计算，任意模数版本请使用质数模数。样本不足时最短递推可能不唯一，返回系数可能与旧版不同，但阶数最短且满足全部已有样本。

复杂度：时间 $O(n \log^2 n)$ ，额外空间 $O(n)$ 。NTT 版本合并矩阵运算中的变换，复用频域结果。

10. 声明 / 调用形式

```
mint FSPE(poly F,poly G,ll k)
```

参数、效果与返回：返回 $[x^k]F/G$, $k \geq 0$, G 非空、 $G[0]$ 可逆； F 可为空，返回 0。

复杂度： $O(d \log(d+1) \log(k+1))$ 。

11. 声明 / 调用形式

```
mint RSPE(poly samples,ll k)
```

参数、效果与返回：推断递推后求 0 起第 k 项，要求 $k \geq 0$ ； k 在输入范围内时直接返回。真实递推阶数 d 时通常至少给 $2d$ 项，不能任意外推。分子由截断卷积构造，不再使用二重循环。

复杂度：时间 $O(n \log^2 n + d \log(d+1) \log(k+1))$ 。

12. 声明 / 调用形式

```
poly lagrange(vector<mint> xs,vector<mint> ys)
```

参数、效果与返回：等长 n 项，输入任意点值 (x_i, y_i) ，返回长度恰为 n 的系数数组 f ，满足 $f(x_i) = y_i$ 且次数不超过 $n-1$ ，末尾零系数保留。 xs 不要求连续，但模意义下须两两不同；合数模数下还要求所有差值可逆。空输入返回空。要求 $n < 2^{23}$ 。

复杂度：时间 $O(n \log^2 n)$ ，空间 $O(n \log n)$ 。乘积树复用于导数的多点求值和插值合并，权重用一次求逆批量计算；沿用所选模板的卷积内核。

```
poly f=lagrange({2,5,9},{9,36,100});
assert(f==poly({1,2,1})); // f(X)=(X+1)^2
```

13. 声明 / 调用形式

```
mint; mint::pow(ll k); mint::inv(); x; cin>>a; cout<<a
```

参数、效果与返回：内置 `mint`，不再粘贴其他模整数文件。构造、四则/复合四则、正负号、前置增减、`==/!=`、`bool/!`、输入输出的用法同模整数；负指数要求可逆。这里未提供后置 `++/--`。

复杂度：基本运算 $O(1)$ ，逆元/幂 $O(\log \text{参数})$ 。

14. 声明 / 调用形式

```
void init(int n=0); mint C(int k,int n); mint binom(int n,int k); fac; ifac; inv
```

参数、效果与返回：预处理阶乘等表， $C(k,n)$ 与 $\text{binom}(n,k)$ 返回 n 选 k ，越界 k 返回 0； C 会扩表。直接读表前先 `init`。

复杂度：扩表 $O(n)$ ，表就绪查询 $O(1)$ 。

15. 声明 / 调用形式

```
void fft(mint *a,int n); void invFft(mint *a,int n); void fft(poly &a); void invFft(poly &a)
```

参数、效果与返回：原地 NTT/逆 NTT， n 正且为 2 次幂，频域按位逆序，逆变换已经乘 n 的逆元。使用对齐的内部缓冲区，调用者不需要自定义 allocator。内部保留 $O(N)$ 的逆元等缓存， N 为调用过的最大精度。vector 重载用 `size`；不自动补零。通常直接用卷积，无需手调底层变换。

复杂度： $O(n \log n)$ 。

16. 声明 / 调用形式

```
poly Sqrt(poly f)
```

参数、效果与返回：仅支持 $f[0]=1$ ，返回同长度且常数项为 1 的形式平方根；不是任意模平方根。

复杂度： $O(n \log n)$ 。

17. 声明 / 调用形式

```
void Div(poly f,poly g,poly &q,poly &r)
```

参数、效果与返回：多项式长除， g 非空且末项非零；写出商 q 、余式 r ，满足 $f=q*g+r$ ，余式次数低于 g 。与运算符 $/$ 完全不同。

复杂度： $O(N \log N)$ 。

18. 声明 / 调用形式

```
poly Eval(poly f,poly xs)
```

参数、效果与返回：批量求 $f(xs[i])$ ，返回与 xs 等长，顺序不变，允许重复点。

复杂度：时间 $O(M(m) + M(n) \log(n+1))$ ； $m=f.size()$ ， $n=xs.size()$ ， $M(k)$ 为长度 k 的多项式乘法代价。使用根部求逆与转置乘积；极长输入回退到余数树以保留变换长度上限。

19. 声明 / 调用形式

```
poly to_ex(poly f)
```

参数、效果与返回：返回 $f(e^x)$ 的前 n 项， $n=f.size()$ ；第 k 项为 $\sum_j f[j]*j^k/k!$ 。这不是 $\text{Exp}(f)$ ，不是 $e^{\{f(x)\}}$ 。

复杂度： $O(n \log^2(n+1))$ 。

20. 声明 / 调用形式

```
poly S2line(int n)
```

参数、效果与返回：返回第二类 Stirling 数 $S(n,0..n)$ ，长度 $n+1$ ； $n \geq 0$ 且阶乘表范围有效。

复杂度： $O(n \log(n+1))$ 。

注意事项

必须先包含 `<immintrin.h>`，运行机器必须支持 AVX2；不支持时选普通 `ntt.cpp`。仅卷积自动检测 AVX-512F；AVX2 仍是最低运行条件，不提供纯标量回退。GCC `target/optimize` 选项只作用于内部内核，不会关闭 `assert`。AVX-512 路径已通过 C++17/20 编译和标量指令模拟对拍，当前测试机器无 AVX-512，尚无该路径的真实硬件性能数据。Inv、Exp、Sqrt 的输入长度可达 2^{23} ；Ln 及其他接口仍按各自的中间卷积检查长度。Div 的 q 、 r 必须是不同对象。精度由输入 vector 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i]=\sum_j f[i+j]*g[j]$ ，不是多项式除法。形式幂级数相除要 $f*\text{Inv}(g)$ 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。不支持任意常数项的模平方根。长除前清除除数尾部多余的零。lagrange 要求互异取值点，允许零点。NTT 上限为 2^{23} 。

固定点的重复求值与插值

```
Poly_Tree t;           // Empty tree.
t.set(xs);             // Replace points and clear cached inverse.
Poly_Tree t2(xs);      // Same as set(xs).
auto ys=t.eval(f);     // Also available as Eval(f,xs).
auto a=t.interpolate(ys); // Also available as lagrange(xs,ys).
t.set();               // Clear points and cache.
```

参数、效果与返回： xs 为 n 个取值点。构造/set 建立共享乘积树，只存 $2n-1$ 个节点； n 为零时不建树。eval 返回与 xs 等长的点值，允许重复点；空多项式在各点的值均为零。interpolate 要求 ys 长度为 n ，返回 n 个系数，点差必须可逆；重复点不允许用于插值。

同一批 xs 反复求值时，直接保留对象调用 eval，复用乘积树与根部逆多项式；传入更长的 f 时会扩充逆多项式缓存。单次调用仍可用 `Eval(f,xs)` 或 `lagrange(xs,ys)`，无需修改旧代码。底层模数改变后必须重新 set，旧的点值与缓存都不能继续使用。

复杂度：建树、插值均为 $O(n \log^2 n)$ 。eval 时间为 $O(M(m) + M(n) \log(n+1))$ ， $m=f.size()$ ； $M(k)$ 表示多项式乘法代价。树占用 $O(n \log n)$ 空间，另缓存已使用的根部逆多项式。小规模直接求值；超出根部相关乘积长度限制时采用余数树回退。

9.6 polynomial/ntt.cpp

带长除、多点求值的多项式

用途

同样固定模数 998244353，提供常用幂级数和 Sqrt、Div、Eval 等接口。

最小示例

```
poly f{1,1,0,0};
auto g=Inv(f); auto h=f*g; h.resize(4);
assert(h[0].x==1 && !h[1] && !h[2] && !h[3]);
auto e=Exp(Ln(f));
for(int i=0;i<4;i++)assert(e[i].x==f[i].x);
assert(FSPE({0,1},{1,-1,-1},10).x==55);
poly q,r; Div({1,2,1},{1,1},q,r);
assert(q.size()==2 && q[0].x==1 && q[1].x==1);
auto v=Eval({1,2,1},{0,1,2});
assert(v[0].x==1 && v[1].x==4 && v[2].x==9);
```

接口参考

1. 声明 / 调用形式

```
MOD=998244353; FFT_MAX=23
```

参数、效果与返回：固定模数；实际变换长度必须是 $\leq 2^{23}$ 的 2 次幂，FPS 的中间卷积也算在内。不要只根据最终返回长度判断可用范围。

复杂度：不适用。

2. 声明 / 调用形式

```
poly; f[i]; f.resize(n)
```

参数、效果与返回：系数类型 mint，0 起下标就是次数；本文件只有小写 poly 类型名。FPS 截断精度由输入长度决定，resize(n) 补高位零后再调用；不要提前去掉要保留的零。

复杂度：resize 与新增项数相关。

3. 声明 / 调用形式

```
poly operator*(poly f,poly g); f+g; f-g; f+=g; f-=g
```

参数、效果与返回：* 为卷积；+/- 按次数相加减并扩展至较长长度；复合加减原地修改。空输入的卷积为空。

复杂度：乘 $O(N \log N)$ ，加减 $O(n+m)$ 。

4. 声明 / 调用形式

```
poly operator*(poly f,mint x); poly operator*(mint x,poly f); poly operator/(poly f,poly g)
```

参数、效果与返回：数乘逐项乘；f/g 是相关运算，结果第 i 项 $= \sum_j f[i+j]*g[j]$ ，不是多项式长除或 FPS 相除。

复杂度：数乘 $O(n)$ ，相关 $O(N \log N)$ 。

5. 声明 / 调用形式

```
mint value(const poly &f,mint x)
```

参数、效果与返回：Horner 法求 $f(x)$ ，空多项式返回 0；不修改 f。

复杂度： $O(n)$ 。

6. 声明 / 调用形式

```
poly Inv(poly f)
```

参数、效果与返回：返回长度 n 的 $1/f \bmod x^n$ ；要求非空且 $f[0]$ 可逆。FPS 相除用 $f*Inv(g)$ 后 `resize` 到目标长度。

复杂度： $O(n \log n)$ 。

7. 声明 / 调用形式

```
poly diff(poly f); poly integ(poly f)
```

参数、效果与返回：求导返回 $n-1$ 项（空输入返回空）；积分返回 $n+1$ 项，常数为零。`integ` 要求 $1..n$ 的逆元存在。

复杂度： $O(n)$ ，积分可能扩展逆元表。

8. 声明 / 调用形式

```
poly Ln(poly f); poly Exp(poly f)
```

参数、效果与返回：`Ln` 要求 $f[0]=1$ ，`Exp` 要求 $f[0]=0$ ；输入非空，返回同长度。计算涉及的整数分母均须可逆。

复杂度： $O(n \log n)$ 。

多项式复合与复合逆

```
poly Compose(poly f, const poly &g, int n=-1);
poly Compose_Inv(poly f, int n=-1);
auto h=Compose({1,2,3},{4,1},4); // {57,26,3,0}
auto r=Compose_Inv({0,1,1},5); // {0,1,-1,2,-5}
// Compose({0,1,1},r,5) == poly({0,1,0,0,0})
```

Compose：返回 $f(g(x)) \bmod x^n$ 的 n 个系数， n 默认取 `f.size()`。允许 $g[0]$ 非零，空 g 按零多项式处理。 $n=0$ 返回空，空 f 返回 n 个零。 $g[0]=0$ 时自动忽略 f 的 n 次及以上项；否则完整保留 f 的系数。显式 n 不能改变输入 f 所表示的多项式，缺少的高次系数均视为零。

Compose_Inv：返回 $f(g(x)) = x \bmod x^n$ 且 $g[0]=0$ 的 g ， n 默认取 `f.size()`； $n=0$ 返回空， $n=1$ 返回 $\{0\}$ 。要求 $f[0]=0$ ， $n \geq 2$ 时还要求 $f[1]$ 可逆，计算中的整数 $1, \dots, n-1$ 也须可逆。只读取 f 的前 n 项，缺少的系数补零。

规模与复杂度：取 $k = \max(n, |f|)$ ， $g[0]=0$ 时先截断 f 。本文件要求 $k \leq 2^{21}$ ，复合逆要求 $n \leq 2^{21}$ 。复合时间 $O(k \log^2 k)$ 、空间 $O(k \log k)$ ；复合逆时间 $O(n \log^2 n)$ 、空间 $O(n)$ 。NTT 版本使用成对频域乘积，避免每层重复进行完整卷积。

9. 声明 / 调用形式

```
poly BM(const poly &a)
```

参数、效果与返回：返回 $C[0]=1$ 的最短递推分母，阶数 $d=C.size()-1$ ，满足 $a_t + \sum_{i=1}^d C_i a_{t-i} = 0$ ($t \geq d$)。实际递推系数是 $-C[i]$ ；只保证拟合已给样本。空序列与全零序列返回 $\{1\}$ 。末尾零系数也表示阶数，不能删去。长序列使用 Half-GCD，短序列直接 BM。要求在域上计算，任意模数版本请使用质数模数。样本不足时最短递推可能不唯一，返回系数可能与旧版不同，但阶数最短且满足全部已有样本。

复杂度：时间 $O(n \log^2 n)$ ，额外空间 $O(n)$ 。NTT 版本合并矩阵运算中的变换，复用频域结果。

10. 声明 / 调用形式

```
mint FSPE(poly F, poly G, ll k)
```

参数、效果与返回：返回 $[x^k]F/G$ ， $k \geq 0$ ， G 非空、 $G[0]$ 可逆； F 可为空，返回 0。

复杂度： $O(d \log(d+1) \log(k+1))$ 。

11. 声明 / 调用形式

```
mint RSPE(poly samples,ll k)
```

参数、效果与返回：推断递推后求 0 起第 k 项，要求 $k \geq 0$ ； k 在输入范围内时直接返回。真实递推阶数 d 时通常至少给 $2d$ 项，不能任意外推。分子由截断卷积构造，不再使用二重循环。

复杂度：时间 $O(n \log^2 n + d \log(d+1) \log(k+1))$ 。

12. 声明 / 调用形式

```
poly lagrange(vector<mint> xs,vector<mint> ys)
```

参数、效果与返回：等长 n 项，输入任意点值 (x_i, y_i) ，返回长度恰为 n 的系数数组 f ，满足 $f(x_i) = y_i$ 且次数不超过 $n-1$ ，末尾零系数保留。 xs 不要求连续，但模意义下须两两不同；合数模数下还要求所有差值可逆。空输入返回空。要求 $n < 2^{23}$ 。

复杂度：时间 $O(n \log^2 n)$ ，空间 $O(n \log n)$ 。乘积树复用于导数的多点求值和插值合并，权重用一次求逆批量计算；沿用所选模板的卷积内核。

```
poly f=lagrange({2,5,9},{9,36,100});
assert(f==poly({1,2,1})); // f(X)=(X+1)^2
```

13. 声明 / 调用形式

```
mint; mint::pow(ll k); mint::inv(); x; cin>>a; cout<<a
```

参数、效果与返回：内置 `mint`，不再粘贴其他模整数文件。构造、四则/复合四则、正负号、前置增减、`==`/`!=`、`bool`/`!`、输入输出的用法同模整数；负指数要求可逆。这里未提供后置 `++`/`--`。

复杂度：基本运算 $O(1)$ ，逆元/幂 $O(\log \text{参数})$ 。

14. 声明 / 调用形式

```
void init(int n=0); mint C(int k,int n); mint binom(int n,int k); fac; ifac; inv
```

参数、效果与返回：预处理阶乘等表， $C(k,n)$ 与 $\text{binom}(n,k)$ 返回 n 选 k ，越界 k 返回 0； C 会扩表。直接读表前先 `init`。

复杂度：扩表 $O(n)$ ，表就绪查询 $O(1)$ 。

15. 声明 / 调用形式

```
void fft(mint *a,int n); void invFft(mint *a,int n); void fft(poly &a); void invFft(poly &a)
```

参数、效果与返回：原地 NTT/逆 NTT， n 正且为 2 次幂，逆变换已经乘 n 的逆元。`vector` 重载用 `size`；不自动补零。通常直接用卷积，无需手调底层变换。

复杂度： $O(n \log n)$ 。

16. 声明 / 调用形式

```
poly Sqrt(poly f)
```

参数、效果与返回：仅支持 $f[0]=1$ ，返回同长度且常数项为 1 的形式平方根；不是任意模平方根。

复杂度： $O(n \log n)$ 。

17. 声明 / 调用形式

```
void Div(poly f,poly g,poly &q,poly &r)
```

参数、效果与返回：多项式长除， g 非空且末项非零；写出商 q 、余式 r ，满足 $f=q*g+r$ ，余式次数低于 g 。与运算符 `/` 完全不同。

复杂度： $O(N \log N)$ 。

18. 声明 / 调用形式

```
poly Eval(poly f, poly xs)
```

参数、效果与返回：批量求 $f(xs[i])$ ，返回与 xs 等长，顺序不变，允许重复点。

复杂度：时间 $O(M(m) + M(n) \log(n+1))$ ； $m=f.size()$ ， $n=xs.size()$ ， $M(k)$ 为长度 k 的多项式乘法代价。使用根部求逆与转置乘积；极长输入回退到余数树以保留变换长度上限。

19. 声明 / 调用形式

```
poly to_ex(poly f)
```

参数、效果与返回：返回 $f(e^x)$ 的前 n 项， $n=f.size()$ ；第 k 项为 $\sum_j f[j] j^k/k!$ 。这不是 $\text{Exp}(f)$ ，不是 $e^{\{f(x)\}}$ 。

复杂度： $O(n \log^2(n+1))$ 。

20. 声明 / 调用形式

```
poly S2line(int n)
```

参数、效果与返回：返回第二类 Stirling 数 $S(n, 0..n)$ ，长度 $n+1$ ； $n \geq 0$ 且阶乘表范围有效。

复杂度： $O(n \log(n+1))$ 。

注意事项

精度由输入 vector 长度决定，想求前 n 项就先 `resize(n)`，不要删掉需要保留的高位零。`operator/` 是相关运算 $(f/g)[i] = \sum_j f[i+j] * g[j]$ ，不是多项式除法。形式幂级数相除要 $f * \text{Inv}(g)$ 后截断。各多项式文件均内置模数类型和全局表，只选一份，不要再粘贴 `ModInt`。不支持任意常数项的模平方根。长除前清除除数尾部多余的零。`lagrange` 要求互异取值点，允许零点。NTT 上限为 2^{23} 。

固定点的重复求值与插值

```
Poly_Tree t;           // Empty tree.
t.set(xs);             // Replace points and clear cached inverse.
Poly_Tree t2(xs);      // Same as set(xs).
auto ys=t.eval(f);     // Also available as Eval(f,xs).
auto a=t.interpolate(ys); // Also available as lagrange(xs,ys).
t.set();               // Clear points and cache.
```

参数、效果与返回： xs 为 n 个取值点。构造/set 建立共享乘积树，只存 $2n-1$ 个节点； n 为零时不建树。`eval` 返回与 xs 等长的点值，允许重复点；空多项式在各点的值均为零。`interpolate` 要求 ys 长度为 n ，返回 n 个系数，点差必须可逆；重复点不允许用于插值。

同一批 xs 反复求值时，直接保留对象调用 `eval`，复用乘积树与根部逆多项式；传入更长的 f 时会扩充逆多项式缓存。单次调用仍可用 `Eval(f,xs)` 或 `lagrange(xs,ys)`，无需修改旧代码。底层模数改变后必须重新 `set`，旧的点值与缓存都不能继续使用。

复杂度：建树、插值均为 $O(n \log^2 n)$ 。`eval` 时间为 $O(M(m) + M(n) \log(n+1))$ ， $m=f.size()$ ； $M(k)$ 表示多项式乘法代价。树占用 $O(n \log n)$ 空间，另缓存已使用的根部逆多项式。小规模直接求值；超出根部相关乘积长度限制时采用余数树回退。

10 计算几何 / geometry

10.1 geometry/geometry-2d.cpp

整数二维几何基础

用途

点坐标和叉积均为 ll，提供凸包、面积与基本相交判断；需要另定义 ld=long double。

最小示例

```
vector<Point> a{{0,0},{2,0},{2,2},{0,2}};
auto h=convex(a); assert(convec_area(h)==8);
assert(on_segment(Point(1,0),Point(0,0),Point(2,0)));
```

接口参考

1. 声明 / 调用形式

```
Point(ll x=0,ll y=0); x; y; cin>>P; cout<<P; void output(string tag="")
```

参数、效果与返回：整数点，流读写两个坐标；output 使用 cerr 调试。坐标及点积、叉积等中间结果须在 ll 范围内。另需 using ld=long double。

复杂度：O(1)。

2. 声明 / 调用形式

```
P+Q; P-Q; P+=Q; P-=Q; P*k; k*P; P*=k
```

参数、效果与返回：向量加减及整数数乘，返回点；复合运算修改点。没有点除法与一元负号，取负用 -1*P。

复杂度：O(1)。

3. 声明 / 调用形式

```
P*Q; P^Q; P==Q; P!=Q; P<Q
```

参数、效果与返回：点乘点的 * 是叉积，^ 才是点积；< 按 x 再 y 排序，不是极角序。

复杂度：O(1)。

4. 声明 / 调用形式

```
double Point::arg(); int sgn(ll x); int sgn(const Point &P); bool cmp(const Point &P,const Point &Q)
```

参数、效果与返回：arg 为 atan2(y,x) 弧度；sgn(ll) 返回 -1/0/1。sgn(Point) 是极角分区辅助，cmp 是极角比较；极角排序应排除零向量。

复杂度：O(1)。

5. 声明 / 调用形式

```
ll area(const Point &A,const Point &B,const Point &C); ld dist(const Point &P); ll dist2(const Point &P)
```

参数、效果与返回：area 返回三角形面积的两倍且非负；dist/dist2 是向量长度及平方，两点距离用 dist(A-B)。

复杂度：O(1)。

6. 声明 / 调用形式

```
int dir(const Point &A,const Point &B,const Point &C); int pre(int i,int n); int suf(int i,int n)
```

参数、效果与返回: dir 是连续向量 AB 与 BC 的叉积符号, 正为左转, 负为右转, 零共线。pre/suf 是 0..n-1 环形下标前后继。

复杂度: $O(1)$ 。

7. 声明 / 调用形式

```
vector<Point> convex(vector<Point> a); ll convec_area(const vector<Point> &h)
```

参数、效果与返回: convex 去重并返回凸包边界, 去掉边上的共线中间点; 不修改原 a。convec_area 返回按环序凸多边形面积两倍, 勿传任意乱序点集。

复杂度: 凸包 $O(n \log n)$, 面积 $O(n)$ 。

8. 声明 / 调用形式

```
bool on_segment(X,A,B); bool strict_intersect(A,B,C,D); bool nonstrict_intersect(A,B,C,D)
```

参数、效果与返回: 判断 X 在闭线段 AB 上; strict 是线段内部真交, 不含端点/共线; nonstrict 包含接触、共线重叠。参数均为 Point。

复杂度: $O(1)$ 。

9. 声明 / 调用形式

```
bool parallel(Point A,Point B,Point C,Point D)
```

参数、效果与返回: 判断 AB 与 CD 平行且不共线; 共线返回 false。不是单纯方向叉积为零的判定, 使用非退化直线。

复杂度: $O(1)$ 。

10. 声明 / 调用形式

```
bool ray_intersect(Point A,Point B,Point C,Point D)
```

参数、效果与返回: 判断从 A 沿 AB、从 C 沿 CD 的闭射线相交, 含起点; 方向为零按单点处理。

复杂度: $O(1)$ 。

11. 声明 / 调用形式

```
bool in_triangle(P,A,B,C)
```

参数、效果与返回: P 是否在闭三角形内, 含边界; 退化三角形按三条闭边的并处理。

复杂度: $O(1)$ 。

12. 声明 / 调用形式

```
int convex_min(int n,F val)
```

参数、效果与返回: 在满足凸多边形线性目标对应环形单峰结构的 val(0..n-1) 上找一个最小值下标, $n>0$ 。不是任意数组求最小值函数。

复杂度: $O(\log n)$ 次 val 调用。

13. 声明 / 调用形式

```
ll min_cross(const vector<Point> &h, Point X); ld min_cross(const vector<Point> &h, ld x, ld y)
```

参数、效果与返回：h 为非空、按环序且无共线中间点的凸包；第一重载最小化 $h[i] \cdot X$ （叉积），第二重载最小化 $h[i].x \cdot x + h[i].y \cdot y$ （点积）。名称相同但目标不同。

复杂度： $O(\log n)$ 。

注意事项

convex 自动排序去重。min_cross 要求顶点按凸包环序排列且没有共线中间点，支持旋转起点、顺逆时针， $O(\log n)$ 。射线包含端点，零方向按单点处理；退化三角形按三条边处理。坐标叉积、点积仍须保证不溢出 ll。Point 输出两个坐标，中间有空格；output 用于调试。